arXiv:2604.12151v2 [cs.LG] 6 Nov 2026

Distinct mechanisms underlying in-context learning in transformers

Cole Gibson,[1,2,*] Wenping Cui,[2,*] and Gautam Reddy[2,†]

[1] *Lewis-Sigler Institute for Integrative Genomics, Princeton University, Princeton, NJ 08544, USA*
[2] *Department of Physics, Princeton University, Princeton, NJ 08544, USA*

Modern distributed networks, notably transformers, acquire a remarkable ability (termed 'in-context learning') to adapt their computation to input statistics, such that a fixed network can be applied to data from a broad range of systems. Here, we provide a mechanistic characterization of this behavior in transformers trained on a finite set $\mathcal{S}$ of discrete Markov chains. The transformer displays four algorithmic phases, characterized by whether the network memorizes or generalizes, and whether it uses 1-point or 2-point statistics. We show that the four phases are implemented by multi-layer subcircuits that exemplify two qualitatively distinct mechanisms for implementing context-adaptive computations. Minimal models isolate the key features of both motifs. Memorization and generalization phases are delineated by two boundaries that depend on data diversity, $K = |\mathcal{S}|$. The first (K_1^*) is set by a kinetic competition between subcircuits and the second (K_2^*) is set by a representational bottleneck. A symmetry-constrained theory of a transformer's training dynamics explains the sharp transition from 1-point to 2-point generalization and identifies key features of the loss landscape that allow the network to generalize. Put together, we show that transformers develop distinct subcircuits to implement in-context learning and identify conditions that favor certain mechanisms over others.

I. INTRODUCTION

Traditionally, learning involves tuning parameters so that a learning system encodes statistical regularities of a particular dataset. A recurrent network trained on trajectories from one dynamical system, for example, internalizes that system's correlations and then extrapolates using these statistics. Presenting it with trajectories from a different system typically degrades the accuracy with which it can predict future states. Modern machine learning systems, most notably transformers [1], display an alternative mode of adaptation once trained on a large dataset. When given a partial sequence or a short list of example input-output pairs, a network is often able to infer the rule that generated the data and apply it to a new input without additional tuning of its parameters. That is, the burden of learning shifts from slow and often expensive parameter updates to a fast computation implemented by network dynamics. This capacity to generalize has been termed 'in-context learning (ICL)' [2–10]. While this feature was first highlighted in natural language processing [2], similar behavior has been recapitulated across diverse paradigms, including nonlinear regression, small tabular datasets, zero-shot forecasting of chaotic dynamical systems, imitation learning and control systems [3, 11–20], in turn motivating the development of so-called foundation models across many domains of science [21–27].

A natural question is if there are particular features of data or network architecture that favor such context-adaptive computations. Previous work has shown that inducing this ability depends crucially on the diversity and scale of data the model encounters during training [4, 6, 7, 28–34]. It is not known what sets this scale, though two distinct hypotheses have been proposed [30, 33].

Why might transformers, in particular, support this behavior? A transformer operates in parallel on an input sequence of vectors [1]. The attention operation mediates interactions by comparing each vector to the rest of the vectors in the sequence. These comparisons let the network retrieve relevant vectors from the context. Retrieved vectors are subsequently acted upon by a nonlinear operation parameterized as a multi-layer perceptron (MLP). One hypothesis is that the transformer leverages retrieved information in earlier layers of the network to adapt its computations in later layers based on sequence-level statistics, but it is not known how different transformer elements interact to enable this behavior.

II. BACKGROUND

We consider a setting that has been empirically shown to capture many key features of ICL observed across synthetic data and natural language paradigms [4, 6, 28, 31, 32, 35] including induction heads [5, 7, 36], learning kinetics [30], and transient generalization [37]. In this setting [31], the transformer is provided an input sequence of states sampled from a finite set $\mathcal{S}$ of K stationary Markov chains, where each chain is over C discrete states. The network must then output a predictive distribution over the next state. The K transition matrices are drawn once from a symmetric Dirichlet ensemble $\mathcal{D}_T$ and frozen thereafter (Figure 1a) (further details in Appendix II 1). K serves as a quantitative measure of data diversity.

Since K is finite, it is possible for the network to develop distinct generalization and memorization strategies depending on K. In one case, the network may predict the next state by simply reproducing the empirical statistics of the presented sequence. This strategy would al-

* Authors contributed equally
† greddy@princeton.edu

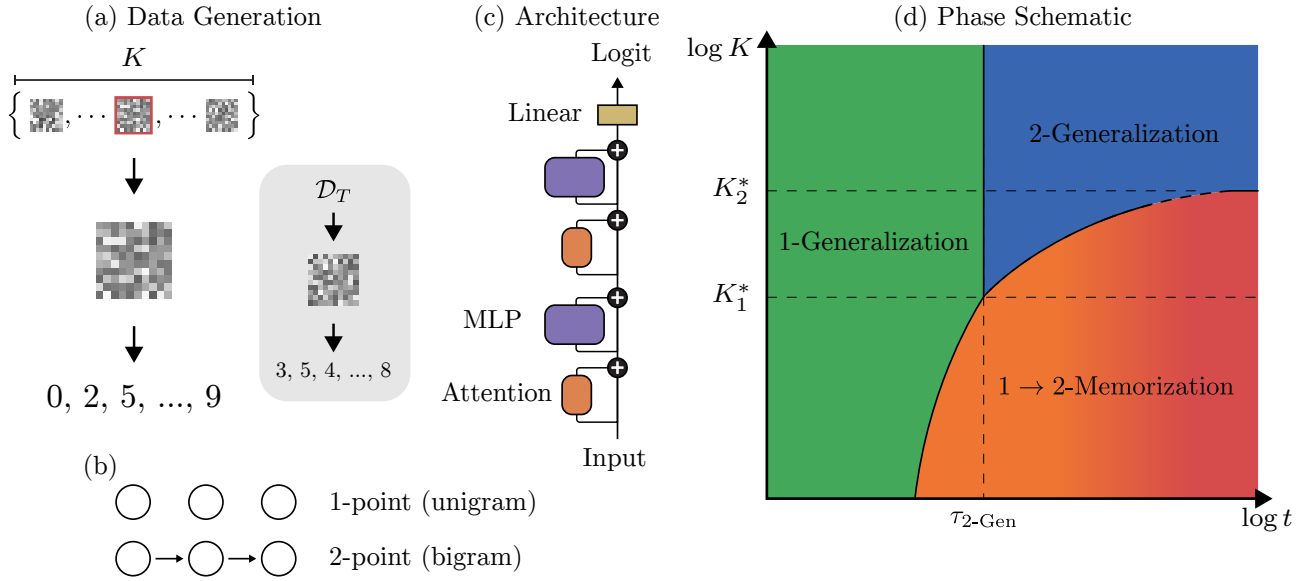

FIG. 1: (a) Data generating process for training and generalization sequences. Training sequences are generated from transition matrices drawn from the K matrices in $\mathcal{S}$ sampled prior to training. Generalization sequences are generated from transition matrices drawn directly from $\mathcal{D}_T$. (b) An illustration of 1-point (unigram) statistics and 2-point (bigram) directed, nearest-neighbor statistics. (c) The 2-layer transformer architecture, which contains one attention block and one multi-layer perceptron (MLP) block in each layer. After the operation of two layers, the linear block maps the embedded representation to a discrete distribution over states that is then normalized to form the logit. (d) Schematic of the four algorithmic phases as the number of chains (K) and training time (t) are varied. Phases are consistent with either memorizing or generalizing predictors, which use either 1-point or 2-point statistics, denoted by 1- and 2- prefixes respectively. Orange to red gradient in memorization phase indicates a gradual transition from M_1 to M_2 in contrast to the abrupt transition from G_1 to G_2. Three quantities characterize the transition between the different phases: (i) $\tau_{2\text{-Gen}}$, which is the training time to transition from G_1 to G_2 given $K > K_1^*$, (ii) K_1^*, which together with $\tau_{2\text{-Gen}}$ defines the 'triple point' between G_1, G_2 and M_1, and (iii) K_2^*, which delineates G_2 and M_2. The dashed phase boundary indicates an extrapolation as $t \to \infty$.

low the network to generalize to sequences drawn from Markov chains not in $\mathcal{S}$. Because Markov chains are characterized by transition statistics between neighboring states (Figure 1b), the network does not benefit from using correlations of order greater than two. We term the two such generalizing strategies that use 1-point or 2-point information as 1-Gen and 2-Gen respectively, and the corresponding phases G_1 and G_2.

Alternatively, the network may use the sequence context to identify which of the K chains the sequence is being sampled from and retrieve the corresponding transition probabilities, even if the network never encounters the same sequence during training. Identification is more sample efficient than generalization (i.e., needs a shorter sequence to predict well), but requires the network to encode each of the K transition matrices in its parameters. The network may use 1-point or 2-point statistics (and possibly intermediate strategies) to identify the chain. We term these two memorizing strategies 1-Mem and 2-Mem respectively, and the corresponding phases M_1 and M_2.

We consider a two-layer transformer with one attention block and one MLP block per layer followed by a linear readout (Figure 1c, further details in Appendix II 2). This is the simplest network architecture that has been empirically shown to implement all four strategies across different K [31]. An attention head allows a state at one position in the sequence to retrieve information about other states (known as values) either based on their position or their content (by implementing a key-query dot product). An MLP is a feedforward neural network that processes the information produced by earlier blocks. Information from attention heads and MLPs is written into a residual stream (vertical path from input to logit in Figure 1c), which tracks the D-dimensional representation of a state as it evolves over the network's layers due to the attention and MLP operations. We fix $C = 10$ and $D = 64$ (refer to Appendix I for a complete summary of values). The network is trained autoregressively to predict the next state s_{n+1} after observing a sequence of discrete states $s_0, s_1, \ldots, s_n$ up to a maximum sequence length N.

We now summarize known phenomenology for two-layer transformers trained on different numbers of chains K over the course of training (illustrated in Figure 1d) [31].

Soon after training begins, the network learns to implement 1-Gen (phase G_1, green region in Figure 1d). When K is below a certain threshold value (denoted K_1^*), the network transitions from G_1 to the memorization phases, first learning 1-Mem in phase M_1 and then gradually implementing 2-Mem in phase M_2 (orange-red regions in Figure 1d). In both M_1 and M_2, the network does not generalize to sequences sampled from a chain not in $\mathcal{S}$. When $K > K_1^*$, the network follows a different trajectory and abruptly transitions from G_1 by rapidly learning 2-Gen (phase G_2, blue region in Figure 1d). In this phase, the network optimally generalizes to sequences from unseen chains. The number of iterations after which the G_2 generalization phase appears, $\tau_{\text{2-Gen}}$, is independent of K provided $K > K_1^*$.

III. SUMMARY OF CONTRIBUTIONS

We present four main results that together identify the circuits that implement each of the aforementioned phases and the factors that govern the transitions between them.

First, we use a circuit tracing technique to identify the four subcircuits. Notably, 2-Mem is implemented by a novel encoder-pool-decoder subcircuit, which builds a latent representation of a chain and uses this representation to modulate network response in later layers. This circuit motif requires both pairs of attention heads and MLPs, which explains why a two-layer transformer is the minimal transformer necessary to recapitulate distinct algorithmic regimes.

Second, we characterize the sharp transition from G_1 to G_2 (as $K \to \infty$). To do so, we introduce the symmetry-constrained attention-only transformer (the *SA-transformer*) architecture, which exploits permutation symmetry in task structure to significantly simplify the standard transformer. We analyze the SA-transformer to develop an effective theory for the learning dynamics of the statistical induction head underpinning G_2. The theory explains its abrupt formation, identifies two symmetry-breaking biases that guide the optimization dynamics, and predicts a scaling relationship between the formation time $\tau_{\text{2-Gen}}$ and the sequence length N which we verify empirically with a standard transformer.

Third, we characterize the memorization-generalization transition at $K = K_1^*$ as resulting from a kinetic competition between subcircuits.

Finally, we provide evidence of a second threshold, K_2^*, and suggest its mechanistic origin. For $K_1^* < K < K_2^*$, the interval $\Delta\tau_K = \tau_{\text{2-Mem}} - \tau_{\text{2-Gen}}$ between the initial appearance of G_2 until it disappears in favor of M_2 is observed to scale as a power law, $\Delta\tau_K \sim (K_2^* - K)^{-\gamma}$, with an exponent $\gamma \approx 2$. Thus, we hypothesize that this threshold marks the upper boundary of M_2: when $K_1^* < K < K_2^*$, the network eventually memorizes and transitions from G_2 to M_2 with sufficient training, whereas when $K > K_2^*$, the network remains in G_2 indefinitely (Figure 1d). The transient behavior of G_2 in the former case can then be interpreted as a form of overfitting.

An analysis of chain-level memorization suggests that the model cannot reliably encode and retrieve any of the K chains when $K > K_2^*$. Thus, we quantify how K_2^* depends on the expressivity of the encoder and decoder via a minimal model of the 2-Mem subcircuit. We show that this circuit motif is also capable of 2-Gen within specific architectural regimes, thereby demonstrating that transformers may in principle implement two distinct in-context mechanisms that achieve optimal generalization.

IV. RESULTS

A. Identifying in-context memorization and generalization regimes

We begin by reproducing the algorithmic phases reported in [31] and illustrated in Figure 1d. To determine which algorithm the network implements at a given training time (t) and number of chains in $\mathcal{S}$ (K), we compare its predictions to four Bayes predictors corresponding to the four strategies (Appendix III). The two generalizing predictors assume knowledge only of $\mathcal{D}_T$: $\hat{\pi}^{\text{1-Gen}}$ predicts from empirical 1-point frequencies, whereas $\hat{\pi}^{\text{2-Gen}}$ predicts from empirical nearest-neighbor 2-point frequencies. The two memorizing predictors instead assume knowledge of $\mathcal{S}$: $\hat{\pi}^{\text{1-Mem}}$ and $\hat{\pi}^{\text{2-Mem}}$ infer which of the K training chains generated the context using 1-point or 2-point statistics respectively, and predict the next state with the corresponding transition matrix. These four predictors isolate the two central choices faced by the network: memorization versus generalization, and 1-point versus 2-point statistics. Because the data are first-order Markovian, $\hat{\pi}^{\text{2-Gen}}$ is the optimal generalizing solution on $\mathcal{D}_T$, whereas $\hat{\pi}^{\text{2-Mem}}$ is the optimal memorizing solution on $\mathcal{S}$.

The network passes through discrete strategies during training, which manifest as plateaus in the training loss near the loss of certain predictors (Figure 2a, upper). At low K, the model quickly improves and, after a transient 1-Mem stage (left panel, brief flattening of black curve near orange line), converges to the optimal memorizing 2-Mem loss (convergence to red line). At high K, the model instead undergoes an abrupt drop to the loss incurred by 2-Gen (right panel, steep drop of black curve to blue line). Intermediate K displays both features, that is, the model can transiently generalize (middle panel, steep drop of black curve to blue line) and later continue lowering the training loss by overfitting to $\mathcal{S}$ (convergence towards red line). The generalization loss (Figure 2a, lower) displays behavior consistent with these interpretations.

We characterize these transitions in more detail by measuring the similarity between the model's output $\hat{\pi}_\theta$ and the expected output from each predictor $\hat{\pi}$. This

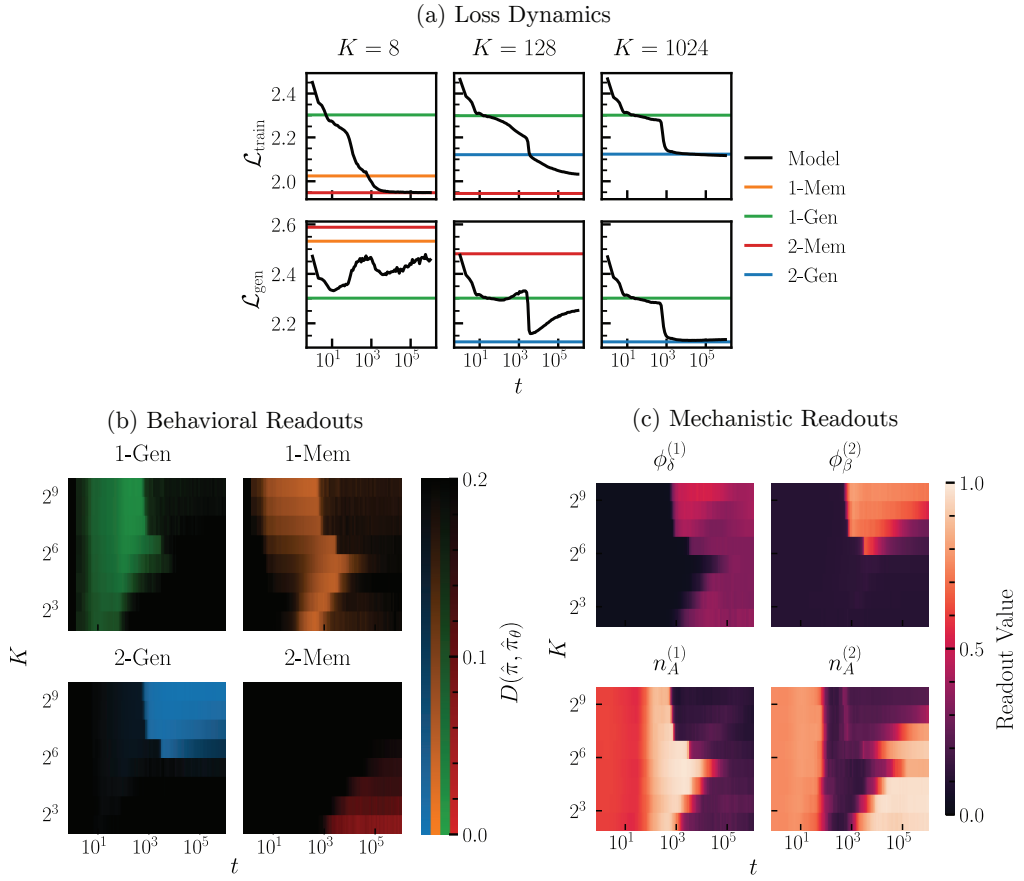

FIG. 2: (a) Training dynamics of the transformer training loss (top) and generalization loss (bottom) at low, medium, and high K. Notice the plateaus during training that occur near the predictor training and generalization losses. The model dynamics and predictor losses have been produced by evaluating model checkpoints and the predictors on a large batch of sequences. We have omitted for clarity the predictor losses near which each model does not exhibit plateaus. (b) Behavioral readouts of the transformer through training across a range of K. Color intensity is inversely proportional to the divergence (in nats) of the transformer predictions from the indicated predictor (Appendix IV 1). Simultaneous similarity of the model to both 1-Gen and 1-Mem predictors at large K results from the relatively high similarity of these predictors to the uniform predictor over states. (c) Mechanistic readouts and order parameters of the transformer through training across a range of K (Appendix IV 2 for details). Here, $n_A^{(1)}$ and $n_A^{(2)}$ have been normalized by the sequence length, $N = 256$. The correspondence between phases and mechanistic readouts are as follows: G_1 and M_1: high $n_A^{(1)}$ and low $\phi_\delta^{(1)}$; G_2: high $\phi_\delta^{(1)}$ and $\phi_\beta^{(2)}$; M_2: high $\phi_\delta^{(1)}$ and $n_A^{(2)}$.

similarity is measured using

$$D(\hat{\pi}, \hat{\pi}_\theta) \equiv \frac{1}{2}\Big\langle D_{\text{KL}}^{S_N}(\hat{\pi}\|\hat{\pi}_\theta)\Big\rangle_{\mathcal{S}} + \frac{1}{2}\Big\langle D_{\text{KL}}^{S_N}(\hat{\pi}\|\hat{\pi}_\theta)\Big\rangle_{\mathcal{D}_T}, \quad (1)$$

where $\langle\cdot\rangle_{\mathcal{S}}$ and $\langle\cdot\rangle_{\mathcal{D}_T}$ denote averages over sequences drawn from $\mathcal{S}$ and $\mathcal{D}_T$, respectively, and $D_{\text{KL}}^{S_N}$ is the autoregressive KL divergence averaged over sequence positions (definitions are given in Appendix IV). Figure 2b plots this divergence across t and K. Early in training, model output across essentially all K is closest to 1-Gen. For small K, the model's output is consistent with a transition to M_1, and then gradually to M_2. For sufficiently large K, the network bypasses memorization and transitions sharply into G_2. Between these limits is a window in which G_2 appears transiently before eventually drifting into M_2.

We next turn to four mechanistic readouts that probe the attention patterns within the transformer (Appendix IV 2). The first is the previous-state parameter $\phi_\delta^{(1)}$, which measures how much the first attention layer attends to the immediately preceding state. Because nearest-neighbor correlations are required for the two 2-point strategies, a rise in $\phi_\delta^{(1)}$ marks the onset of M_2 or G_2. The second is the induction head parameter $\phi_\beta^{(2)}$, which measures whether the second attention layer selectively attends to states that follow previous occurrences of the current state. This is the hallmark of a

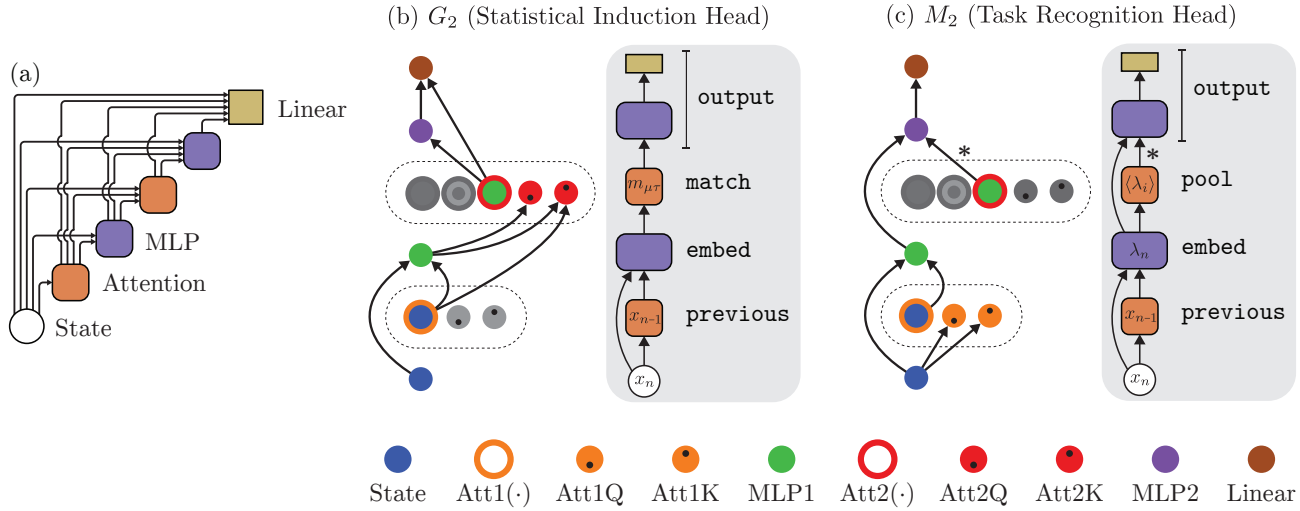

FIG. 3: Circuit tracing. (a) The transformer may be unrolled into a directed graph with each block taking input from the previous blocks. (b-c) Left: Visualization of the circuit traced for the phase. See Appendix V for the definitions of each node in the circuit and an interpretation of the connections. Any nodes without incoming and outgoing edges are depicted as desaturated. Right: Interpretative schematic of the circuit traced for the phase, with arrows indicating input-output relationships and layer function labeled. See the main text for more details. Isolated layers are omitted entirely. In panel (c), the asterisk $*$ denotes the task vector discussed in Section IV C.

statistical induction head [5, 7, 9, 36, 38] (described in further detail below) and specifically delineates G_2. The remaining two readouts, $n_A^{(1)}$ and $n_A^{(2)}$, are entropy-based measures of the effective number of sequence positions attended to by the two attention layers. Large $n_A^{(\ell)}$ indicates diffuse pooling over the sequence, whereas small $n_A^{(\ell)}$ indicates focused retrieval.

These mechanistic observables recapitulate the phase diagram (Figure 2c). In the 1-point phases, $\phi_\delta^{(1)}$ is small and $n_A^{(1)}$ is large, showing that the first layer broadly pools over the sequence rather than extracting local transitions. Entry into either 2-point phase is marked by a sharp increase in $\phi_\delta^{(1)}$ together with a collapse of $n_A^{(1)}$, indicating that the first layer extracts previous states. The split between the 2-point phases is then resolved in the second layer. In G_2, $\phi_\beta^{(2)}$ becomes large and $n_A^{(2)}$ decreases, whereas in M_2, $\phi_\beta^{(2)}$ remains small while $n_A^{(2)}$ stays large.

Taken together, Figure 2 shows that, as t and K are varied, the network traverses a small number of discrete algorithmic phases, each with a distinct predictive signature and a distinct internal mechanism. This correspondence provides the empirical basis for the circuit analyses that follow.

B. Tracing circuits

We next identify how the computation in each phase is implemented (see Appendix V for full details). Because every computational block in a transformer writes additively to the residual stream (Figure 1c), the two-layer transformer can be unrolled into a directed graph whose nodes are the state representation, the two attention blocks, the two MLP blocks, and the final linear readout (Figure 3a). For the attention blocks, we further separate the query, key and value operations. We then trace a circuit by caching the vector transmitted along each edge in an unperturbed forward pass, ablating one edge at a time by replacing its transmitted vector with its batch-mean value, and measuring the resulting change in the model prediction by the KL divergence from the unperturbed output. Minor alterations to these traced circuit prototypes are sufficient for fully circuit-constrained models to retain much of their performance. These results together with the readouts in Figure 2b-c allow us to identify four sparse circuits that emulate each of the four Bayes predictors. The operations implemented by these circuits are presented in Figure 3b-c for G_2 and M_2, and in Figure A5 for G_1 and M_1.

In G_1, the first attention layer pools almost uniformly over the sequence ($\phi_\delta^{(1)}$, $n_A^{(1)}$ in Figure 2c), producing a representation of the empirical 1-point frequencies, and MLP1 maps this representation directly to produce the logits (Figure A5a). The current state plays essentially no role, as expected for an algorithm that predicts based on the stationary distribution of a Markov chain (rather than its transition matrix). In M_1, the same pooled 1-point representation is computed ($\phi_\delta^{(1)}$, $n_A^{(1)}$ in Figure 2c) but it is now combined with the current state and decoded through MLP1 and MLP2 to retrieve a memorized transition matrix (Figure A5b).

In G_2, the first attention layer pays attention to the

previous state, so that the residual stream at each position carries information about neighboring pairs ($\phi_\delta^{(1)}$ in Figure 2c). Pair information is then presented to the second attention layer which implements a matching operation. This two-layer circuit, known as the induction head, has been well-characterized: the representation of the current state is used to match earlier occurrences of that state in the context, and the second attention layer reads out the states that followed those matches. Pooling over these matches yields an estimate of the empirical conditional distribution $\hat{P}(s_{n+1} = \tau \mid s_n = \mu)$, which is then mapped to the output logits. In our traced circuit, the relevant query, key, and value representations run primarily through MLP1 (Figure 3c) which is not necessary to implement an induction head, but the essential computation is preserved. A more detailed analysis of G_2 is presented further below. Next, we examine the M_2 circuit in greater detail, which involves a qualitatively different circuit motif.

C. The task recognition head and the formation of task vectors

In the M_2 circuit, the first layer again extracts the previous state (Figure 2c), but the second attention layer no longer performs content-based matching. Instead, it attends diffusely across the sequence and averages the 2-point embeddings produced by MLP1 (Figure 2c, Figure 3c). We infer that the M_2 circuit has an encoder-pool-decoder structure. Specifically, Att1 and MLP1 encode each neighboring pair (s_{i-1}, s_i) into a pair embedding $\lambda(s_{i-1}, s_i)$, Att2 pools these pair embeddings into a vector, and MLP2 decodes that vector together with the local state information carried by MLP1 to produce the next-state distribution,

$$\varphi_n = \frac{1}{n}\sum_{i=1}^{n} \lambda(s_{i-1}, s_i), \quad \hat{\pi}_n = \psi(s_n, \varphi_n), \tag{2}$$

where $\hat{\pi}_n$ is the predicted probability distribution over the next state, s_{n+1}.

This encoder-pool-decoder picture has an intuitive interpretation. For memorization, the network estimates a compact statistic φ_n that distinguishes among the finite set of chains stored in the weights. Averaging over pair embeddings is a natural way to build such a statistic. For this reason, we refer to φ_n as a *task vector* and M_2 circuit as a *task recognition head*, where a 'task' corresponds to prediction on sequences from one chain. The embedding λ must be nonlinear, as averaging any linear representation of neighboring pairs collapses to a function of 1-point frequencies and does not encode the 2-point information necessary to implement 2-Mem.

We perform three specific experiments to further support this interpretation (Appendix IX). First, task vectors from the same chain become increasingly separable through training: a nonlinear projection shows diffuse clouds early in training and well-separated task-specific clusters later on (Figure 4a). Second, we patch the Att2 output from one sequence into another sequence, i.e., the task vector φ_B from a sequence sampled from task B is patched into a sequence sampled from task A. The patch causes the model that sees the sequence from A to predict according to sequence B's transition matrix (Figure 4b). These patching results support the interpretation that MLP2 predicts using task-dependent information carried by the statistic φ together with current-state information supplied outside the averaging operation, consistent with equation 2. Third, the mutual information between φ and the latent task identity grows toward the channel capacity $\log_2 K$ during training (Figure 4c), indicating that the task vector φ is specifically constructed by earlier layers to discriminate tasks. This is corroborated by a linear decoding analysis which finds that task vectors represent the task posterior rather than directly encode 2-point statistics (Figure A15). More so, performing the patch experiment on the M_1 circuit (Figure A14) yields similar results to those seen for M_2, indicating that the task vector construction is used generically during memorization.

Put together, this analysis characterizes the circuit motifs that underpin each of the four phases. Next, we ex-

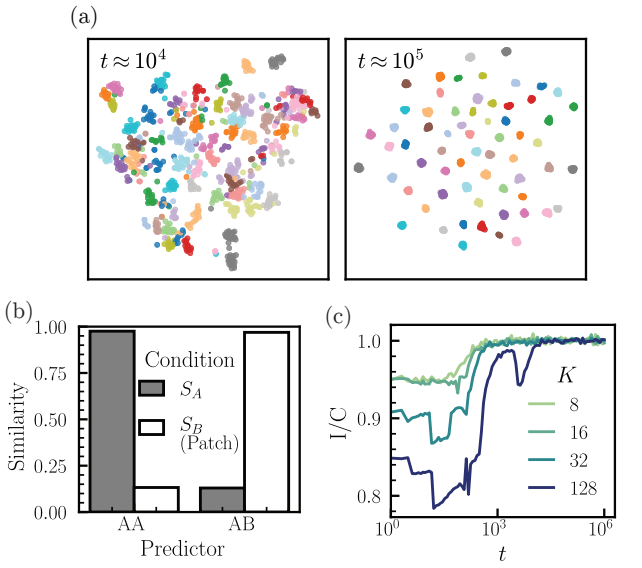

FIG. 4: (a) A 2-D t-SNE visualization of task vectors for the $K = 64$ model demonstrates increased separability through training. (b) Results of task vector patching experiment. Replacing the task vector formed on a sequence from transition matrix A with that formed on a sequence from transition matrix B results in the transformer predicting according to the transition matrix B. See Appendix IX 3 for details. (c) The information content of the task vectors is optimized through training. Mutual information is presented as a fraction of the channel capacity $\log_2 K$. See Appendix IX 5 for details.

amine the factors that determine the three boundaries delineating these phases (Figure 1d): (i) the vertical boundary from G_1 to G_2 for $K > K_1^*$ characterized by the transition time $\tau_{\text{2-Gen}}$, (ii) the 'triple point' between G_1, M_1 and G_2 set by K_1^* and $\tau_{\text{2-Gen}}$, and (iii) the boundary between G_2 and M_2 set by K_2^*.

D. The transition from G_1 to G_2

To develop a theory for the transition to G_2, we derive a simpler trainable network that exploits symmetries in $\mathcal{S}$ as $K \to \infty$ (so that $\mathcal{S}$ is equivalent to $\mathcal{D}_T$) and that could implement the statistical induction head isolated in the full transformer (see Appendix VII for full details). After a series of modifications, the content-dependent part of the key-query computation in an attention head, denoted $M^{(\ell)}$, is reduced to a block diagonal form. The key-query computation is then parameterized by one scalar $\beta_1^{(1)}$ in the first attention layer and four scalars $\beta_1^{(2)}, \beta_2^{(2)}, \beta_3^{(2)}, \beta_4^{(2)}$ in the second. The readout contributes three independent mixture weights. The resulting symmetry-constrained, attention-only transformer (*SA-transformer*) therefore has only eight independent parameters, in addition to learned relative positional biases $P^{(\ell)}$ for the two layers.

Since there are four paths that an input state can take through the computational graph of a two-layer attention-only transformer, this leads to four terms that contribute to the final prediction $\hat{\pi}$,

$$\hat{\pi} = w_A x_N + w_B \sum_{i \le N} A_{iN}^{(1)} x_i + w_C \sum_{i \le N} A_{iN}^{(2)} x_i + w_D \sum_{i \le N} \sum_{j \le i} A_{iN}^{(2)} A_{ji}^{(1)} x_j, \quad \text{where} \tag{3}$$

$$\log A_{ji}^{(1)} = \beta_1^{(1)} \delta_{s_i s_j} + P_{j-i}^{(1)} - \log Z^{(1)},$$

$$\log A_{ji}^{(2)} = \beta_1^{(2)} \delta_{s_i s_j} + \beta_2^{(2)} \sum_{k \le i} A_{ki}^{(1)} \delta_{s_k s_j} + \beta_3^{(2)} \sum_{k \le j} A_{kj}^{(1)} \delta_{s_i s_k} + \beta_4^{(2)} \sum_{k \le i,\, k' \le j} A_{ki}^{(1)} A_{k'j}^{(1)} \delta_{s_k s_{k'}} + P_{j-i}^{(2)} - \log Z^{(2)},$$

with $w_A, w_B, w_C, w_D \ge 0$, $w_A + w_B + w_C + w_D = 1$, $A_{ji}^{(\ell)}$ is the attention paid by state i to state j at layer ℓ, and the $Z^{(\ell)}$s are normalization constants. The weighted sum is interpreted as a mixture-of-experts operation, that is, each expert corresponds to a distinct path and its contribution to the final output is weighted by its ability (namely, the four w terms) to accurately predict the next state.

The SA-transformer is trained non-autoregressively on transition matrices drawn from $\mathcal{D}_T$ and sequences of length N. Figure 5b shows that the reduced model recapitulates the same phenomenology: it first reaches G_1, remains there for an extended time, and then undergoes a sharp drop to G_2. The parameter trajectories in Figure 5c show that this transition is structured. Specifically, the first-layer content matrix $M^{(1)}$, the second-layer positional bias $P^{(2)}$, and three of the four blocks of $M^{(2)}$ remain at zero (Figure A8). The dynamics are dominated by two parameters: the previous-state attention in the first layer, $\delta \equiv P_{-1}^{(1)}$, and the parameter that reads the content that was written to by the first attention head, $\beta \equiv \beta_3^{(2)}$ (which correspond precisely to the readouts $\phi_\delta^{(1)}$ and $\phi_\beta^{(2)}$ in Figure 2c). The attention heads therefore further reduce to

$$\log A_{ji}^{(1)} = \delta\, \delta_{j,i-1} - \log Z^{(1)},$$
$$\log A_{ji}^{(2)} = \beta \sum_{k \le j} A_{kj}^{(1)} \delta_{s_i s_k} - \log Z^{(2)}. \tag{4}$$

This form delineates the mechanism of the statistical induction head presented schematically in Figure 5d and described in detail in Appendix VI. When $\delta \to \infty$, the first layer's operation can be interpreted as a write operation that stores a state s_i's information in a 'buffer' subspace of the next state s_{i+1}. When $\beta \to \infty$, the second attention head attends only to positions whose buffered state matches the most recent state $\mu = s_N$. This head then retrieves an average of the states that immediately followed previous occurrences of μ in the sequence. With $w_C = 1$, the readout reduces to the empirical 2-point estimator, i.e., the 2-Gen predictor.

With the SA-transformer further simplified, we turn to the dynamics of the 1-Gen solution. At initialization, both attention layers are uniform over the context, so the experts corresponding to w_B, w_C, and w_D all reduce to the same 1-point estimator. The only term that does something different is $w_A x_N$, which predicts by repeating the final state. This repeat bias is disadvantageous on average for the Dirichlet ensemble, and we show that gradient descent rapidly drives w_A to zero (Appendix VIII 1). The G_1 plateau is therefore well-approximated by $w_A = 0$, $w_B = w_C = w_D = 1/3$, $\beta = \delta = 0$. Expanding the loss around this point to first order, we have

$$\mathcal{L}(\beta, \delta) = \mathcal{L}^{\text{1-Gen}} - c_\beta \beta - c_\delta \delta \cdots . \tag{5}$$

A naive self-averaging argument that replaces sums with expectations shows that the first nontrivial term is the second-order mixed contribution $\beta\delta$, so that the origin is a saddle point. This argument, however, does not explain why the network flows along $\beta > 0, \delta > 0$ rather than $\beta < 0, \delta < 0$. A careful calculation of the first-order terms shows that there are two important biases that tilt the landscape towards $\beta > 0, \delta > 0$. These biases imply that the network consistently converges to G_2, and that these terms determine how long it takes for the network to transition to G_2, i.e., $\tau_{\text{2-Gen}}$. We summarize key results below, and present the full details in Appendix VIII.

The coefficient c_δ has contributions from the w_B and w_D experts. Intuitively, these contributions arise because, given a sequence $s_1, s_2, \ldots, s_n$, it is slightly more likely for the next state s_{n+1} to return to s_{n-1} when averaged over the ensemble of stationary transition matrices.

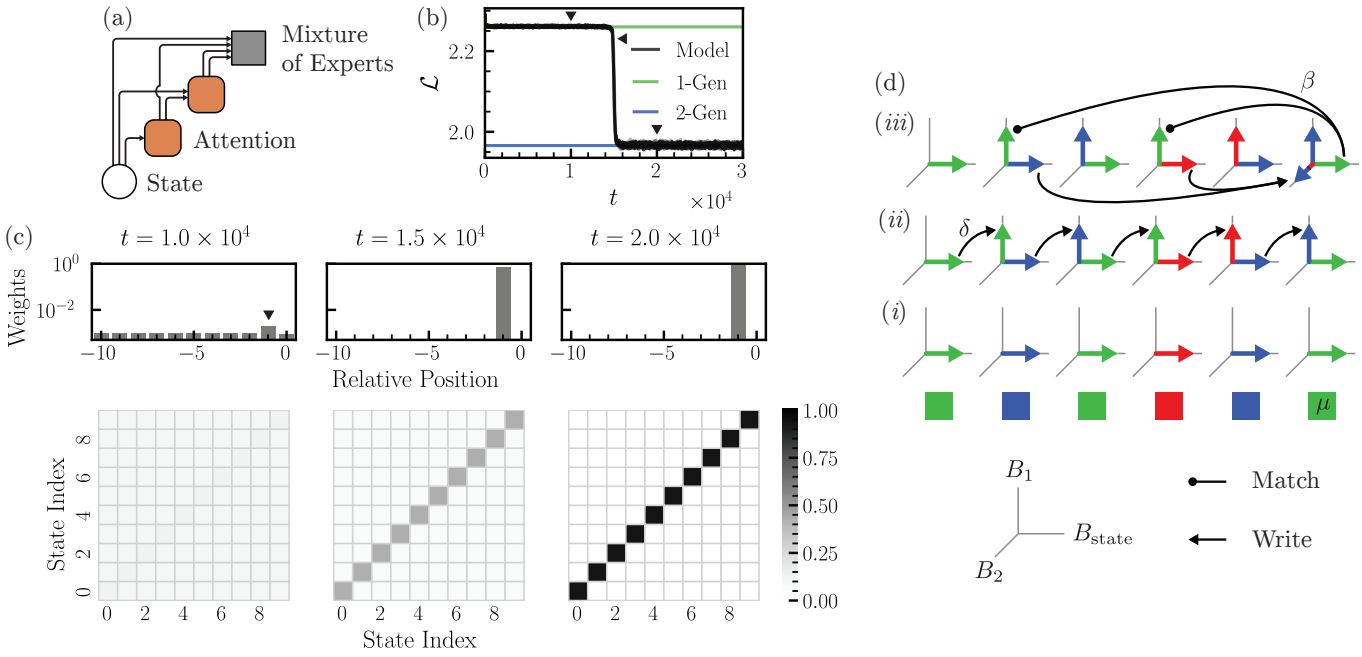

FIG. 5: (a) Schematic of the SA-Transformer for generalization. (b) Loss dynamics for the SA-Transformer trained in the limit $K \to \infty$ display the same sharp transition from G_1 (green line) to G_2 (blue line). Data is shown from 5 model seeds, with one darkened for clarity. (c) Model dynamics are dominated by the previous-state attention and the diagonal of one block of the key-query matrix. Top: Normalized weights from the relative positional bias in the first attention layer at different iterations. Bottom: Normalized state-wise attention derived from the bottom-left block of the key/query matrix in the second attention layer at different iterations. (d) A schematic of the induction-head mechanism underlying the model prediction in G_2 (refer to Appendix VI for a detailed explanation). (i) Given the state embeddings in subspace B_{state}, (ii) the first attention block selectively attends to the previous state (with specificity $\propto \delta$) and copies that state to an orthogonal subspace B_1, and (iii) the second attention block produces the empirical next-state distribution by comparing the current state with the contents of B_1 (with specificity $\propto \beta$) and copying the contents of B_2 from matching positions.

This leads the network to develop a slight bias towards paying attention to the previous state ($\delta > 0$). We calculate this bias by expanding $A^{(1)}$ to first order, which gives terms proportional to $P(s_{n-1} = \tau \mid s_N = \mu) - p_\tau$, where p is the stationary distribution of a particular chain. For most values of n, this difference is negligible because the chain has mixed, but it cannot be neglected when $n \approx N$. Averaging first over sequences conditioned on the final state and then over the transition matrix ensemble gives

$$c_\delta \approx \frac{w_B F_1}{N} + \frac{w_D}{N} \sum_{i \le N} \frac{F_{N-i+1}}{i}, \text{ where}$$
$$F_d \equiv \left\langle \sum_\mu (T^{d+1})_{\mu\mu} \right\rangle_{\mathcal{D}_T} - 1. \tag{6}$$

For a symmetric Dirichlet ensemble with parameter α, $F_1 = (C-1)/(C^2\alpha + C)$, so for the parameters used here ($C = 10$, $\alpha = 1$) one has $F_1 \approx 0.08$. Numerically, F_d decays rapidly with d (see Figure A10), so the second term in equation 6 is sub-leading and $c_\delta \simeq w_B F_1/N$.

The bias in coefficient c_β comes mainly from the w_C expert. Here, the bias can be understood by considering a state (say, s) that consistently follows the current state. The current state is slightly over-represented in the output of the first attention layer at previous occurrences of s (recall that the first attention layer pools when $\delta = 0$). This weak correlation between the current state and the over-representation of the current state at occurrences of s is sufficient to increase β. Each position i contributes only $\sim 1/i$, but the sum over the whole sequence produces a harmonic sum $H_N = \sum_{i=1}^N 1/i \sim \log N$. The resulting coefficient is

$$c_\beta \approx w_C \frac{H_N}{N} I, \text{ where } I \equiv \left\langle \sum_\mu p_\mu^2 \left(\sum_\tau \frac{T_{\tau\mu}^2}{p_\tau} - 1 \right) \right\rangle. \tag{7}$$

It is possible to show (using the Cauchy-Schwarz inequality) that $I > 0$ for every transition matrix, so this term contributes a second systematic bias ($\beta > 0$) toward induction head formation. Together, equation 6 and equation 7 show that the transition is not governed by a rare fluctuation that allows it to escape G_1, but rather by weak statistical biases that are fortuitously oriented towards the induction head solution. Notably, the w_B ex-

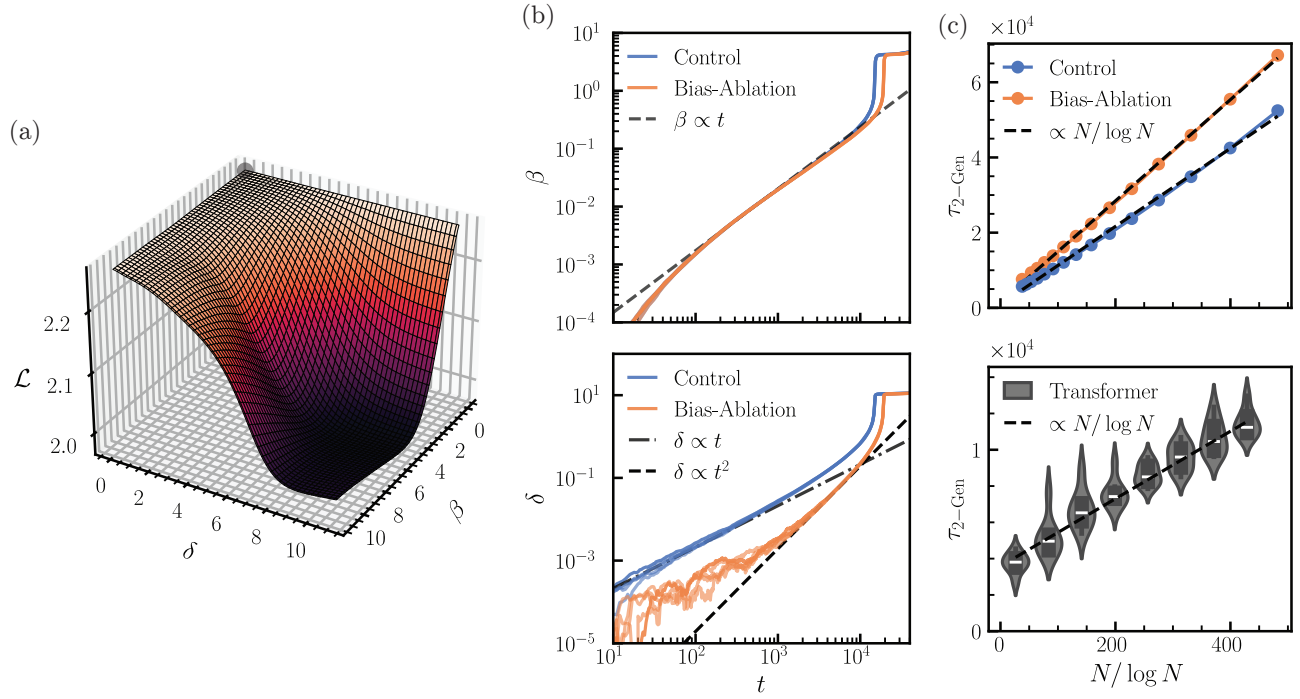

FIG. 6: (a) Loss landscape in the (β, δ) parameter space given by equation 3 and 4 (see equation VIII25 in Appendix) where w_{A-D} are obtained by optimization over the full landscape. (b) Top: Scaling of the effective parameter β with training iterations t. Bottom: Scaling of the effective parameter δ with training iterations t. Different lines in each plot with the same color represent numerics with different seeds. In the "Control" setting, the direct-δ bias predicts a linear growth of both δ, β. In the "Bias-Ablation" setting, the second-to-last sequence state has been resampled to remove the direct-δ bias which results in $\delta \sim t^2$ instead. (c) Scaling of the transition time $\tau_{2\text{-Gen}}$ with sequence length N. The time $\tau_{2\text{-Gen}}$ is measured as the number of iterations of SGD required until the sharp drop in loss indicating the development of the induction head. Top: Measurements for the SA-transformer (described in eq. VII13) in the two settings considered in panel (b). Bottom: Measurements for a non-autoregressive 2-layer transformer with MLP1 removed and additive relative positional biases. Optimization uses SGD with momentum. Kernel density estimated from 8 seeds.

pert does not contribute to the final 2-Gen solution, yet it provides the leading contribution to δ. Keeping only the leading terms near the plateau gives,

$$\frac{d\beta}{dt} \approx \frac{IH_N}{3N}, \qquad \frac{d\delta}{dt} \approx \frac{F_1}{3N}, \tag{8}$$

with $w_B = w_C = 1/3$ to lowest order. Figure 6a shows the corresponding reduced loss landscape after optimizing w_A-w_D at fixed (β, δ). The origin is a shallow 1-Gen basin. The two linear biases tilt the surface towards $\beta > 0, \delta > 0$, and once β and δ are order one, the model encounters a sharp cliff (due to the nonlinear softmax operation in an attention head) and rapidly converges to the 2-Gen basin. This picture explains why the drop in Figure 5b appears discontinuous.

The theory makes two further predictions that can be tested numerically. First, if the direct δ-bias is removed while leaving the long-range statistics intact, then β should still grow linearly but δ should grow only through its coupling to β. We test this by resampling the second-to-last state after sequence generation, which destroys the short-range correlation responsible for F_1 while preserving the longer-range structure of the task. We refer to this perturbed setting as the "bias-ablation" case, and to the original setting as the control case. The result of this perturbation matches predictions, as shown in Figure 6b,c: $\beta \sim t$ in both the original and perturbed settings, whereas $\delta \sim t$ in the original task but $\delta \sim t^2$ in the bias-ablation setting. Second, because β is the faster variable and reaches the nonlinear regime at a rate set by H_N/N, the transition time for a non-autoregressive transformer obeys $\tau_{2\text{-Gen}} \sim N/H_N \sim N/\log N$. Figures 6c verify this scaling both in the SA-transformer and for the transformer architecture (albeit, without MLP1) trained non-autoregressively. For autoregressive training, one should average gradients over all possible context lengths n uniformly weighted until N, which leads to

$$\tau_{2\text{-Gen}} \sim \left(\frac{1}{N} \sum_{n=1}^{N} \frac{H_n}{n} \right)^{-1} \approx 2N/(\log N)^2, \tag{9}$$

for $N \gg 1$ ($N = 1024$ in our setting).

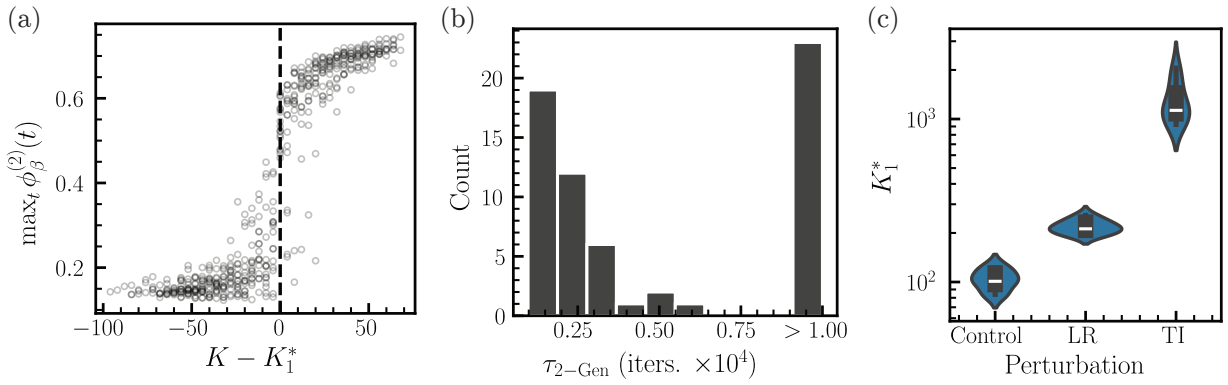

FIG. 7: (a) The maximum value of the induction head parameter $\phi_\beta^{(2)}$ through training is sigmoidal as the data diversity crosses over K_1^*. This indicates that the model either reaches the threshold and proceeds to G_2 (high $\phi_\beta^{(2)}$) or does not reach the threshold and proceeds to M_1, with few intermediates. Evaluated over 16 different model seeds trained on the same task distribution. Each seed has been shifted along the horizontal axis such that $\phi_\beta^{(2)}$ first crosses the threshold value ϕ_β^* at $K = K_1^*$. See Appendix X for details. (b) Bimodality of the model dynamics near K_1^*, shown for $K = 94 \approx K_1^*$ and 64 seeds trained on the same $\mathcal{D}_T$. Different model seeds show either small $\tau_{2\text{-Gen}}$ or never achieve G_2. All model seeds in the rightmost bin did not enter G_2 after 5×10^4 iterations. (c) Perturbations to the learning dynamics of the competing sub-circuits have effects on the memorization-generalization transition that are explainable by a kinetic competition. K_1^* increases when the effective learning rate of the 2-Gen circuit $\gamma_{2\text{-Gen}}$ is reduced (LR perturbation). Increasing the rate of memorization by providing information about the ground truth task identity also increases K_1^* (TI perturbation). Kernel density estimated from 4 seeds.

To determine whether these theoretical predictions extend beyond the SA-transformer, we also evaluated a standard architecture with learnable embeddings, query, key, and value matrices, and a MLP readout (detailed in Appendix VIII 7). While the derived scaling is valid for autoregressive training with SGD and RPB, more sophisticated optimizer and positional encoding schemes introduce elements that are not captured by the theory.

E. K_1^* is determined by a kinetic competition

The first transition from memorization to generalization appears at $K = K_1^*$ (Figure 1d and Figure 2). When $K < K_1^*$, the network transitions from G_1 to M_1 whereas for $K > K_1^*$, the network transitions from G_1 to G_2. The induction head parameter $\phi_\beta^{(2)}$ is nonzero only in G_2 and thus we measure K_1^* based on whether $\phi_\beta^{(2)}$ crosses a threshold (0.45) during training. This measurement leads to an estimate of $K_1^* \approx 94$. Figure 7a shows that the transition is sharp and effectively bimodal: models either cross the threshold and develop an induction head or transition into M_1, with few intermediates. Bimodality close to the transition is recapitulated in the distribution of $\tau_{2\text{-Gen}}$ close to K_1^* across seeds (Figure 7b).

The transition at K_1^* can be intuitively understood from a mixture-of-experts viewpoint. Suppose a network contains distinct subcircuits indexed by a and b (for e.g., 2-Gen and 1-Mem) and the next-state prediction is

$$\hat{\pi} \approx \vartheta \hat{\pi}_a + (1 - \vartheta)\hat{\pi}_b, \tag{10}$$

where $0 \leq \vartheta \leq 1$ gates the importance given to each subcircuit. This gating parameter is set by the relative reliability of each subcircuit in predicting the next state: the gradient update in ϑ due to a given sequence that incurs a loss $\mathcal{L} = -\log \hat{\pi}$ is $d\vartheta/dt \propto -\partial\mathcal{L}/\partial\vartheta \propto (\hat{\pi}_a - \hat{\pi}_b)$. That is, ϑ grows if $\hat{\pi}_a$ is a better predictor than $\hat{\pi}_b$. Further, if distinct sets of parameters contribute to $\hat{\pi}_a$ and $\hat{\pi}_b$, then their respective gradients have prefactors ϑ and $(1 - \vartheta)$. In short, the subcircuit that already assigns a larger probability to the correct next state is upweighted by the gate, while the gradient flowing into the losing subcircuit is suppressed. If the gate is learned on a faster timescale than the subcircuits, a small early performance difference in the two subcircuits favors the leading subcircuit regardless of which subcircuit is a better predictor after convergence. This kinetic competition between subcircuits can lead to a sharp transition (Figure 7a) if one of the subcircuits (here, 2-Gen) forms abruptly.

We directly test this kinetic picture with two perturbations that shift the two timescales in opposite directions, an effect which we verify in Figure A17 (further details in Appendix X). In the first perturbation, we slow the learning dynamics of only the 2-Gen circuit using gradient reweighting. This method reweights the gradient on previous-state attention in the first layer while leaving the forward pass unchanged. The perturbation shifts K_1^* upward approximately by a factor of two (Figure 7c). Importantly, this perturbation has no impact on the model

architecture, thus highlighting that the shift in K_1^* is due to a shift in training dynamics. In the second experiment, we apply a complementary intervention that accelerates memorization by injecting the ground-truth task identity into MLP2. This perturbation shifts K_1^* upward by a factor of ten (Figure 7c). Gradient reweighting and task injection therefore move the boundary in the same direction by increasing $\tau_{\text{2-Gen}}$ and decreasing $\tau_{\text{1-Mem}}$, respectively.

F. K_2^* and the representational capacity of the task recognition head

We anticipate a threshold K_2^* above which M_2 is never learned (Figure 1d). However, this is challenging to detect as the network's loss continues to gradually decline during training and may eventually transition into M_2 if trained long enough. To establish an estimate for K_2^*, we measure the time $\Delta\tau_K$ from the onset of G_2 until the transformers achieve a training loss $\mathcal{L}_{\text{train}} < \mathcal{L}^{\text{2-Gen}}$ (which is not possible without memorization) for different values of K. $\Delta\tau_K$ diverges as $(K_2^* - K)^{-\gamma}$, with an exponent $\gamma \approx 2$ and yields an estimate $K_2^* \approx 7000$ (Figure 8a). This estimate is consistent with an alternative measurement obtained from the fraction of memorized chains versus K (Figure 8b).

Since implementing 2-Mem requires the network to encode the transition matrices in its weights, we consider the hypothesis that the primary factor that determines K_2^* is tied to a representational bottleneck in one of the elements of the M_2 circuit, namely, the task recognition head. Recall that this circuit includes three core elements (equation 2): MLP1 in the encoder (λ), the task vector (φ) and MLP2 in the decoder (ψ). To systematically quantify how K_2^* changes with variations in these elements, we turn to a minimal autoregressive model of the task recognition head that isolates its essential features. As shown in Figure 8c and derived in Appendix XII, we replace the first attention head by providing MLP1 direct access to neighboring pairs and the second attention head by averaging over the pair embeddings written by MLP1 (Figure 8c). The resulting network retains only the operations that are essential for M_2. Concretely, the minimal model performs

$$\varphi_n = \Big\langle \text{MLP1}\left(x_{i+1} \oplus x_i\right) \Big\rangle_{i<n}, \quad z_n = \text{MLP2}(x_n \oplus \varphi_n)\,, \tag{11}$$

where x_i is the one-hot representation of s_i and the logits z_n determine the predicted probability of state s_{n+1}. The only control parameter relative to the full transformer is the task vector dimension D_φ. Note that when $D_\varphi \gtrsim C^2$, the network has sufficient expressivity to encode the full transition matrix in the task vector (whether training makes use of this capacity depends on K, Section IV G).

For a reference architecture (one hidden layer in MLP1 and two hidden layers in MLP2), we reproduce the defining signature of M_2 (Figures 8d, f). When K is small, the model achieves a training loss $\mathcal{L}_{\text{train}} < \mathcal{L}^{\text{2-Gen}}$ while displaying suboptimal generalization $\mathcal{L}_{\text{gen}} > \mathcal{L}^{\text{2-Gen}}$. As K increases, there is a crossover and 2-Gen is optimal. Because the minimal architecture is not expressive enough to form an induction head, the loss of this advantage isolates the point at which the task vector mechanism can no longer implement a 2-Mem circuit that outperforms 2-Gen. We therefore interpret this crossover as the analogue of the K_2^* boundary in the full transformer, and denote the corresponding threshold by $\hat{K}_2^*$.

We obtain an estimate $\hat{K}_2^*$ in the minimal model via binary search while varying D_φ and the depths of MLP1 and MLP2. Figure 8e shows that $\hat{K}_2^*$ is only weakly sensitive to the encoder. That is, for fixed D_φ and MLP2 depth, increasing MLP1 from one to two layers leaves $\hat{K}_2^*$ nearly unchanged. The primary determinants are D_φ, such that $\hat{K}_2^*$ scales approximately exponentially with D_φ when $D_\varphi < C^2$, and MLP2, such that removing a hidden layer from this block decreases $\hat{K}_2^*$ by a factor of ten. Thus, encoding a pair embedding is relatively simple, but representing and retrieving the corresponding collection of transition matrices in $\mathcal{S}$ is constrained by the finite-dimensional residual stream (set by the model dimension) and the expressivity of the decoder (set by MLP2). Specifically, the bottleneck at K_2^* primarily reflects a constraint on the decoder's ability to map task vectors to transition probabilities, rather than an information bottleneck in the encoder's capacity to isolate tasks and encode bigram distributions.

G. Task vectors as an alternative mechanism for in-context generalization

Importantly, the task recognition head is not intrinsically a memorizing mechanism. As $K \to \infty$, both $\mathcal{L}_{\text{train}}$ and $\mathcal{L}_{\text{gen}}$ approach $\mathcal{L}^{\text{2-Gen}}$ provided $D_\varphi \gtrsim C^2$, which means MLP2 reads transition statistics from φ and approximates the 2-Gen predictor without an induction head. In contrast, at small K, the model achieves $\mathcal{L}_{\text{train}} < \mathcal{L}^{\text{2-Gen}}$ but generalizes poorly (Figure 8d,f). This is true even when the task vector could in principle hold all C^2 transition probabilities (i.e., $D_\varphi \approx C^2$), indicating the network favors distinguishing among stored chains and retrieving their transition matrices.

These observations also demonstrate that memorization and generalization by the task recognition head are governed by both representational capacity (D_φ/C^2) and task diversity (K). These dependencies are recapitulated in the transformer, where ablating the G_2 circuit by restricting the second attention layer to uniform attention yields the usual four phase diagram when $D \approx C^2$ (Figure A19).

Finally, the ability of the task recognition head to generalize suggests two possible encoding schemes for the task vector, namely directly encoding the transition statistics or using a statistics-independent code such as

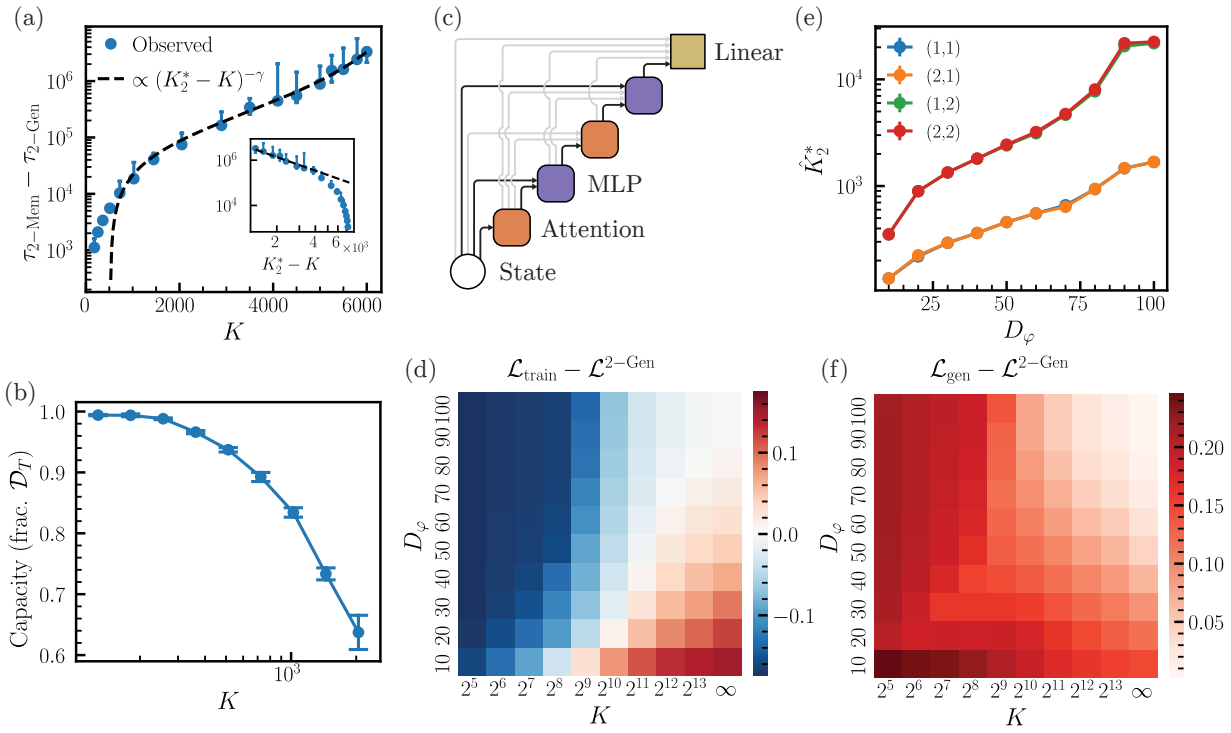

FIG. 8: (a) The time between the onset of G_2 and its disappearance in favor of M_2 diverges as $(K_2^* - K)^{-\gamma}$ with $\gamma \approx 2$ and $K_2^* \approx 7000$. The model is assumed to enter M_2 when it achieves a training loss $\mathcal{L}_{\text{train}} < \mathcal{L}^{\text{2-Gen}}$. Median value over 4 different seeds trained on the same task distribution $\mathcal{D}_T$ shown, with errorbars indicating the 95% confidence interval. Dashed line indicates a divergence fit to the medians with a linear offset. Inset: Replot of the data highlighting asymptotic behavior. The fit value for K_2^* was used to compute the offset $K_2^* - K$. Dashed line with slope $-\gamma$ provided for reference. (b) The mean fraction of memorized tasks extrapolated to convergence decreases linearly past $K \approx 2^8$. See Appendix XI for details. A linear extrapolation of these data suggests that the model will not memorize any chain for $K \approx 5000$, which is consistent with the estimate for K_2^* from the data in panel (a). Errorbars show standard deviation over the 4 seeds. (c) Schematic of the minimal network inspired from the circuit-tracing results in Figure 3, used to reproduce the 2-Mem behavior shown in the remaining panels. (d,f) Heat maps of $\mathcal{L}_{\text{train}} - \mathcal{L}^{\text{2-Gen}}$ and $\mathcal{L}_{\text{gen}} - \mathcal{L}^{\text{2-Gen}}$ as functions of the task-vector embedding dimension D_φ and data diversity K. Blue (red) indicates that the model is over (under) performing relative to the 2-Gen predictor; white denotes the crossover. (e) Average critical data diversity $\hat{K}_2^*$ over three independent trials. The labels (ℓ_1, ℓ_2) indicate the number of hidden layers in MLP1 and MLP2, respectively. The standard deviation error bar is estimated with three independent trials.

representing the task posterior. We distinguish between these in the M_2 transformer by training linear decoders to identify each quantity in the task vectors (Figure A15), the results of which suggest that the M_2 task vectors encode the task posterior rather than the transition statistics.

V. DISCUSSION

Through numerical experiments and phenomenology-driven theory, we characterize four distinct multi-layer circuits that implement distinct in-context learning algorithms in transformers. These circuits show that sequence context is used by the model in two qualitatively different ways: to estimate n-point statistics solely from the sequence, or to identify and retrieve a latent representation of a data generating process seen during training (both motifs shown schematically in Figure 3b,c). While the former allows the model to generalize outside the training set, the latter requires fewer examples in the context to achieve equivalent predictive accuracy if the data is sampled from a known process.

In the generalizing regime, 2-point statistics are extracted by a statistical induction head [36], a generalized version of an induction head first identified in large language models [5]. This circuit primarily relies on a multi-layer interaction between two attention heads. That is, the first attention layer Att1 writes local pair information into the residual stream, and the second attention layer Att2 performs a matching operation that estimates the relevant transition statistics [5, 7, 9, 36]. In the memorizing regime, we show that the model instead uses

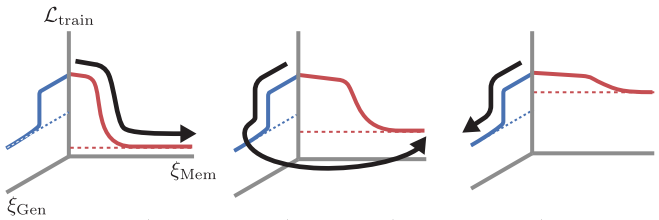

FIG. 9: A schematic summarizing the factors that govern memorization-generalization transitions. Suppose the transformer's training dynamics proceeds along a generalized coordinate for memorization (ξ_{Mem}) and generalization (ξ_{Gen}). (a) When the data diversity K is small relative to K_1^*, memorization is both faster than generalization and reaches a lower loss, so the model proceeds along ξ_{Mem} alone. (b) For K between K_1^* and K_2^*, generalization develops before memorization and the model begins progressing along ξ_{Gen}. However, the model eventually switches over to memorization, which can achieve a lower loss. (c) When K is well above K_2^*, constraints on model expressivity render memorization unfavorable relative to generalization, and dynamics are constrained to ξ_{Gen} as a result.

a distinct encoder-pool-decoder circuit, which we term the task recognition head. Here, the feedforward block MLP1 in the first layer constructs nonlinear embeddings of neighboring state pairs created by the first attention layer Att1, the second attention layer Att2 pools these pair embeddings across the sequence, and the second feedforward block MLP2 decodes the resulting sequence-level representation together with local state information. This pooled representation is naturally interpreted as a task vector, a compact latent representation of the process that generated the contextual sequence which can subsequently gate the operation performed on new inputs.

Two plausible mechanisms for in-context learning in large language models have been proposed: induction heads [5], and retrieval-oriented latent task representations [39, 40], though a circuit that implements the latter has not yet been found. Both these mechanisms are reproduced in our setting, and our proposed minimal models isolate the key features of both motifs. Importantly, we show that the task recognition head can also be a mechanism for generalization, though the constraints are much stronger than those required for a statistical induction head. Specifically, a task vector can support perfect generalization when a network with a sufficiently large residual stream and expressive feedforward block (here, MLP2) is trained on sufficiently diverse data (as quantified in Figure 8f). The task recognition head thus represents an expressive and seemingly general-purpose predictive mechanism, requiring only an encoder-pool-decoder structure, which could be implemented in a variety of substrates including chemical reaction networks [41] and RNNs [42].

Our analysis highlights the importance of the feedforward block, which is absent in previous analyses of in-context learning. Here, MLP1 is required to construct a nonlinear embedding of state pairs and MLP2 serves as a decoder that maps task vectors to the next-state distribution. In short, both attention and MLP blocks play distinct and algorithmically essential roles. More generally, this picture suggests a functional separation between layers in larger transformer models: early layers first compile evidence from subsequences in the context, these pieces of evidence are then pooled together to form a vector representation of a latent variable (task vector), and later layers act as decoders that efficiently process the new input in a context-dependent manner. This interpretation is consistent with empirical work in large language models which suggest the same general pattern of specialization [43–45].

The effective model for the 2-Gen circuit, based on our SA-transformer architecture, reveals the importance of statistical biases in the early formation of statistical induction heads. Although these biases do not reflect the 2-point structure of the data and weakly align with the final predictor in parameter space, they provide fortuitously oriented drift terms that enable the system to reliably escape flat regions of the loss landscape. This perspective could explain why pre-training with even nonlinguistic synthetic data can accelerate the performance of large language models [46]. Our data are limited to 2-point correlations; in the limit $K \to \infty$, we expect ladder-like generalization transitions to emerge as higher-order correlations and deeper transformer architectures are incorporated [47]. For instance, transformers also pick up on 3-point correlations [36]. It is straightforward to extend the SA-transformer architecture to more than two layers and multiple heads per layer, offering an opportunity for theory in settings that involve more complex statistical structure.

This work reconciles competing views of memorization-generalization transitions [30, 31, 33]. The first threshold, K_1^*, is kinetic: as data diversity increases, memorization slows, whereas the 2-Gen circuit forms on a largely K-independent timescale. The resulting kinetic competition between circuits determines whether the network transitions from G_1 to M_1 or from G_1 to G_2. The second threshold, K_2^*, is likely due to an expressivity constraint: even once the model can infer task identity from context, the finite capacity of the residual stream to represent task vectors and the limited decoding capacity of MLP2 limit how many distinct tasks can be encoded and reliably retrieved. The intuition behind these two mechanisms is presented schematically in Figure 9. This two-threshold picture explains why memorization and generalization can look like a competition of timescales in some settings, but like a capacity constraint in others [30, 33].

More broadly, this picture suggests a common language for related phenomena in diffusion models, where recent work likewise points to an early generalization time, a later dataset-size-dependent memorization time, and a separate model-dependent threshold beyond which long-time overfitting disappears [48, 49].

Our results are not limited to transformers and contribute to the emerging viewpoint that complex networks, once trained on large and diverse datasets, acquire remarkably rich mechanisms to represent and infer structure from a small number of examples. Understanding how and when such systems implement such computations should sharpen hypotheses about context-dependent learning in biological systems [42, 50] and offer design principles for physical learning systems in which direct parameter tuning is often difficult or impossible [41]. We therefore view the present results as fundamental mechanisms for how rapid learning can arise from a small set of reusable computational motifs.

VI. CODE AVAILABILITY

The code used to produce the results of this article is openly available at [51].

ACKNOWLEDGMENTS

We thank Colin Scheibner and members of the Reddy lab for insightful comments. GR was partially supported by a joint research agreement between NTT Research Inc. and Princeton University. This material is based upon work supported by the National Science Foundation Graduate Research Fellowship Program under Grant No. DGE-2444107. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the National Science Foundation. A portion of the simulations presented in this article were performed on computational resources managed and supported by Princeton Research Computing, a consortium of groups including the Princeton Institute for Computational Science and Engineering (PICSciE) and Research Computing at Princeton University.

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, Attention is all you need, Advances in neural information processing systems **30** (2017).
[2] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, *et al.*, Language models are few-shot learners, Advances in neural information processing systems **33**, 1877 (2020).
[3] S. Garg, D. Tsipras, P. S. Liang, and G. Valiant, What can transformers learn in-context? a case study of simple function classes, Advances in neural information processing systems **35**, 30583 (2022).
[4] S. Chan, A. Santoro, A. Lampinen, J. Wang, A. Singh, P. Richemond, J. McClelland, and F. Hill, Data distributional properties drive emergent in-context learning in transformers, Advances in neural information processing systems **35**, 18878 (2022).
[5] C. Olsson, N. Elhage, N. Nanda, N. Joseph, N. DasSarma, T. Henighan, B. Mann, A. Askell, Y. Bai, A. Chen, T. Conerly, D. Drain, D. Ganguli, Z. Hatfield-Dodds, D. Hernandez, S. Johnston, A. Jones, J. Kernion, L. Lovitt, K. Ndousse, D. Amodei, T. Brown, J. Clark, J. Kaplan, S. McCandlish, and C. Olah, In-context learning and induction heads (2022), arXiv:2209.11895 [cs.LG].
[6] L. Kirsch, J. Harrison, J. Sohl-Dickstein, and L. Metz, General-purpose in-context learning by meta-learning transformers, arXiv preprint arXiv:2212.04458 (2022).
[7] G. Reddy, The mechanistic basis of data dependence and abrupt learning in an in-context classification task, in *The Twelfth International Conference on Learning Representations* (2023).
[8] J. Von Oswald, M. Schlegel, A. Meulemans, S. Kobayashi, E. Niklasson, N. Zucchet, N. Scherrer, N. Miller, M. Sandler, M. Vladymyrov, *et al.*, Uncovering mesa-optimization algorithms in transformers, arXiv preprint arXiv:2309.05858 (2023).
[9] A. Bietti, V. Cabannes, D. Bouchacourt, H. Jegou, and L. Bottou, Birth of a transformer: A memory viewpoint (2023), arXiv:2306.00802 [stat.ML].
[10] P. A. Ortega, J. X. Wang, M. Rowland, T. Genewein, Z. Kurth-Nelson, R. Pascanu, N. Heess, J. Veness, A. Pritzel, P. Sprechmann, *et al.*, Meta-learning of sequential strategies, arXiv preprint arXiv:1905.03030 (2019).
[11] J. Von Oswald, E. Niklasson, E. Randazzo, J. Sacramento, A. Mordvintsev, A. Zhmoginov, and M. Vladymyrov, Transformers learn in-context by gradient descent, in *International Conference on Machine Learning* (PMLR, 2023) pp. 35151–35174.
[12] K. Ahn, X. Cheng, H. Daneshmand, and S. Sra, Transformers learn to implement preconditioned gradient descent for in-context learning, Advances in Neural Information Processing Systems **36**, 45614 (2023).
[13] R. Zhang, S. Frei, and P. L. Bartlett, Trained transformers learn linear models in-context, arXiv preprint arXiv:2306.09927 (2023).
[14] N. Hollmann, S. Müller, K. Eggensperger, and F. Hutter, Tabpfn: A transformer that solves small tabular classification problems in a second, arXiv preprint arXiv:2207.01848 (2022).
[15] Y. Zhang and W. Gilpin, Zero-shot forecasting of chaotic systems, arXiv preprint arXiv:2409.15771 (2024).
[16] A. Bao, J. Lai, and W. Gilpin, Transformers for dynamical systems learn transfer operators in-context, arXiv

preprint arXiv:2602.18679 (2026).
[17] L. Fu, H. Huang, G. Datta, L. Y. Chen, W. C.-H. Panitch, F. Liu, H. Li, and K. Goldberg, In-context imitation learning via next-token prediction, arXiv preprint arXiv:2408.15980 (2024).
[18] M. Laskin, L. Wang, J. Oh, E. Parisotto, S. Spencer, R. Steigerwald, D. Strouse, S. Hansen, A. Filos, E. Brooks, *et al.*, In-context reinforcement learning with algorithm distillation, arXiv preprint arXiv:2210.14215 (2022).
[19] J. Lee, A. Xie, A. Pacchiano, Y. Chandak, C. Finn, O. Nachum, and E. Brunskill, Supervised pretraining can learn in-context reinforcement learning, Advances in Neural Information Processing Systems **36**, 43057 (2023).
[20] A. Moeini, J. Wang, J. Beck, E. Blaser, S. Whiteson, R. Chandra, and S. Zhang, A survey of in-context reinforcement learning, arXiv preprint arXiv:2502.07978 (2025).
[21] R. Bommasani, D. A. Hudson, E. Adeli, R. Altman, S. Arora, S. von Arx, M. S. Bernstein, J. Bohg, A. Bosselut, E. Brunskill, *et al.*, On the opportunities and risks of foundation models, arXiv preprint arXiv:2108.07258 (2021).
[22] G. Brixi, M. G. Durrant, J. Ku, M. Naghipourfar, M. Poli, G. Sun, G. Brockman, D. Chang, A. Fanton, G. A. Gonzalez, *et al.*, Genome modelling and design across all domains of life with evo 2, Nature , 1 (2026).
[23] Y. Zhang, X. Chen, B. Jin, S. Wang, S. Ji, W. Wang, and J. Han, A comprehensive survey of scientific large language models and their applications in scientific discovery, in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing* (2024) pp. 8783–8817.
[24] M. Moor, O. Banerjee, Z. S. H. Abad, H. M. Krumholz, J. Leskovec, E. J. Topol, and P. Rajpurkar, Foundation models for generalist medical artificial intelligence, Nature **616**, 259 (2023).
[25] S. Subramanian, P. Harrington, K. Keutzer, W. Bhimji, D. Morozov, M. W. Mahoney, and A. Gholami, Towards foundation models for scientific machine learning: Characterizing scaling and transfer behavior, Advances in Neural Information Processing Systems **36**, 71242 (2023).
[26] E. O. Pyzer-Knapp, M. Manica, P. Staar, L. Morin, P. Ruch, T. Laino, J. R. Smith, and A. Curioni, Foundation models for materials discovery–current state and future directions, Npj Computational Materials **11**, 61 (2025).
[27] H. Dalla-Torre, L. Gonzalez, J. Mendoza-Revilla, N. Lopez Carranza, A. H. Grzywaczewski, F. Oteri, C. Dallago, E. Trop, B. P. De Almeida, H. Sirelkhatim, *et al.*, Nucleotide transformer: building and evaluating robust foundation models for human genomics, Nature Methods **22**, 287 (2025).
[28] A. Raventós, M. Paul, F. Chen, and S. Ganguli, Pretraining task diversity and the emergence of non-bayesian in-context learning for regression, Advances in neural information processing systems **36**, 14228 (2023).
[29] J. Wu, D. Zou, Z. Chen, V. Braverman, Q. Gu, and P. L. Bartlett, How many pretraining tasks are needed for in-context learning of linear regression?, arXiv preprint arXiv:2310.08391 (2023).
[30] A. Nguyen and G. Reddy, Differential learning kinetics govern the transition from memorization to generalization during in-context learning, arXiv preprint arXiv:2412.00104 (2024).
[31] C. F. Park, E. S. Lubana, I. Pres, and H. Tanaka, Competition dynamics shape algorithmic phases of in-context learning, arXiv preprint arXiv:2412.01003 (2024).
[32] D. Wurgaft, E. S. Lubana, C. F. Park, H. Tanaka, G. Reddy, and N. D. Goodman, In-context learning strategies emerge rationally, arXiv preprint arXiv:2506.17859 (2025).
[33] Y. M. Lu, M. I. Letey, J. A. Zavatone-Veth, A. Maiti, and C. Pehlevan, Asymptotic theory of in-context learning by linear attention, arXiv preprint arXiv:2405.11751 (2024).
[34] C. Goddard, L. M. Smith, V. Ngampruetikorn, and D. J. Schwab, When can in-context learning generalize out of task distribution?, arXiv preprint arXiv:2506.05574 (2025).
[35] Z. Lin and K. Lee, Dual operating modes of in-context learning, in *Forty-first International Conference on Machine Learning* (2024).
[36] B. L. Edelman, E. Edelman, S. Goel, E. Malach, and N. Tsilivis, The evolution of statistical induction heads: In-context learning markov chains (2024), arXiv:2402.11004 [cs.LG].
[37] A. Singh, S. Chan, T. Moskovitz, E. Grant, A. Saxe, and F. Hill, The transient nature of emergent in-context learning in transformers, Advances in neural information processing systems **36**, 27801 (2023).
[38] A. K. Singh, T. Moskovitz, F. Hill, S. C. Chan, and A. M. Saxe, What needs to go right for an induction head? a mechanistic study of in-context learning circuits and their formation, arXiv preprint arXiv:2404.07129 (2024).
[39] R. Hendel, M. Geva, and A. Globerson, In-context learning creates task vectors, in *Findings of the Association for Computational Linguistics: EMNLP 2023* (2023) pp. 9318–9333.
[40] E. Todd, M. L. Li, A. S. Sharma, A. Mueller, B. C. Wallace, and D. Bau, Function vectors in large language models, arXiv preprint arXiv:2310.15213 (2023).
[41] C. Floyd, H. M. L. Rios, A. R. Dinner, and S. Vaikuntanathan, In-context learning emerges in chemical reaction networks without attention, arXiv preprint arXiv:2601.06712 (2026).
[42] W.-L. Zheng, Z. Wu, A. Hummos, G. R. Yang, and M. M. Halassa, Rapid context inference in a thalamocortical model using recurrent neural networks, Nature Communications **15**, 8275 (2024).
[43] O. Press, N. A. Smith, and O. Levy, Improving transformer models by reordering their sublayers (2020), arXiv:1911.03864 [cs.CL].
[44] V. Lad, J. H. Lee, W. Gurnee, and M. Tegmark, The remarkable robustness of llms: Stages of inference? (2025), arXiv:2406.19384 [cs.LG].
[45] Y. Dong, L. Noci, M. Khodak, and M. Li, Is random attention sufficient for sequence modeling? disentangling trainable components in the transformer (2025), arXiv:2506.01115 [cs.LG].
[46] D. Lee, S. Han, A. Kumar, and P. Agrawal, Training language models via neural cellular automata, arXiv preprint arXiv:2603.10055 (2026).
[47] R. Rende, F. Gerace, A. Laio, and S. Goldt, A distributional simplicity bias in the learning dynamics of transformers, Advances in Neural Information Processing Systems **37**, 96207 (2024).
[48] T. Bonnaire, R. Urfin, G. Biroli, and M. Mézard, Why diffusion models don't memorize: The role of im-

plicit dynamical regularization in training, arXiv preprint arXiv:2505.17638 (2025).
[49] A. Favero, A. Sclocchi, and M. Wyart, Bigger isn't always memorizing: Early stopping overparameterized diffusion models, arXiv preprint arXiv:2505.16959 (2025).
[50] J. B. Heald, D. M. Wolpert, and M. Lengyel, The computational and neural bases of context-dependent learning, Annual Review of Neuroscience **46**, 233 (2023).
[51] C. Gibson, Context-adaptive transformer, https://github.com/cole-gibson/context-adaptive-transformer (2026).
[52] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, Roformer: Enhanced transformer with rotary position embedding, Neurocomputing **568**, 127063 (2024).
[53] D. Hendrycks and K. Gimpel, Gaussian error linear units (gelus) (2023), arXiv:1606.08415 [cs.LG].
[54] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever, *et al.*, Language models are unsupervised multitask learners, OpenAI blog **1**, 9 (2019).
[55] I. Loshchilov and F. Hutter, Decoupled weight decay regularization (2019), arXiv:1711.05101 [cs.LG].
[56] K. Wang, A. Variengien, A. Conmy, B. Shlegeris, and J. Steinhardt, Interpretability in the wild: a circuit for indirect object identification in gpt-2 small (2022), arXiv:2211.00593 [cs.LG].
[57] N. Goldowsky-Dill, C. MacLeod, L. Sato, and A. Arora, Localizing model behavior with path patching (2023), arXiv:2304.05969 [cs.LG].
[58] D. Friedman, A. Wettig, and D. Chen, Learning transformer programs, Advances in Neural Information Processing Systems **36**, 49044 (2023).
[59] A. M. Saxe, J. L. McClelland, and S. Ganguli, Exact solutions to the nonlinear dynamics of learning in deep linear neural networks, arXiv preprint arXiv:1312.6120 (2013).

Supplementary Information

CONTENTS

I. Introduction 1

II. Background 1

III. Summary of Contributions 3

IV. Results 3
- A. Identifying in-context memorization and generalization regimes 3
- B. Tracing circuits 5
- C. The task recognition head and the formation of task vectors 6
- D. The transition from G_1 to G_2 7
- E. K_1^* is determined by a kinetic competition 10
- F. K_2^* and the representational capacity of the task recognition head 11
- G. Task vectors as an alternative mechanism for in-context generalization 11

V. Discussion 12

VI. Code Availability 14

Acknowledgments 14

References 14

I. Summary of notation 18
1. Data generating process 18
2. Reference transformer architecture 19
3. Four algorithmic phases 19
4. Behavioral and mechanistic readouts 19
5. Minimal model for 2-Gen 19
6. Memorization and task vectors 20

II. Transformer architecture 21
1. Data generation 21
2. Network architecture 21
3. Training process 22

III. Defining the four algorithmic phases 23
1. Form of the Bayes predictor 23
2. Memorizing predictors 23
3. Generalizing predictors 24
4. Scaling of the four Bayes predictors with K and N 25

IV. Phase identification 26
1. Behavioral readouts 26
2. Mechanistic readouts 27
3. Hyperparameter variations 28

V. Transformer circuits 29
1. Path expansion 30
2. Circuit tracing 31
   a. 1-Gen 33
   b. 2-Gen 33
   c. 1-Mem 34
   d. 2-Mem 34

VI. Statistical induction head 34

VII. Minimal network for generalization in the limit $K \to \infty$ 35
1. Reduction of the network and circuits 35
2. Symmetry reduction 36
3. Numerical validation 37
a. Training details 37
b. Training results 38

VIII. Dynamics of the SA-transformer 39
1. Competition between x_N and the 1-Gen solution 39
2. The 2-Gen solution after convergence 40
3. Acquisition of the 2-Gen solution 41
a. First-order contributions in δ 41
b. First-order contributions in β 41
c. Dynamics near $\beta, \delta = 0$ 43
4. Testing predictions 43
a. Loss landscape 43
5. Ablating the first-order contribution in δ 44
6. The time to transition from the 1-Gen to the 2-Gen solution 44
7. Beyond the SA-Transformer 44
a. Architecture 45
b. Context-length dependence of the transition time 46
c. Initialization-dependent drift dynamics 47

IX. The memorization circuit 47
1. Task vector intuition 47
2. Representation geometry 48
3. Task vector patching 48
4. Identifying encoded information 49
5. Measuring task information content 50

X. Memorization to generalization transition 51
1. Task injection perturbation 52
2. Gradient reweighting perturbation 52

XI. Transient generalization and partial memorization 52

XII. Minimal model for 2-point memorization 54
1. Path expansion in M_2 54
2. Minimal network construction 54
3. Phase diagram 55
4. Dependence of $\hat{K}_2^*$ on encoder and decoder capacity 55

I: Summary of notation

We provide a brief description of each quantity and its value (unless otherwise specified). The subscripted symbols $\phi_\delta^{(1)}$ and $\phi_\beta^{(2)}$ denote first- and second-layer attention readouts, respectively; φ denotes the task vector.

1. Data generating process

C	number of states (set to 10)
T	$C \times C$ column-stochastic transition matrix, element $T_{\tau\mu}$ is transition probability $\mu \to \tau$
$\mathcal{D}_T$	distribution over T imposed by drawing each column i.i.d. from $\mathrm{Dir}(\alpha, C)$ α set to 1
$\mathcal{S}$	set of training transition matrices (or 'tasks') $\{T^{(1)} \dots T^{(K)}\}$ drawn from $\mathcal{D}_T$
K	data diversity $\lvert\mathcal{S}\rvert$

N maximum number of states in a sequence (typical value 2^8 for autoregressive, 2^{10} for fixed-length training)

p_μ C-dimensional stationary distribution, which satisfies $\sum_\mu p_\mu T_{\tau\mu} = p_\tau$ for corresponding transition matrix T.

s_i state at position i

S_n length n ordered sequence of states $\{s_i\}_{1:n}$

2. Reference transformer architecture

x_τ embedding vector for the state τ (interchangeably one-hot or learned embedding)

D state embedding dimension (typical value 64)

$\hat{\pi}^\theta$ predictive distribution output by a network

$A^{(\ell)}_{ni}$ attention weight given by sequence position n to position i

$\mathcal{L}_{\text{train}}$ training loss evaluated on sequences generated from chains in $\mathcal{S}$

$\mathcal{L}_{\text{gen}}$ generalization loss evaluated on sequences generated from chains in $\mathcal{D}_T$

3. Four algorithmic phases

$\hat{\pi}^{\text{1-Gen}}$ Bayes optimal predictor which uses one-point statistics to infer the chain from $\mathcal{D}_T$ that generated a sequence and predict the next state

G_1 algorithmic phase in which the transformer implements 1-Gen

$\hat{\pi}^{\text{2-Gen}}$ Bayes optimal predictor which uses two-point statistics to infer the chain from $\mathcal{D}_T$ that generated a sequence and predict the next state

G_2 algorithmic phase in which the transformer implements 2-Gen

$\hat{\pi}^{\text{1-Mem}}$ Bayes optimal predictor which uses one-point statistics to infer the chain from $\mathcal{S}$ that generated a sequence and predict the next state

M_1 algorithmic phase in which the transformer implements 1-Mem

$\hat{\pi}^{\text{2-Mem}}$ Bayes optimal predictor which uses two-point statistics to infer the chain from $\mathcal{S}$ that generated a sequence and predict the next state

M_2 algorithmic phase in which the transformer implements 2-Mem

K_1^* first data diversity threshold, above which the model enters G_2 at some point during training

K_2^* second data diversity threshold, above which the model remains in G_2 indefinitely

n_τ number of occurrences of state τ in a given sequence

$m_{\tau\mu}$ number of occurrences of the transition $\mu \to \tau$ in a given sequence

4. Behavioral and mechanistic readouts

$\phi^{(\ell)}_\delta$ order parameter indicating selective attention to the previous sequence position in the layer ℓ attention block

$\phi^{(\ell)}_\beta$ order parameter indicating selective induction-head-like attention in the layer ℓ attention block

$n^{(\ell)}_A$ entropy-based measure of the effective number of sequence positions attended to by the layer ℓ attention block

5. Minimal model for 2-Gen

δ effective previous state positional bias parameter in the first attention layer

β effective mean of the diagonal entries in the dot product between key and query matrices in the second attention layer

$w_{\{A,B,C,D\}}$ mixture weight of each expert in minimal model

$k_{\tau n}$ number of occurrences of state τ in the first n states of a given sequence

6. Memorization and task vectors

φ	Task vector, representation of the generating chain produced internally by the M_2 transformer
D_φ	Dimension of the task vector in the minimal model

II: Transformer architecture

1. Data generation

We follow a meta-training setup, where the parameters of a transformer are optimized on data generated from multiple distinct stochastic processes ('tasks'). Each process is specified by a Markovian transition matrix over C states. The transformer is trained on a fixed set of chains $\mathcal{S} = \{T^{(1)}, T^{(2)}, \ldots, T^{(K)}\}$. These transition matrices are sampled from a distribution over $C \times C$ transition matrices, which we denote $\mathcal{D}_T$. In our model, each column of each transition matrix is independently and identically drawn from a symmetric Dirichlet distribution, $\mathrm{Dir}(\alpha = 1, C = 10)$, where α is the Dirichlet concentration hyperparameter. The number of transition matrices (K) in $\mathcal{S}$ is a hyperparameter fixed before training.

During training, we first sample a transition matrix T from among the K transition matrices in $\mathcal{S}$. A sequence of N states is then sampled from T, beginning from a state drawn from the stationary distribution of T. We denote a sequence generated in this manner as $S_N \sim T$. To probe generalization, we sample *out-of-distribution* sequences of N states from a new transition matrix drawn from $\mathcal{D}_T$. Each state is implicitly represented as a C-dimensional one-hot vector (x_τ) before it is passed into the transformer.

2. Network architecture

Here, we describe our primary architecture, which is the two-layer transformer illustrated in Figure 1c. A transformer takes a sequence of vector-embedded states as input and produces a probability distribution over the next state as output [1]. Since the data generating process generates a sequence over C states, each sequence is first mapped to a sequence of D-dimensional vectors via a learnable embedding matrix, $W_E \in \mathbb{R}^{D\times C}$ which maps each state's one-hot representation to D dimensions. We typically refer to the D-dimensional representation of a state in a sequence as a *state*.

Each layer ℓ of the transformer consists of a self-attention block followed by a position-wise feedforward (MLP) block. Both blocks first normalize their input states via *layer-normalization*:

$$\begin{aligned}
\mathrm{LN}(x; \mu, \sigma^2) &= \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}, \\
\mu(x) &= \frac{1}{D}\sum_{i=1}^{D} x_i, \\
\sigma^2(x) &= \frac{1}{D}\sum_{i=1}^{D} (x_i - \mu)^2,
\end{aligned} \tag{II1}$$

where $\epsilon = 10^{-5}$ is a small constant added for numerical stability. The layer normalization acts state-wise when applied to a state sequence. For the typical case when the statistics are computed from the input, we write $\mathrm{LN}(x) \equiv \mathrm{LN}(x; \mu(x), \sigma^2(x))$. We write $\overline{x} \equiv \mathrm{LN}(x)$ to denote a vector normalized in this manner.

Self-attention computes a scalar score for a pair of states which depends on the query state x_q, key state x_k, and their relative distance r in the sequence

$$s^{(\ell)}(x_q, x_k, r) \equiv \frac{1}{\sqrt{D}} \overline{x}_q^\top W_Q^{(\ell)\top} W_r W_K^{(\ell)} \overline{x}_k, \tag{II2}$$

where $W_Q^{(\ell)}$, $W_K^{(\ell)} \in \mathbb{R}^{D\times D}$ are learnable *query* and *key matrices*, $W_Q x_q$ is the *query vector*, and $W_K x_k$ is the *key vector*. The fixed matrix $W_r \in \mathbb{R}^{D\times D}$ implements the rotary positional encoding [52], and is suppressed from the notation going forward. These scores are normalized to form *attention weights* between the current state and each sequence state

$$A_{ni}^{(\ell)} \equiv \frac{e^{s^{(\ell)}\left(x_n^{(\ell-1)}, x_i^{(\ell-1)}, n-i\right)}}{\sum_{j\le n} e^{s^{(\ell)}\left(x_n^{(\ell-1)}, x_j^{(\ell-1)}, n-j\right)}} \quad \text{for } i \le n. \tag{II3}$$

The output of the attention block is a sum over the sequence weighted by the attention weights

$$\mathrm{Att}_n^{(\ell)}\left(x_{i\le n}^{(\ell-1)}\right) \equiv \sum_{i\le n} A_{ni}^{(\ell)} W_V^{(\ell)} \overline{x}_i^{(\ell-1)}, \tag{II4}$$

where $W_V^{(\ell)} \in \mathbb{R}^{D\times D}$ is a learnable *value matrix* and $W_V^{(\ell)}\bar{x}_i^{(\ell-1)}$ is the *value vector* for state i.

Each attention block is followed by a position-wise feedforward (MLP) block. The MLP block consists of a single hidden layer with hidden dimension $d_{\text{hidden}} = 4D$ and Gaussian Error Linear Unit (GELU) activation function [53], and is computed

$$\text{MLP}^{(\ell)}\left(y_n^{(\ell-1)}\right) \equiv W_{\text{MLP},U}^{(\ell)}\,\text{GELU}\left(W_{\text{MLP},E}^{(\ell)}\overline{y}_n^{(\ell-1)}\right), \tag{II5}$$

where $W_{\text{MLP},E}^{\top(\ell)}$, $W_{\text{MLP},U}^{(\ell)} \in \mathbb{R}^{D\times 4D}$ and the activation acts point-wise $\text{GELU}(x) \equiv x \cdot \frac{1}{2}\left[1 + \text{erf}(x/\sqrt{2})\right]$.

Since each block adds its output to its input, we can recursively construct the current state embedding after the operation of each block. This quantity is commonly referred to as the *residual stream*, which tracks the evolution of the state representations over the transformer layers. Beginning from the state embedding, the residual stream after each block operation is

$$x_n^{(0)} = W_E x_n \tag{II6}$$
$$y_n^{(1)} = x_n^{(0)} + \text{Att}^{(1)}\left(x_{\leq n}^{(0)}\right) \tag{II7}$$
$$x_n^{(1)} = x_n^{(0)} + \text{Att}^{(1)}\left(x_{\leq n}^{(0)}\right) + \text{MLP}^{(1)}\left(y_n^{(1)}\right) \tag{II8}$$
$$y_n^{(2)} = x_n^{(0)} + \text{Att}^{(1)}\left(x_{\leq n}^{(0)}\right) + \text{MLP}^{(1)}\left(y_n^{(1)}\right) + \text{Att}^{(2)}\left(x_{\leq n}^{(1)}\right) \tag{II9}$$
$$x_n^{(2)} = x_n^{(0)} + \text{Att}^{(1)}\left(x_{\leq n}^{(0)}\right) + \text{MLP}^{(1)}\left(y_n^{(1)}\right) + \text{Att}^{(2)}\left(x_{\leq n}^{(1)}\right) + \text{MLP}^{(2)}\left(y_n^{(2)}\right) \tag{II10}$$

Note that we denote the residual stream after the layer ℓ attention block as $y_n^{(\ell)}$ and the residual stream after the MLP block as $x_n^{(\ell)}$. The remaining operation of the transformer is a linear output projection $W_U \in \mathbb{R}^{D\times C}$ which is applied to the residual stream after the final MLP block to produce the C-dimensional logit vector $x_n^{(3)}$

$$\text{Linear}(x) \equiv W_U^{\top}\bar{x} \tag{II11}$$
$$x_n^{(3)} = \text{Linear}(x_n^{(2)}) \tag{II12}$$

to which a softmax is applied to obtain a predictive distribution over the next state $\hat{\pi}_n^{\theta}(\tau) = e^{x_{n,\tau}^{(3)}} / \sum_{\tau'} e^{x_{n,\tau'}^{(3)}}$. Going forward, we write the output predictive distribution of the transformer given an input sequence S_n as $\hat{\pi}^{\theta}(\tau \mid S_n)$.

The model weights are initialized by independently sampling from a normal distribution with zero mean and variance $1/D$ except for weights that project to the residual stream, which are sampled from a normal distribution with zero mean and variance $1/4D$. This follows the GPT-2 initialization scheme [54].

3. Training process

The parameters of the model are optimized by minimizing the auto-regressive cross-entropy sequence prediction loss,

$$\mathcal{L}_{\text{train}}(\hat{\pi}^{\theta}) = \left\langle -\frac{1}{N}\sum_{n=1}^{N}\log\hat{\pi}^{\theta}(s_{n+1} \mid S_n)\right\rangle_{\substack{T\sim\mathcal{S} \\ S_{N+1}\sim T}}, \tag{II13}$$

where $S_n = (s_1, s_2, \ldots, s_n)$ for all n. We drop the argument and write $\mathcal{L}_{\text{train}}(\hat{\pi}) = \mathcal{L}_{\text{train}}$ when the predictor the loss is evaluated for is clear from context. A transition matrix T is sampled independently and uniformly at random from $\mathcal{S}$ for each sequence in the batch.

Minimization of this objective is accomplished through minibatch stochastic gradient descent. At each training step, the model's loss is computed on a batch of $B = 128$ sequences and the model's weights are updated via the AdamW optimizer [55] with learning rate $\gamma = 10^{-3}$, beta parameters $(\beta_1, \beta_2) = (0.9, 0.95)$, and weight decay $\lambda = 10^{-3}$.

III: Defining the four algorithmic phases

1. Form of the Bayes predictor

Across task diversities and over the course of training, we find that a transformer's behavior can be well-characterized by four algorithms (illustrated in Figure 1d). These algorithms are specific implementations of the Bayes-optimal predictor, which first infers the underlying transition matrix from an observed sequence S_N and then predicts the distribution $\hat{\pi}(s_{n+1} = \tau | s_n = \mu, S_n)$ over next states τ conditioned on the current state μ accordingly. Specifically,

$$\hat{\pi}_n(\tau|\mu) \equiv \hat{\pi}(s_{n+1} = \tau | s_n = \mu, S_n) = \int dT \, P(T \mid S_n) T_{\tau\mu}, \tag{III1}$$

$$= \int dT \, \frac{P(S_n \mid T) P(T)}{P(S_n)} T_{\tau\mu}. \tag{III2}$$

The four algorithms vary in the choice of prior $P(T)$ and the choice of sequence statistic (1-point or 2-point, see Figure 1b) when computing the likelihood $P(S_n \mid T)$.

2. Memorizing predictors

We model memorization by ideal predictors that have complete information about the transition matrices in $\mathcal{S}$. In this case, memorizing predictors correspond to the general Bayes predictor (Eq. III2) when the prior $P(T)$ matches $\mathcal{S}$. Making the substitution $P(T) = \frac{1}{K} \sum_{k=1}^{K} \delta(T - T^{(k)})$, we have

$$\hat{\pi}_n^{\text{Mem}}(\tau|\mu) = \frac{1}{K} \sum_{k=1}^{K} \frac{P(S_n \mid T^{(k)})}{P(S_n)} T^{(k)}_{\tau\mu}. \tag{III3}$$

Such memorizing predictors can be further classified into two types based on the choice of sequential statistics in $P(S_n \mid T^{(k)})$.

a. 1-point memorization. Computing the likelihood using 1-point sequence statistics,

$$P(S_n \mid T^{(k)}) = \prod_{i=1}^{n} P(s_i \mid T^{(k)}), \tag{III4}$$

yields the *1-Mem predictor*,

$$\hat{\pi}_n^{\text{1-Mem}}(\tau|\mu) = \frac{1}{K} \sum_{k=1}^{K} \frac{\prod_{i=1}^{n} P_k(s_i)}{P(S_n)} T^{(k)}_{\tau\mu}, \tag{III5}$$

where $P_k(s_i) = P(s_i \mid T^{(k)})$ is the stationary distribution over states for transition matrix $T^{(k)}$.

b. 2-point memorization. Computing the likelihood from 2-point sequence statistics,

$$P(S_n \mid T^{(k)}) = P_k(s_1) \prod_{i=2}^{n} P(s_i \mid s_{i-1}, T^{(k)}) \tag{III6}$$

in turn yields the *2-Mem predictor*,

$$\hat{\pi}_n^{\text{2-Mem}}(\tau|\mu) = \frac{1}{K} \sum_{k=1}^{K} \frac{P_k(s_1) \prod_{i=2}^{n} T^{(k)}_{s_i s_{i-1}}}{P(S_n)} T^{(k)}_{\tau\mu} \tag{III7}$$

where $T^{(k)}_{s_i s_{i-1}} = P(s_i \mid s_{i-1}, T^{(k)})$.

If the posterior over the transition matrices collapses to the true transition matrix $T^{(k)}$ from which the sequence is drawn (which is possible when K is small and n is large), we expect both predictors to recover the oracle solution, namely,

$$\hat{\pi}_n^{\text{2-Mem}}(\tau|\mu) = \hat{\pi}_n^{\text{1-Mem}}(\tau|\mu) = T^{(k)}_{\tau\mu}.$$

More generally, however, $\hat{\pi}_n^{\text{2-Mem}}(\tau|\mu)$ outperforms $\hat{\pi}_n^{\text{1-Mem}}(\tau|\mu)$, as the 2-point statistics enables more accurate discrimination when the data diversity is high.

3. Generalizing predictors

We model generalization by ideal predictors that predict the next state given complete knowledge of the data distribution $\mathcal{D}_T$ from which the transition matrices in $\mathcal{S}$ are sampled. Generalizing predictors thus correspond to the Bayes-optimal predictors (Eq. III2) when the assumed prior $P(T)$ matches $\mathcal{D}_T$.

a. 1-point generalization. For 1-point statistics, the relevant prior corresponds to the distribution over stationary distributions induced by transition matrices $T \sim \mathcal{D}_T$. Since the exact form of this distribution is unknown, we approximate it using a Dirichlet prior $\mathrm{Dir}(\alpha', C)$. Under this assumption, a standard Bayesian calculation yields the 1-Gen predictor, which predicts by counting the frequency of the target state in the finite sequence of length n:

$$\hat{\pi}_n^{\text{1-Gen}}(\tau|\mu) = \frac{n_\tau + \alpha'}{n + C\alpha'}, \tag{III8}$$

where n_τ denotes the number of occurrences of state τ in the length-n sequence. The role of α' is to act as a regularizer when n is small. Throughout our analysis, we fix $\alpha' = 1$.

In the limit $n \to \infty$, this estimate converges to the stationary distribution p_τ of T. The minimum cross-entropy loss in this regime is therefore given by

$$\mathcal{L}^{\text{1-Gen}} = -\left\langle \sum_\tau p_\tau \log p_\tau \right\rangle, \tag{III9}$$

which corresponds to the optimal generalizing loss when no information about the preceding state μ is available.

b. 2-point generalization. For 2-point statistics, the prior corresponds to the distribution over next-state frequencies conditioned on the current state. For $\mathcal{D}_T$, this prior is exactly a Dirichlet distribution $\mathrm{Dir}(\alpha = 1, C)$ applied independently to each row of the transition matrix. Consequently, the posterior factorizes over the rows and a standard Bayesian calculation yields the *2-Gen predictor*

$$\hat{\pi}_n^{\text{2-Gen}}(\tau|\mu) = \frac{m_{\tau\mu} + 1}{n_\mu + C}, \tag{III10}$$

where $m_{\tau\mu}$ denotes the number of occurrences of state τ given that the preceding state is the current state μ (note $\sum_\tau m_{\tau\mu} = n_\mu$).

In the large-n limit, we can show $\hat{\pi}_n^{\text{2-Gen}}(\tau|\mu) \to T_{\tau\mu}$ through $m_{\tau\mu} = np_\mu T_{\tau\mu} = n_\mu T_{\tau\mu}$. The optimal 2-point cross-entropy loss is

$$\mathcal{L}^{\text{2-Gen}} = -\left\langle \sum_{\mu,\tau} p_\mu T_{\tau\mu} \log T_{\tau\mu} \right\rangle. \tag{III11}$$

Moreover, in the large-n limit, 2-Gen has at most the loss incurred by 1-Gen:

$$\mathcal{L}^{\text{2-Gen}} - \mathcal{L}^{\text{1-Gen}} = \left\langle \sum_{\mu,\tau} p_\mu T_{\tau\mu} \log \frac{p_\tau}{T_{\tau\mu}} \right\rangle \leq 0, \tag{III12}$$

where we have used the inequality $\log x \leq x - 1$. Equality holds if and only if $T_{\tau\mu} = p_\tau$, i.e., when transitions are independent of the current state.

c. Optimal generalizing solutions for models with n-point dependencies. Here, we provide a mathematical argument that formalizes the intuition that an n-point model is sufficient to describe the optimal general solution when the next state depends at most on the preceding n states. Because the data-generating process in our experiments is first-order Markovian, the 2-point model emerges as the optimal general solution, even when longer histories are available. For simplicity, we only provide the argument for large and fixed (i.e., not autoregressive) n.

We begin by considering a predictor π that does not condition on any preceding state. In this case, the expected cross-entropy objective can be written as

$$\mathcal{C}(\pi) = -\left\langle \sum_\mu p_\mu \sum_\tau T_{\tau\mu} \log \pi_\tau + \lambda \left(\sum_\tau \pi_\tau - 1 \right) \right\rangle, \tag{III13}$$

where λ is a Lagrange multiplier enforcing normalization of π. Minimizing $\mathcal{C}(\pi)$ yields the optimal solution $\pi_\tau = p_\tau$, corresponding to the 1-Gen predictor.

Next, suppose that the predictor conditions only on the immediately preceding state μ. The objective becomes

$$\mathcal{C}(\pi) = -\left\langle \sum_{\mu} \left[p_\mu \sum_{\tau} T_{\tau\mu} \log \pi_{\tau|\mu} + \lambda_\mu \left(\sum_{\tau} \pi_{\tau|\mu} - 1 \right) \right] \right\rangle, \tag{III14}$$

where λ_μ enforces normalization of $\pi_{\tau|\mu}$ for each μ. Optimizing with respect to $\pi_{\tau|\mu}$ yields

$$\pi_{\tau|\mu} = \frac{p_\mu T_{\tau\mu}}{\lambda_\mu}. \tag{III15}$$

Imposing the normalization constraint gives $\lambda_\mu = p_\mu$, and therefore $\pi_{\tau|\mu} = T_{\tau\mu}$, which is precisely the 2-Gen predictor. Finally, if the predictor is allowed to condition on two preceding states (ν, μ), the objective can be written as

$$\mathcal{C}(\pi) = -\left\langle \sum_{\nu,\mu} \left[p_\nu T_{\mu\nu} \sum_{\tau} T_{\tau\mu} \log \pi_{\tau|\mu,\nu} + \lambda_{\mu\nu} \left(\sum_{\tau} \pi_{\tau|\mu,\nu} - 1 \right) \right] \right\rangle. \tag{III16}$$

Optimizing this objective again yields $\pi_{\tau|\mu,\nu} = T_{\tau\mu}$, which coincides with the 2-Gen predictor. Thus, although a longer history is available, the Markov property of the data-generating process implies that the optimal general solution is always the 2-Gen predictor.

4. Scaling of the four Bayes predictors with K and N

Figure A1 compares the loss incurred by the four Bayesian predictors as a function of data diversity K and sequence length N on training sequences.

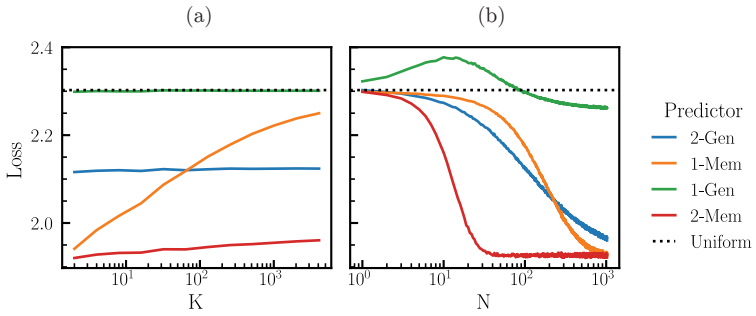

FIG. A1: Scaling of the four predictor losses with data diversity K and sequence length N computed over a large batch of sequences and averaged over 8 task distributions $\mathcal{D}_T$. The $N = 256$ autoregressive loss is shown for K scaling. N scaling of the non-autoregressive loss is shown at data diversity $K = 128$.

a. Dependence on data diversity (K). In Figure A1a, we plot the autoregressive loss at fixed context length $N = 256$ while varying K. The 1-Mem predictor degrades as K increases. This behavior is expected: when many processes may have generated a sequence, short sequences provide insufficient information to reliably identify which transition matrix the sequence is sampled from. A notable feature is a cross-over between the 1-Mem and 2-Gen curves, suggesting that the loss landscape favors a different solution with increasing K. Thus, the preferred predictor for training sequences switches from 1-Mem to 2-Gen.

By contrast, the other three predictors remain comparatively stable across a broad range of K. In the idealized limit of exact Bayesian inference with unbounded memory, we therefore expect the ordering

$$\mathcal{L}^{\text{2-Mem}} < \mathcal{L}^{\text{2-Gen}} < \mathcal{L}^{\text{1-Gen}},$$

for our experimental setup. While this ordering applies for ideal Bayes predictors, architectural constraints could reverse the comparison between 2-Gen and 2-Mem at large K, that is, $\mathcal{L}^{\text{2-Gen}} < \mathcal{L}^{\text{2-Mem}}$.

b. Dependence on sequence length (N). In Figure A1b, we fix $K = 128$ and vary the sequence length. All four predictors improve with increasing N, but for different reasons.

For memorization, the dominant bottleneck is task inference: the estimator needs enough evidence in the sequence to identify the latent task. Consistent with this, the 2-Mem predictor approaches its asymptotic performance at $N \sim 10^2$, whereas the 1-Mem predictor requires substantially longer sequences, $N \sim 10^3$.

For generalization, the limiting factor is finite-sample noise in estimating transition statistics from the sequence. Consequently, the loss decreases more gradually with N than for the memorization predictors. The 1-Gen predictor performs worst throughout, as it ignores sequential structure and predicts only the stationary distribution. The 2-Gen predictor steadily improves and becomes comparable to the optimal predictor (2-Mem) only at much longer sequence lengths, $N \sim 10^4$.

As in the K-scaling, we again observe a crossover between the 1-Mem and 2-Gen predictors, consistent with a transition between a task-identification regime (small N/small K) and a statistics-estimation regime (large N/large K).

IV: Phase identification

We trained the transformer model (Section II 2) in settings where the data diversity K was varied over powers of 2 from 2^3 to 2^{10}, corresponding to logarithmically spaced diversities between 8 and 1024 transition matrices. A total of 10^6 iterations of stochastic gradient descent were performed in each setting, with checkpoints of the model state saved at logarithmically spaced intermediate iterations to allow for post-training evaluation. The training task distributions for a single data diversity sweep were constructed such that $\mathcal{S}$ at data diversity k is a subset of all $\mathcal{S}'$ for task diversities $k' \geq k$.

Across data diversities and throughout training, we find that transformer behavior is well-characterized by one of the four predictors discussed in the previous section. These observations reproduce those found in [31]. We denote these regimes G_1, M_1, G_2 and M_2 which correspond to the transformer implementing 1-Gen, 1-Mem, 2-Gen, and 2-Mem respectively. This characterization is supported by two complementary observations:

1. **Behavioral readouts**: Close agreement between the predictions of the transformer and a Bayesian predictor.
2. **Mechanistic readouts**: Attention patterns with clear interpretations.

1. Behavioral readouts

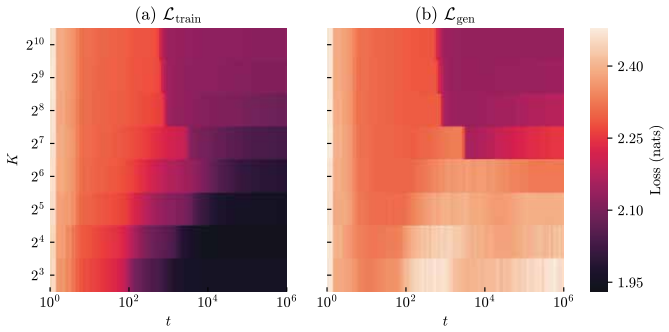

FIG. A2: Training and generalization losses for the same models shown in Figure 2.

We evaluated the training and generalization loss of each model checkpoint on a sample of $8 \times K$ and 2048 sequences respectively, where the generalization loss is given by

$$\mathcal{L}_{\text{gen}} = \left\langle -\frac{1}{N} \sum_{n=1}^{N} \log \hat{\pi}_\theta(s_{n+1} \mid S_n) \right\rangle_{\substack{T \sim \mathcal{D}_T \\ S_{N+1} \sim T}} . \tag{IV1}$$

This allowed us to compare the loss values through training to the loss of each predictor on the same sequences. We find plateaus in both the training and generalization losses, which correspond closely with loss values of the predictors

(Figure 2a , Figure A2 visualizes these trajectories for all K.). To construct a quantitative metric, we compute the divergence between a predictor $\hat{\pi} \in \{\hat{\pi}^{\text{1-Mem}}, \hat{\pi}^{\text{2-Mem}}, \hat{\pi}^{\text{1-Gen}}, \hat{\pi}^{\text{2-Gen}}\}$ and transformer model $\hat{\pi}_\theta(S_n)$ as

$$\mathrm{D}(\hat{\pi}, \hat{\pi}_\theta) \equiv \frac{1}{2} \left\langle D_{\mathrm{KL}}^{S_N}(\hat{\pi} \,\|\, \hat{\pi}_\theta) \right\rangle_{\substack{T\sim\mathcal{S} \\ S_N\sim T}} + \frac{1}{2} \left\langle D_{\mathrm{KL}}^{S_N}(\hat{\pi} \,\|\, \hat{\pi}_\theta) \right\rangle_{\substack{T\sim\mathcal{D}_T \\ S_N\sim T}}, \tag{IV2}$$

where $D_{\mathrm{KL}}^{S_N}(\hat{\pi} \,\|\, \hat{\pi}_\theta) \equiv \frac{1}{N}\sum_{n=1}^{N} D_{\mathrm{KL}}\left(\hat{\pi}(S_n) \,\|\, \hat{\pi}_\theta(S_n)\right)$. In practice, for a model trained on K tasks, the expectation values over $\mathcal{S}$ and $\mathcal{D}_T$ are approximated by averaging over $8 \times K$ and 2048 sequences respectively.

In Figure 2b, the plots of the divergence to each predictor in the (K, t) parameter space reveal that the transformer's response almost always approximates one of the four predictors, and that these regimes form contiguous regions of parameter space. We refer to these regions as algorithmic phases of the transformer.

2. Mechanistic readouts

Only the attention layers of the transformer architecture can mix information along the sequence dimension. Consequently, for the model to infer nearest-neighbor 2-point correlations (i.e., bigrams) in a sequence, at least one attention layer must attend to the previous state. To identify when an attention layer exhibits this behavior, we compute the expected average attention weight assigned to the previous state across a sequence:

$$\phi_\delta^{(\ell)} = \left\langle \frac{1}{N-1} \sum_{i=2}^{N} A_{i,i-1}^{(\ell)} \right\rangle_{\substack{T\sim\mathcal{S} \\ S_N\sim T}}. \tag{IV3}$$

When the layer does not preferentially attend to the previous state, $\phi_\delta^{(\ell)} = \mathcal{O}(1/N)$, whereas $\phi_\delta^{(\ell)} = 1$ when the layer attends exclusively to the previous state. We therefore refer to $\phi_\delta^{(\ell)}$ as the *previous-state order parameter* of the attention layer. Empirically, we find that only the first attention layer attains a non-negligible value of $\phi_\delta^{(\ell)}$, reflecting the necessity for 2-point information to be extracted in the first layer so that it can be aggregated across the sequence by the second attention layer.

The G_2 phase of the transformer is implemented through the composition of the first and second attention layers, forming a 'statistical induction head' which we discuss at length in Appendix VI [7, 9, 36]. A defining feature of an induction head circuit is that the second-layer attention concentrates on sequence positions corresponding to states that follow the current state in the sequence. To detect this behavior, we compute the expected total attention weight that the final state assigns to all states that immediately follow occurrences of itself in the sequence:

$$\phi_\beta^{(\ell)} \equiv \left\langle \frac{1}{N} \sum_{n=2}^{N} \sum_{i=2}^{n} \delta_{s_n, s_{i-1}} A_{ni}^{(\ell)} \right\rangle_{\substack{T\sim\mathcal{S} \\ S_N\sim T}} \tag{IV4}$$

Since $\phi_\beta^{(2)}$ becomes non-zero when an induction head circuit is formed, we refer to $\phi_\beta^{(2)}$ as the *induction-head order parameter* of the second attention layer.

In Figure 2c, the upper two plots of the order parameters in the K, t parameter space reveal regions of nontrivial $\phi_\delta^{(1)}$ and $\phi_\beta^{(2)}$ which are in strong correspondence with the behavioral readouts indicating general 2-point utilization and G_2, respectively.

To complement the above order parameters, which indicate the presence of highly specific attention patterns, we compute the effective number of sequence states an attention block attends to at the final position:

$$n_A^{(\ell)} \equiv e^H, \tag{IV5}$$

$$H \equiv \left\langle -\sum_{i \le n} A_{ni}^{(\ell)} \log A_{ni}^{(\ell)} \right\rangle_{\substack{T\sim\mathcal{S} \\ S_N\sim T}} \tag{IV6}$$

where H is the expected entropy of the attention map for the largest sequence length N. When an attention layer has $n_A^{(\ell)} \approx N$, the layer can be well approximated by an average of the states over the sequence.

In practice, the expectation value over sequences in equations IV3, IV4, and IV6 is approximated by averaging over a batch of 2048 sequences (with the exception of the results in Figure 7a, which were evaluated on 4096 sequences).

3. Hyperparameter variations

To better understand the generality of the identified algorithmic phases, we trained models with variations of architectural and dataset parameters relative to the baseline configuration. The behavioral readouts from these models reveal that the phase structure is maintained across a wide variety of settings(Figure A3).

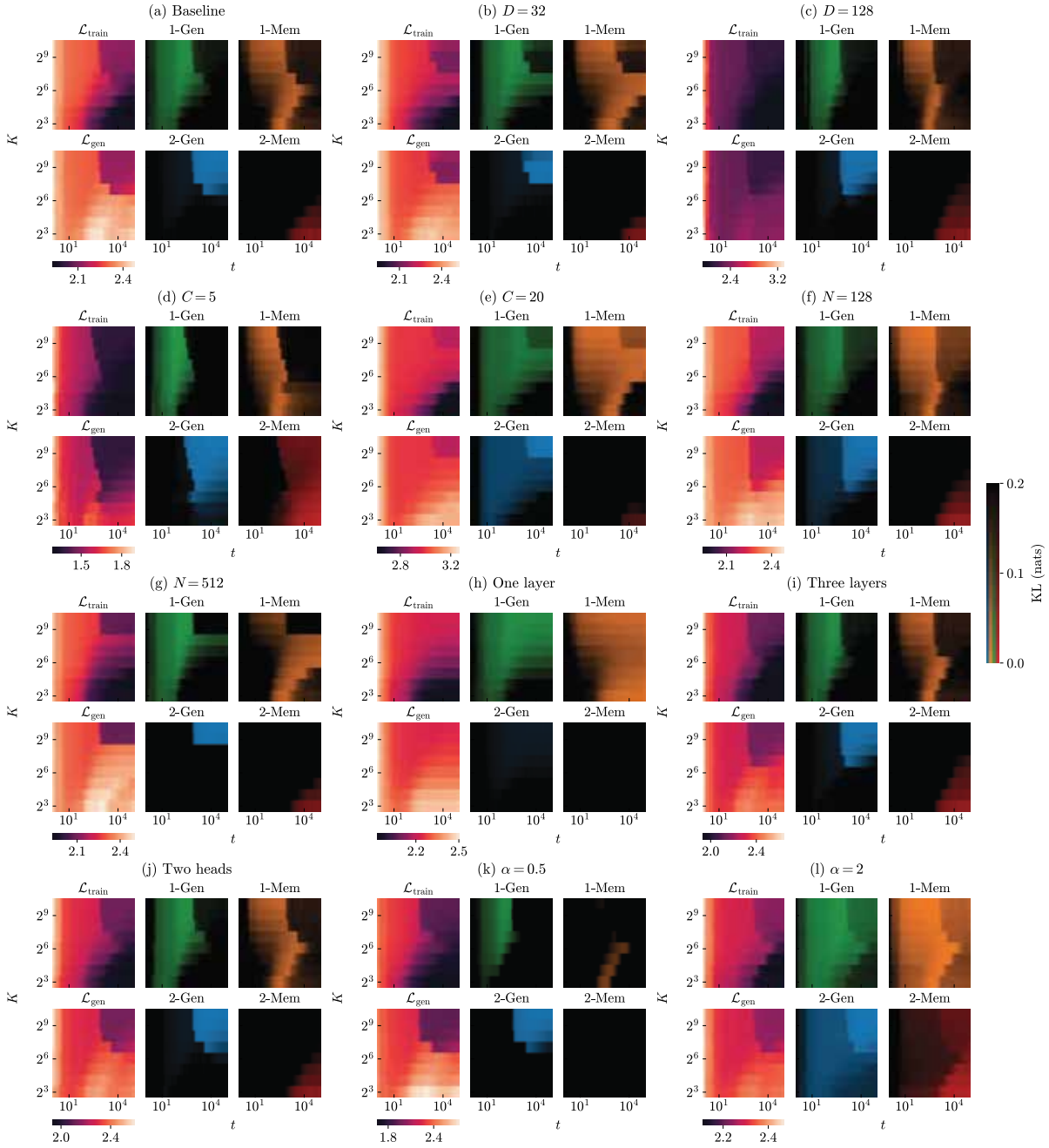

FIG. A3: Training and generalization loss dynamics, and behavioral readouts as K is varied for a variety of architectural and dataset modifications. The baseline phase diagram is reproduced in (a), and is truncated to $t \leq 10^5$ training iterations to ease comparison. The colorbar below each subplot corresponds to loss values. The four phases are present for all models except the one-layer model (h), in which case the absence of both 2-point phases can be understood by the circuit analyses for these phases (Section V). For sparse transition distributions (k), the M_2 phase is indicated by the training loss but not the similarity, likely due to extreme predictions made out-of-distribution by the Bayes predictor.

Source Name	Vector
State	$x_n^{(0)}$
A1(State)	$\sum_{i\le n} A_{ni}^{(1)} W_V^{(0)} \overline{x}_i^{(0)}$
MLP1	$\mathrm{MLP}^{(1)}\left(y_n^{(1)}\right)$
A2(State)	$\sum_{i\le n} A_{ni}^{(2)} W_V^{(2)} \mathrm{LN}\left(x_i^{(0)}; \mu\left(x_i^{(1)}\right), \sigma^2\left(x_i^{(1)}\right)\right)$
A2(A1)	$\sum_{i\le n} \mathrm{A}_{ni}^{(2)} W_V^{(2)} \mathrm{LN}\left(\mathrm{Att}^{(1)}\left(x_{\le i}^{(0)}\right); \mu\left(x_i^{(1)}\right), \sigma^2\left(x_i^{(1)}\right)\right)$
A2(MLP1)	$\sum_{i\le n} \mathrm{A}_{ni}^{(2)} W_V^{(2)} \mathrm{LN}\left(\mathrm{MLP}^{(1)}\left(y_i^{(1)}\right); \mu\left(x_i^{(1)}\right), \sigma^2\left(x_i^{(1)}\right)\right)$
MLP2	$\mathrm{MLP}^{(2)}\left(y_n^{(2)}\right)$

(a) Source vector contributions to the residual stream.

Target Name	Perturbation
A[ℓ]Q	$\mathrm{A}_n^{(\ell)}(i) = \frac{e^{s^{(\ell)}\left(\tilde{x}_n^{(\ell-1)}, x_i^{(\ell-1)}, n-i\right)}}{\sum_{j\le n} e^{s^{(\ell)}\left(\tilde{x}_n^{(\ell-1)}, x_j^{(\ell-1)}, n-j\right)}}$
A[ℓ]K	$\mathrm{A}^{(\ell)}(i) = \frac{e^{s^{(\ell)}\left(x_n^{(\ell-1)}, \tilde{x}_i^{(\ell-1)}, n-i\right)}}{\sum_{j\le n} e^{s^{(\ell)}\left(x_n^{(\ell-1)}, \tilde{x}_j^{(\ell-1)}, n-j\right)}}$
MLP[ℓ]	$\mathrm{MLP}^{(\ell)}\left(\tilde{y}_n^{(\ell)}\right)$
Linear	$\mathrm{Linear}\left(\tilde{x}_n^{(2)}\right)$

(b) Perturbation targets used in analysis. The perturbed input is marked with a tilde. For the query and key perturbations, all attention weights are perturbed following this prescription.

TABLE I: Explicit definition of source vectors contributing to the residual stream, and the target perturbations. To ablate the edge from source SRC to a target TGT at layer ℓ, we compute the perturbed input as $\tilde{x}_n^{(\ell-1)} \equiv x_n^{(\ell-1)} - \left(x_{\mathrm{SRC},n} - \langle x\rangle_{\mathrm{SRC},n}\right)$ where $x_{\mathrm{SRC},n}$ is the source vector corresponding to SRC (Subtable (a)) and $\langle\cdot\rangle$ denotes the batch average. The target is then perturbed during the forward pass by injection of $\tilde{x}_n^{(\ell-1)}$ according to the prescription for target TGT (Subtable (b)). The layer-normalization each block applies to the perturbed input recomputes the mean μ from the perturbed input and reuses the variance σ^2 from the unperturbed input. This allows, in particular, the attention output Eq. V8 to be decomposed into additive contributions from earlier blocks.

V: Transformer circuits

We find strong evidence that each of the four algorithmic phases of the model is governed by a sparse circuit. This sparsity enables us to decompose the full, complex network into reduced and interpretable sub-networks, and to study the mechanisms by which different Bayesian algorithms are implemented.

1. Path expansion

We first explain the nature of the circuit edges we consider. Recall the construction of the residual stream after each block

$$x_n^{(0)} = W_E x_n \tag{V1}$$
$$y_n^{(1)} = x_n^{(0)} + \text{Att}^{(1)}\left(x_{\leq n}^{(0)}\right) \tag{V2}$$
$$x_n^{(1)} = x_n^{(0)} + \text{Att}^{(1)}\left(x_{\leq n}^{(0)}\right) + \text{MLP}^{(1)}\left(y_n^{(1)}\right) \tag{V3}$$
$$y_n^{(2)} = x_n^{(0)} + \text{Att}^{(1)}\left(x_{\leq n}^{(0)}\right) + \text{MLP}^{(1)}\left(y_n^{(1)}\right) + \text{Att}^{(2)}\left(x_{\leq n}^{(1)}\right) \tag{V4}$$
$$x_n^{(2)} = x_n^{(0)} + \text{Att}^{(1)}\left(x_{\leq n}^{(0)}\right) + \text{MLP}^{(1)}\left(y_n^{(1)}\right) + \text{Att}^{(2)}\left(x_{\leq n}^{(1)}\right) + \text{MLP}^{(2)}\left(y_n^{(2)}\right) \tag{V5}$$
$$x_n^{(3)} = \text{Linear}(x_n^{(2)}) \tag{V6}$$

Each term in the input to a block corresponds to the output of previous blocks[1], so we consider these terms as connections from prior blocks to the input block. This produces $1+2+3+4+5 = 15$ edges in the transformer graph. This graph is depicted in Figure 3a.

a. Attention subcircuit. We decompose the attention block into three operations and consider each as a node in an attention subcircuit: the computation of the queries, the computation of the keys, and the linear weighted sum along the sequence. Each of these operations read from the full residual stream and thus have 1 and 3 incoming edges in the first and second attention blocks respectively. The query and key computations are internal to the attention circuit and do not produce outgoing edges. On the other hand, the sum over the sequence is linear and acts on inputs independently, producing an equal number of outgoing edges as incoming. This observation is trivial for the first attention block since it has one input and output regardless. For the second attention block, the three outputs correspond to a linear transformation by $W_V^{(2)}$ of each layer-normalized term of the residual stream summed over the sequence,

$$\text{Att}^{(2)}\left(x_{\leq n}^{(1)}\right) = \sum_{i\leq n} A_{ni}^{(2)} W_V^{(2)} \text{LN}\left(x_i^{(0)} + \text{Att}^{(1)}\left(x_{\leq i}^{(0)}\right) + \text{MLP}^{(1)}\left(y_i^{(1)}\right)\right) \tag{V7}$$
$$= \sum_{i\leq n} A_{ni}^{(2)} W_V^{(2)} \text{LN}\left(x_i^{(0)} + \text{Att}^{(1)}\left(x_{\leq i}^{(0)}\right) + \text{MLP}^{(1)}\left(y_i^{(1)}\right); \mu\left(x_i^{(1)}\right), \sigma^2\left(x_i^{(1)}\right)\right) \tag{V8}$$

where we have written the layer-normalization explicitly following the definition in Eq. II1.

Substituting the attention subcircuit equation V8 into equation V4, $y_n^{(2)}$ has 2 additional terms which result in a total of 6 inputs to MLP2 and 7 inputs to the linear unembedding. Including the key and query computations of each attention block as nodes, there are now $2 \cdot 1 + 2 + 2 \cdot 3 + 6 + 7 = 23$ edges on the graph.

[1] We do not explicitly count edges between blocks at different sequence positions

2. Circuit tracing

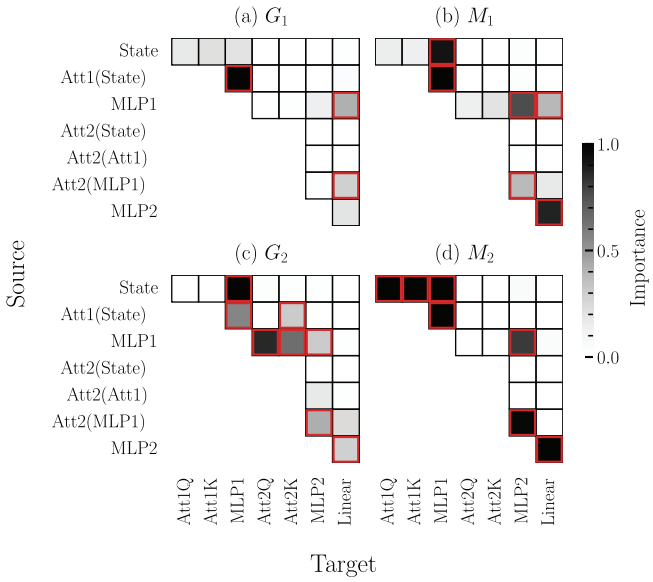

FIG. A4: Connectivity matrices between the circuit components as inferred from single-edge ablations. See Table I for the definitions of sources and targets. Dark squares indicate edges that demonstrate a large change in model behavior when they are ablated. Generalizing (memorizing) prototype circuits are traced from the $K = 1024$ ($K = 32$) models at the training checkpoint with lowest divergence to the corresponding predictor, with the exception of M_2 which is evaluated at the final checkpoint. Red boxes highlight edges which exceed the threshold importance of 0.25 for inclusion in the prototype circuits.

Circuit tracing followed a two-stage process in which prototype circuits were identified from single-edge ablations and then refined according to the performance of circuit-constrained models.

a. Circuit prototypes We first measured the importance of each connection in producing the observed transformer behavior. To do so, we developed a custom Python implementation of the model forward pass that explicitly exposes the vector passed along each block connection. This allows the passed vectors along all edges to be cached during a forward pass of the unperturbed model on a batch of 512 sequences. We then ablated each connection during additional forward passes on the same sequences by replacing the passed vector with the mean passed vector over the batch [56], and measured for each sequence the KL-divergence of the predictions from the unperturbed model $D_{\mathrm{KL}}^{S_N}(\hat{\pi}^\theta \,\|\, \tilde{\pi}^\theta)$, where $\tilde{\pi}^\theta$ are the perturbed model predictions. The batch average of this divergence forms the edge importance weights, so connections are given small weight if they can be ablated without significantly disrupting the behavior. This process is similar to the path patching method proposed in [57]. Prototype circuits for each phase were then formed by including only connections with (normalized) importance above a threshold of 0.25. The results of these tracing experiments are visualized as connectivity matrices of the transformer (Figure A4).

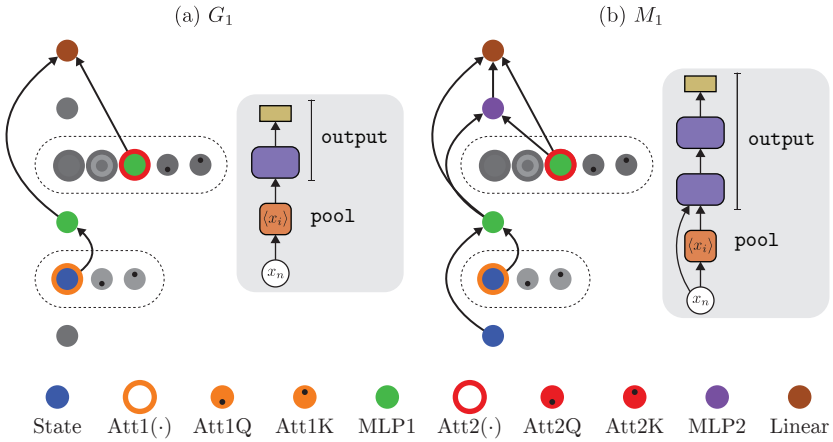

FIG. A5: Complete visualization and interpretive schematic of the traced circuits as in Figure 3 for the remaining two phases (a) G_1 and (b) M_1.

b. Circuit refinement The prototype circuits were then tested by simultaneously ablating all omitted edges (following the same batch-mean replacement scheme as earlier) and measuring the same importance score. In addition, perturbations to these prototypes were applied by adding/removing an edge to/from the circuit to verify that the proposed circuit was both sufficient and necessary. For the M_2 circuit, the State → Linear edge was found to be unnecessary and we removed it from the identified circuit. For the G_2 circuit, the MLP1 → MLP2 edge was identified as unnecessary and removed, and the Att2(MLP1) → Linear edge produced significant improvement and was added as a result. For the M_1 circuit, the Att2(MLP1) → Linear edge was added. No modifications to the prototype circuit for G_1 was necessary.

We interpret these final, refined circuits as follows.

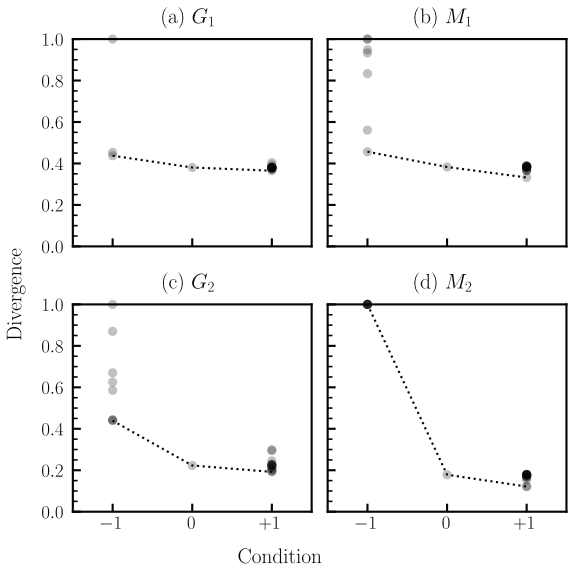

FIG. A6: Effect of perturbing the circuit-constrained model about the identified circuits. The divergence is normalized by the divergence expected for uniform prediction. Condition values indicate the number of edges either removed or added to the identified circuit. Dashed lines connect the minimum divergence for each condition, and values above 1 have been clipped. Edges in the identified circuits are necessary (divergence increases when any are removed) and sufficient (divergence is reasonably low, and does not improve much when any edge is added). The representative models correspond to those used for Figure A4.

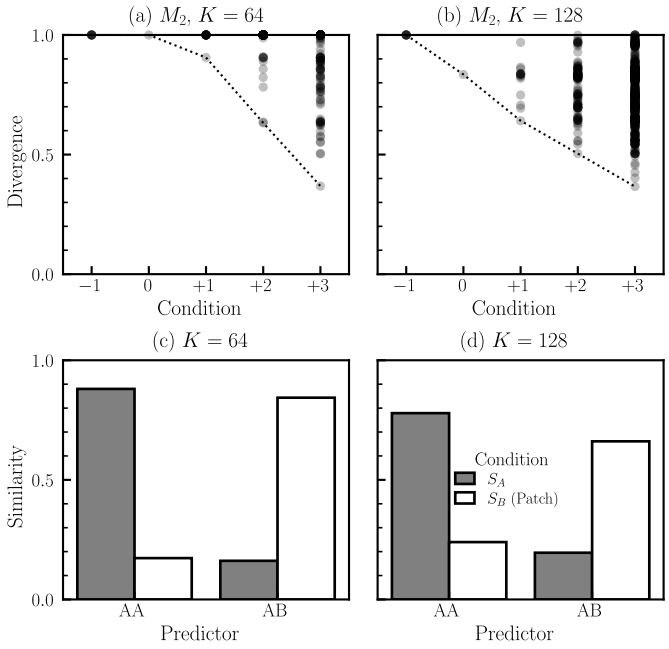

FIG. A7: (a, b) Results from the circuit perturbation analysis as in Figure A6 for M_2 at larger values of K. Models for $K = 64, 128$ do not show strong circuit-constrained performance, and adding edges produces only incremental improvement. (c, d) The task vector patching experiment as in Figure 4b for $K = 64$ and $K = 128$. The effect of task vector patching is retained for these M_2 models, despite the indicated circuit complications.

a. 1-Gen

The 1-Gen predictor requires an estimate of the empirical marginal distribution over states and produces predictions independent of the current state.

In G_1, the large value of $n_A^{(1)}$ (Figure 2c, bottom left) indicates that Att1 attends approximately uniformly over all positions in the sequence. Att1 thus computes an average over the sequence states, which effectively encodes the empirical 1-point statistics. A state-wise nonlinear operation which maps this representation to the output logits is sufficient to predict according to the stationary distribution.

The traced circuit (Figure A4(a)) indicates that MLP1 computes the logits for the prediction, with the 1-point statistics read directly from Att1. Since pathways downstream of the current state all have negligible importance, the logit is produced independent of the current state.

In summary, the transformer in G_1 functions as a simple pool-and-readout circuit implemented by the first layer, with pooling accomplished by Att1 and subsequent readout by MLP1 (Figure A5a).

b. 2-Gen

The 2-Gen predictor requires an estimate of the empirical frequencies of states which follow the current state μ, i.e. the quantities $\frac{m_{\tau\mu}}{n_\mu}$ for each τ as shown in equation III10.

In G_2, both $\phi_\delta^{(1)}$ and $\phi_\beta^{(2)}$ take on large values (Figure 2c, upper row). These order parameters indicate the selective attention of Att1 to the previous state and the selective attention of Att2 to positions in the context which follow the current state μ respectively. Together, this strongly signals the presence of a statistical induction head, a canonical transformer circuit motif that composes two transformer layers and offers a natural method to compute the empirical distribution needed by 2-Gen. We provide a detailed introduction to the induction head motif in Section VI. For now, we interpret how the traced circuit (Figure 3b) maps onto the two basic operations of a statistical induction head:

a. Pair construction. The selective attention of Att1 produces a residual-stream representation of the current and previous states. This representation alone could be sufficient to support the statistical induction head behavior. However, the traced circuit reveals the importance of State → MLP1 and Att1 → MLP 1 pathways. Thus, MLP1 may first denoise or otherwise map the 2-point representation to an embedding space.

b. Match and readout. The match and readout operation of a statistical induction head is often presumed to depend on the composition of the Att2 query with the current state and the Att2 key with the output of Att1, along with Att2 writing the filtered states to the residual stream [5, 7]. We would thus expect the pathways State → Att2Q, Att1 → Att2K, and Att2(State) → MLP2 to be of high importance. The traced circuit instead implicates the pathways MLP1 → Att2Q, MLP1 → Att2K, and Att2(MLP1) → MLP2 → Linear. Both constructions appear functionally degenerate, with the resulting operation aggregating the empirical transition frequencies conditioned on μ and MLP2 reading out this representation to produce the next-state logits.

In summary, the transformer in G_2 forms a statistical induction head through selective attention over embedded state pairs to produce the empirical distribution of transitions from the current state; the logit is then produced according to this distribution (Figure 3b).

c. 1-Mem

The 1-Mem predictor requires estimating the empirical 1-point statistics from the sequence and using those statistics to infer the latent task identity. Given the inferred task identity and the current state, the next-state probabilities are retrieved and used as predictions.

In M_1, the large value of $n_A^{(1)}$ (Figure 2c, bottom left) again indicates uniform attention of Att1; the output of Att1 thus encodes the empirical 1-point statistics. Given the current state and this encoding of the 1-point statistics, a state-wise nonlinear operation may be sufficient to predict according to the 1-Mem prescription.

The traced circuit (Figure A5b) indicates that MLP1 and MLP2 compose to produce the prediction, with the 1-point statistics entering via the Att1 → MLP1 path, and the current state information via the State → MLP1 path.

In summary, the transformer functions as a similar pool-and-readout circuit as in G_1. However, in M_1 both MLP blocks are recruited to produce the logit, and the prediction is made conditioned on the current state (Figure A5b).

d. 2-Mem

The 2-Mem predictor requires estimating the empirical 2-point statistics from the sequence and using those statistics to infer the latent task identity. Given the inferred task identity and the current state, the next-state probabilities are retrieved and used as predictions.

In M_2, the large value of $\phi_\delta^{(1)}$ (Figure 2c, top left) indicates the selective attention of Att1 to the previous state. This produces a representation of each sequence transition in the residual stream. Likewise, the large value of $n_A^{(2)}$ (Figure 2c, bottom right) indicates that Att2 produces a representation of information aggregated over the sequence. It is reasonable to expect that the logit may be produced by a state-wise operation given the current state and the aggregated sequence information.

The traced circuit (Figure 3c) reveals the exact nature of this computation. Each transition in the sequence is embedded nonlinearly by MLP1, which operates on the transition representation formed by State and Att1 in the residual-stream. These embeddings are aggregated by Att2 to produce a compact representation of the current task (see Section IV C for a discussion of this representation). MLP2 receives the current state via the MLP1 → MLP2 path, the inferred task identity via the Att2(MLP1) → MLP2 path, and produces the logits accordingly.

In summary, the M_2 transformer functions in two stages (Figure 3c). In the first stage, a compact representation of the inferred task is constructed by the pooling of nonlinear embedding of state pairs across the sequence. In the second stage, the logit is produced given the representation of the inferred task and the current state.

Note that significantly more edges must be included to recover performance for the $K = 64$ and 128 circuit-constrained models. (Figure A7a,b). Despite this, the general features of our M_2 circuit interpretation appear to hold in this regime, most importantly the task vector construction (Figure 4b,c). Overall, these results suggest that the M_2 circuit learned for $K_1^* < K < K_2^*$ continues to leverage the encoder-pool-decoder structure while exhibiting inter-block dependencies not observed for $K < K_1^*$.

VI: Statistical induction head

The only sequence information necessary to compute the 2-Gen prediction is the count $m_{\mu\tau}$ of occurrences of state τ that follow the current state μ (since the remaining quantity which normalizes the prediction is $n_\mu = \sum_\tau m_{\mu\tau}$). The *statistical induction head* is a simple circuit motif commonly observed in transformers that is capable of computing $m_{\mu\tau}$. Specifically, the statistical induction head computes the empirical conditional distribution $\hat{P}_n(x_{i+1} = \tau \mid x_i = \mu)$

where i ranges over the sequence length n. The sufficient circuit for this computation comprises two consecutive attention operations detailed below.

a. Pair construction. The first attention block attends preferentially to the previous state at each position to produce a representation of the current and previous state in the residual stream. The state embeddings for the input sequence lie in a subspace B_{state} of the residual stream, which is guaranteed to have dimension $\dim B_{\text{state}} \leq C$. Thus, the value matrices may readily rotate the state embeddings of attended positions to an orthogonal subspace when the model dimension D is large compared to the number of states C. This condition holds for our setting, and we consider the previous states as rotated into an orthogonal subspace B_1. The formation of state-pair representations via the B_1 subspace is depicted in Figure 5(d, ii), where the strength of the previous-state selectivity is summarized by the scalar δ.

b. Match and readout. The second attention block uses the contents of both B_{state} and B_1 to attend such that its output is precisely a vector representation of the empirical conditional distribution. To accomplish this, the attention block matches the current state (in B_{state}) to the previous state (in B_1) at each position. As a result, the attention weights concentrate on positions where the previous state matches the current state. The value matrix rotates the state in B_{state} to a third subspace B_2, and these new representations are then averaged over the matching positions to form a representation of the empirical conditional distribution in B_2. This representation is of the form $\sum_\tau \hat{P}_n(x_{i+1} = \tau \mid x_i = \mu) \cdot x'_\tau$ where the x'_τ are the rotated state embeddings residing in B_2. The operation of the second attention block is depicted in Figure 5(d, iii), where the strength of the match operation is summarized by the scalar β.

In principle, a linear transformation of the empirical conditional distribution is insufficient to implement the 2-Gen predictor, since the predictions must be in the form of log-probabilities. However, an MLP block which follows the two attention blocks may read the contents of B_2 and compute log-probabilities for the next-state prediction according to the 2-Gen prescription.

VII: Minimal network for generalization in the limit $K \to \infty$

A direct theoretical analysis of the full circuit in equations II6–II10 in Sec. II 2 is analytically intractable. To retain the essential computational mechanism revealed by the circuit-tracing results while achieving analytical tractability, we construct a symmetry-constrained, attention-only transformer model that preserves the generalization features of the full architecture. We henceforth call this model the *SA-transformer*. The SA-transformer is related to the disentangled transformer [58]. As detailed below, three key assumptions alongside symmetry arguments lead to a substantially reduced description. Although the argument is presented for a two-layer transformer with single-headed attention, the model can be extended naturally to architectures with multiple attention heads per layer and greater depth.

1. Reduction of the network and circuits

We first introduce simplifications at the architectural level.

(i) Fixed one-hot embeddings. Since sequence states are embedded as orthogonal one-hot vectors, we eliminate the embedding matrix W_E and directly work with the one-hot state representations.

(ii) Disentangled value subspaces. We assume that the value matrices $W_V^{(1)}$ and $W_V^{(2)}$ write their respective contributions into distinct, non-overlapping subspaces (either directly, or via MLP1 as seen in the traced G_2 circuit). This separation can be represented explicitly through concatenation. Under this assumption, MLP1 becomes redundant and can be removed from the model.

The resulting simplified network is

$$x_n^{(0)} = x_n, \tag{VII1}$$

$$y_n^{(1)} = \text{Att}^{(1)}\left(x_{\leq n}^{(0)}\right), \tag{VII2}$$

$$x_n^{(1)} = x_n^{(0)} \oplus y_n^{(1)}, \tag{VII3}$$

$$y_n^{(2)} = \text{Att}^{(2)}\left(x_{\leq n}^{(1)}\right), \tag{VII4}$$

$$x_n^{(2)} = x_n^{(1)} \oplus y_n^{(2)}, \tag{VII5}$$

where the final prediction is obtained by applying an MLP readout to $x_n^{(2)}$. We further introduce two assumptions at the level of the attention-mechanism.

(i) Relative positional bias. We replace rotary embeddings with a relative positional bias, which disentangles positional and content-based attention and facilitates analytical treatment.

(ii) Value-matrix elimination. Under the assumption of non-overlapping value subspaces, the value matrices $W_V^{(1)}$ and $W_V^{(2)}$ can be absorbed into the concatenated representation. As a result, they can be removed from equation II4 without altering the effective computation.

With the above considerations, we can write the output of the network as

$$\hat{\pi}_\tau = \frac{e^{z_\tau}}{\sum_{\tau'} e^{z_{\tau'}}}, \quad z = g\left(x_N \oplus \sum_{i\le N} A^{(1)}_{iN} x_i \oplus \sum_{i\le N} A^{(2)}_{iN} x_i \oplus \sum_{i\le N}\sum_{j\le i} A^{(2)}_{iN} A^{(1)}_{ji} x_j \right), \tag{VII6}$$

with

$$A^{(1)}_{ji} = \frac{e^{x_j^T M^{(1)} x_i + P^{(1)}_{j-i}}}{\sum_{j'\le i} e^{x_{j'}^T M^{(1)} x_i + P^{(1)}_{j'-i}}}, \quad A^{(2)}_{ji} = \frac{e^{\left(x^{(1)}_j\right)^T M^{(2)} x^{(1)}_i + P^{(2)}_{j-i}}}{\sum_{j'\le i} e^{\left(x^{(1)}_{j'}\right)^T M^{(2)} x^{(1)}_i + P^{(2)}_{j'-i}}}. \tag{VII7}$$

where we introduce $M^{(\ell)} = (K^T Q)^{(\ell)}$ for convenience, $P^{(l)}_{j-i}$ for relative positional bias and $\oplus$ denotes concatenation. Note that $A^{(2)}$ takes the output of the first layer $x^{(1)}_{1:N}$ as its input. With our simplifications, $x^{(1)}_i = x_i \oplus \sum_{j\le i} A^{(1)}_{ji} x_j$. Since $x^{(1)}_i$ is $2C$-dimensional, the query-key product in the second layer, $M^{(2)}$, has dimensions $2C \times 2C$. The non-linearity g maps the $4C$-dimensional input to a C-dimensional logit vector, which is then passed through a softmax layer to produce the final output.

Replacement of MLP2. Finally, since the x_i are one-hot vectors, each of the four contributions in equation VII6 are a C-dimensional vector with non-negative entries that sum to one. Each term can therefore be interpreted as a probability distribution over state classes, and thus represents a fully normalized predictive distribution for the next state. Under this observation, we can replace the nonlinear MLP readout g by a convex combination of these four distributions. This approximation amounts to restricting g to act linearly on the simplex spanned by the four contributions, thereby preserving normalization and interpretability while substantially reducing parameter complexity. With this approximation, the predicted probability distribution over next states becomes

$$\hat{\pi} = W_A x_N + W_B \sum_{i\le N} A^{(1)}_{iN} x_i + W_C \sum_{i\le N} A^{(2)}_{iN} x_i + W_D \sum_{i\le N}\sum_{j\le i} A^{(2)}_{iN} A^{(1)}_{ji} x_j, \tag{VII8}$$

where W_A, W_B, W_C, and W_D are matrices which are restricted such that the output still lies on the probability simplex (these are further simplified below).

2. Symmetry reduction

In our case, since the generative model for transition matrices T is symmetric over the C state classes, all C state classes are statistically identical. This symmetry implies that the four W matrices will have equal diagonal terms and equal off-diagonal terms. The off-diagonal terms can be set to zero as they contribute a constant offset; for example, if W_A has the form

$$\begin{bmatrix} w_{\mathrm{d}} & w_{\mathrm{od}} & \cdots & w_{\mathrm{od}} \\ w_{\mathrm{od}} & w_{\mathrm{d}} & \ddots & \vdots \\ \vdots & \ddots & \ddots & w_{\mathrm{od}} \\ w_{\mathrm{od}} & \cdots & w_{\mathrm{od}} & w_{\mathrm{d}} \end{bmatrix}, \tag{VII9}$$

then the first term on the right hand side of equation VII8 evaluates to $(w_{\mathrm{d}} - w_{\mathrm{od}})x_N + w_{\mathrm{od}}\mathbf{1}$, where $\mathbf{1}$ is a C-dimensional vector with all ones. This term can be ignored as it introduces a constant offset. We therefore set $w_{\mathrm{od}} = 0$, without loss in generality. Repeating this argument for W_B, W_C, W_D, we observe that the four W matrices can be replaced with non-negative scalar weights w_A, w_B, w_C, w_D that obey $w_A + w_B + w_C + w_D = 1$. In other words, we interpret the MLP g as implementing a mixture-of-experts operation as shown in equation VII10, where the four 'experts' correspond to the four terms in equation VII8.

A similar argument also applies to the matrices $M^{(1)}$ and $M^{(2)}$ in the attention head. The exponents involved in the attention weights $A^{(\ell)}$ contain vector-matrix-vector products of the form $x_i M x_j$ for an arbitrary D-dimensional matrix

M. This feature is apparent for $A^{(1)}$ from equation VII7. For $A^{(2)}$, since $u_i = x_i \oplus \sum_{j\le i} A^{(1)}_{ji} x_j$, the exponent will have four vector-matrix-vector products of the form $x_i M x_j$ and our symmetry argument below will apply independently to the corresponding four block matrices in $M^{(2)}$. Now, proceeding with the argument, since the x_i are one-hot vectors with non-zero elements in the first C components, only the top left $C \times C$ submatrix of M will influence this product. This implies that $M^{(1)}$ and the four block matrices in $M^{(2)}$ can be reduced to $C \times C$ matrices. The problem is then entirely independent of D.

Next, as argued for the W matrices, since the diagonal and off-diagonal elements respectively have identical gradients, all diagonal/off-diagonal elements that grow significantly beyond their initial values will be approximately equal. To be concrete, say these values are $\beta_{\rm d}$ and $\beta_{\rm od}$. A product of the form $x_i M x_j$ can be written as $\beta_{\rm d}\delta_{ij} + \beta_{\rm od}(1-\delta_{ij}) = (\beta_{\rm d} - \beta_{\rm od})\,\delta_{ij} + \beta_{\rm od}$. The constant term $\beta_{\rm od}$ appears both in the numerator and denominator of the softmax operation. The off-diagonal terms therefore cancel out. Thus, only the difference between the diagonal and non-diagonal element matters, and any such matrix M can be replaced with a diagonal matrix.

Combining these arguments leads to an SA-transformer with two layers and one head per layer:

$$\hat{\pi} = w_A x_N + w_B \sum_{i\le N} A^{(1)}_{iN} x_i + w_C \sum_{i\le N} A^{(2)}_{iN} x_i + w_D \sum_{i\le N}\sum_{j\le i} A^{(2)}_{iN} A^{(1)}_{ji} x_j, \text{where} \tag{VII10}$$

$$\log A^{(1)}_{ji} = \beta^{(1)}_1 \delta_{s_i s_j} + P^{(1)}_{j-i} - \log Z^{(1)},$$

$$\log A^{(2)}_{ji} = \beta^{(2)}_1 \delta_{s_i s_j} + \beta^{(2)}_2 \sum_{k\le i} A^{(1)}_{ki}\delta_{s_k s_j} + \beta^{(2)}_3 \sum_{k\le j} A^{(1)}_{kj}\delta_{s_i s_k} + \beta^{(2)}_4 \sum_{k\le i, k'\le j} A^{(1)}_{k'j} A^{(1)}_{ki} \delta_{s_k s_{k'}} + P^{(2)}_{j-i} - \log Z^{(2)},$$

where $Z^{(1)}, Z^{(2)}$ are normalization constants, $\beta^{(1)}_1$ is the scalar that parameterizes $M^{(1)}$, i.e., $M^{(1)} = \beta^{(1)}_1 \mathbf{I}_C$. And $\beta^{(2)}_1, \beta^{(2)}_2, \beta^{(2)}_3, \beta^{(2)}_4$ are four scalars that parameterize the matrix $M^{(2)}$, i.e.,

$$M^{(2)} = \begin{pmatrix} M^{(2)}_{11} & M^{(2)}_{12} \\ M^{(2)}_{21} & M^{(2)}_{22} \end{pmatrix} = \begin{pmatrix} \beta^{(2)}_1 \mathbf{I}_C & \beta^{(2)}_2 \mathbf{I}_C \\ \beta^{(2)}_3 \mathbf{I}_C & \beta^{(2)}_4 \mathbf{I}_C \end{pmatrix}. \tag{VII11}$$

where we partition $M^{(2)}$ into 4 $C \times C$ blocks. These modifications remove redundancy in the parameter space and substantially simplify the analysis by focusing on the irreducible parameters.

3. Numerical validation

a. *Training details*

Denote the last state in the sequence x_N as μ. The cross-entropy loss averaged over input sequences given transition matrix T is

$$\mathcal{L} = -\left\langle \sum_{\mu,\tau} p_\mu T_{\tau\mu} \log \pi_\tau(x_{1:N}) \right\rangle. \tag{VII12}$$

where $T_{\tau\mu}$ is the probability that the next state is τ given that the current state is μ and p_μ is the stationary probability of state μ for transition matrix T. Unless otherwise specified, the expectation $\langle . \rangle$ is over all possible sequences that could be drawn given T and the current state μ, averaged over transition matrices T. For the SA-transformer, we fix the sequence length N during optimization instead of using autoregressive training.

Without loss of generality, we choose the following prediction function to investigate the generalization phase:

$$\hat{\pi} = w_A x_N + w_B \sum_{i\le N} A^{(1)}_{iN} x_i + w_C \sum_{i\le N} A^{(2)}_{iN} x_i + w_D \sum_{i\le N}\sum_{j\le i} A^{(2)}_{iN} A^{(1)}_{ji} x_j, \quad \text{where} \tag{VII13}$$

$$A^{(1)}_{ji} = \frac{e^{x_j^T M^{(1)} x_i + P^{(1)}_{j-i}}}{\sum_{j'\le i} e^{x_{j'}^T M^{(1)} x_i + P^{(1)}_{j'-i}}}, \quad A^{(2)}_{ji} = \frac{e^{\left(x^{(1)}_j\right)^T M^{(2)} x^{(1)}_i + P^{(2)}_{j-i}}}{\sum_{j'\le i} e^{\left(x^{(1)}_{j'}\right)^T M^{(2)} x^{(1)}_i + P^{(2)}_{j'-i}}}.$$

The relative positional bias terms $P^{(\ell)}$ and the elements of the $M^{(\ell)}$ matrices (with $\ell = 1, 2$) are initialized at zero. To test our symmetry argument, we *do not* impose the constraint that the $M^{(\ell)}$ matrices have block-identity structure as in equation VII11. The constraints $w_A, w_B, w_C, w_D \ge 0, w_A + w_B + w_C + w_D = 1$ are imposed by expressing them using a softmax with three learnable parameters. We initialize $w_A = w_B = w_C = w_D = \frac{1}{4}$. The SA-transformer is trained using stochastic gradient descent (SGD) with learning rate 1 and a batch size of 256.

b. Training results

The training loss is shown in Figure 5b, and shows that the SA-transformer reproduces the abrupt learning of the full network (compare with Figure 2a for $K = 1024$, where the transformer rapidly learns the 1-Gen solution and subsequently transitions to G_2 after a plateau).

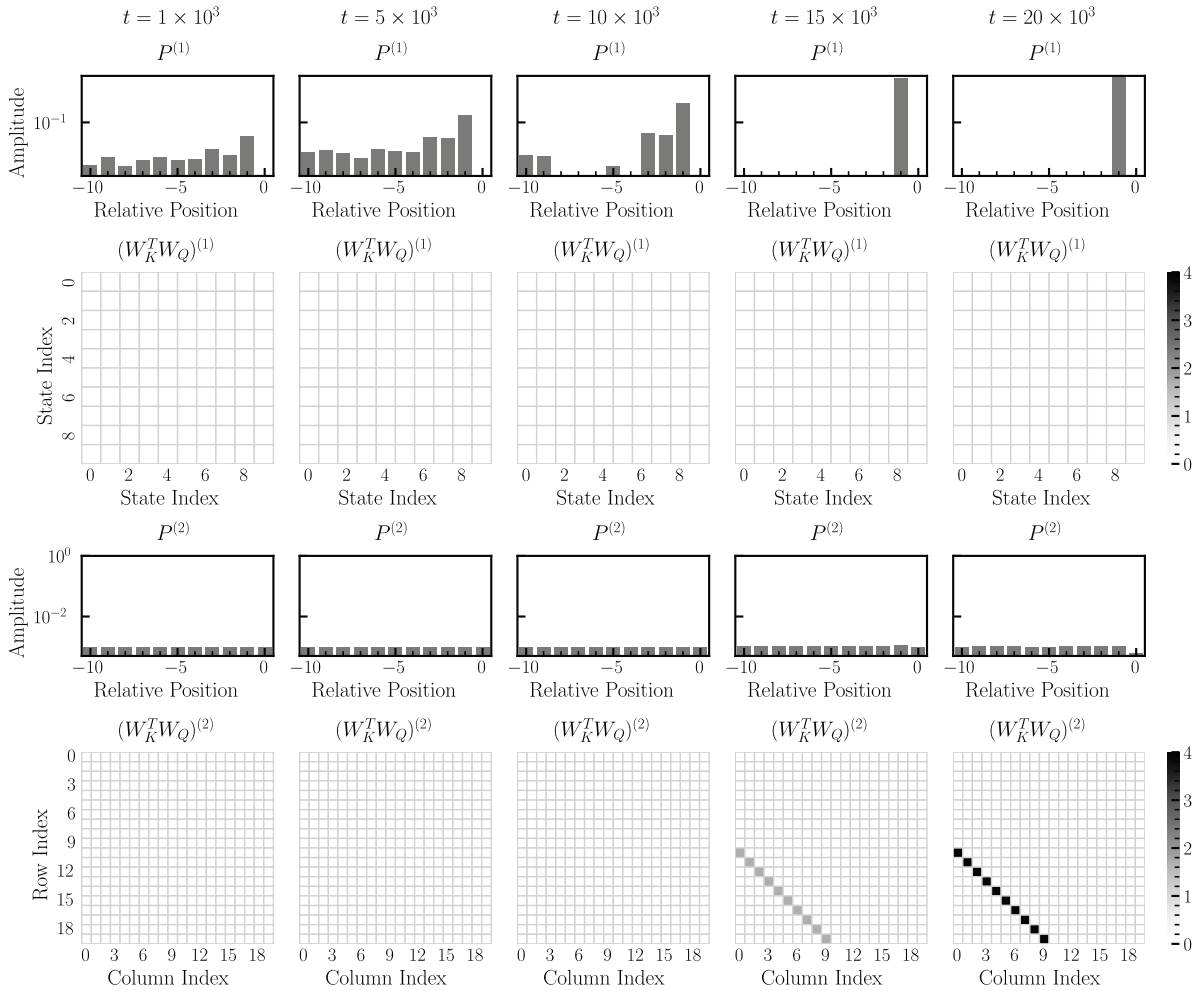

FIG. A8: Parameters of the attention circuits defined in equation VII13 at different iterations. The corresponding loss dynamics is given in Figure 5 (b).

In Figure A8, we further observe that $M^{(1)}$ in $A^{(1)}_{ij}$, $P^{(2)}$, and the upper-triangular blocks of $M^{(2)}$ in $A^{(2)}_{ij}$ remain unchanged when the parameters are initialized to zero. The equal increase of the diagonal elements of $M^{(2)}_{21}$ also demonstrates that our symmetry-reduction hypothesis holds well. Therefore, we can simplify the model described by equation VII10 to

$$\hat{\pi} = w_A x_N + w_B \sum_{i \le N} A^{(1)}_{iN} x_i + w_C \sum_{i \le N} A^{(2)}_{iN} x_i + w_D \sum_{i \le N} \sum_{j \le i} A^{(2)}_{iN} A^{(1)}_{ji} x_j, \quad \text{where} \tag{VII14}$$

$$\log A^{(1)}_{ji} = \delta \delta_{j(i-1)} - \log Z^{(1)}, \quad \log A^{(2)}_{ji} = \beta \sum_{k \le j} A^{(1)}_{kj} \delta_{s_i s_k} - \log Z^{(2)}. \tag{VII15}$$

To simplify notation, we define $\delta = P^{(1)}_{-1}$ and $\beta = \beta^{(2)}_3$. With these reductions, the minimal model is characterized by two parameters, β and δ, together with three independent weight parameters w_B, w_C, w_D.

VIII: Dynamics of the SA-transformer

We now examine in detail how the SA-transformer described with equation VII13 can implement the 1-Gen and 2-Gen solutions through the organization of functional circuits across different layers. The analysis is organized as follows:

1. The network first enters G_1 before entering G_2. We show that $w_A \to 0$ and $w_B = w_C = w_D = 1/3$ in G_1, whereas the rest of the parameters remain at zero.
2. Next, we show that G_2 corresponds to $w_C = 1, \beta \to \infty, \delta \to \infty$ in the SA-transformer. The other terms involving w_A, w_B, w_D do not contribute to the solution after convergence.
3. Third, we show that accurately capturing the training dynamics of β, δ requires a careful computation of expectations when expanding the loss in a Taylor series near the 1-Gen solution. While the 2-Gen solution involves a second-order term in the loss of the form $\beta\delta$ (which would imply a saddle-point at the origin $\beta, \delta = 0$), there are subtle first-order contributions that have an important influence on the dynamics of β, δ. One of these contributions arises from the term involving w_B, even though this term does not matter after convergence. Importantly, these subtle first-order contributions determine the duration of the loss plateau while the network transitions from G_1 to G_2 (Figure 5b).

1. Competition between x_N and the 1-Gen solution

The network nearly implements the 1-Gen solution at initialization except for a contribution due to the first term involving w_A in equation VII14. At initialization, we have $A^{(\ell)}_{ji} = 1/i$ for $\ell = 1, 2$. Plugging this into equation VII14, we observe that the terms involving w_B, w_C, w_D compute the 1-point statistics. Since these three terms have identical contributions, we have $w_B = w_C = w_D = (1 - w_A)/3$. The term $w_A x_N$ leads to a bias towards repeating the final state class μ. We can write the output of the network as

$$\hat{\pi}_\tau = w_A \delta_{\mu\tau} + (1 - w_A) p_\tau. \tag{VIII1}$$

The gradient descent dynamics of w_A is

$$\begin{aligned} \frac{dw_A}{dt} &= \left\langle \sum_{\mu,\tau} \left(\frac{p_\mu T_{\tau\mu}}{w_A \delta_{\mu\tau} + (1 - w_A) p_\tau} \delta_{\mu\tau} - \frac{p_\mu p_\tau T_{\tau\mu}}{w_A \delta_{\mu\tau} + (1 - w_A) p_\tau} \right) \right\rangle \\ &= \left\langle \sum_{\mu} \frac{p_\mu T_{\mu\mu}(1 - p_\mu)}{p_\mu + w_A(1 - p_\mu)} - \sum_{\mu,\tau;\mu\neq\tau} \frac{p_\mu T_{\tau\mu}}{(1 - w_A)} \right\rangle \equiv \mathcal{F}(w_A). \end{aligned} \tag{VIII2}$$

It is obvious that when $w_A \to 1$, the $-\frac{1}{1-w_A}$ term is negative and diverges, driving the decrease of w_A. Since $\mathcal{F}(0) = 0$ and $\mathcal{F}(w_A)$ monotonically decreases with w_A when $p_\mu < 1$ for $0 < w_A < 1$, $\mathcal{F}(w_A)$ is strictly negative throughout his interval $0 < w_A < 1$. We can estimate the decay time as

$$s_d \approx -\int_0^1 \frac{1}{\mathcal{F}(w_A)} dw_A. \tag{VIII3}$$

Since $\mathcal{F}(w_A)$ decreases monotonically with w_A, we use the dynamics at $w_A \ll 1$ to obtain an upper bound on s_d. In this regime,

$$\begin{aligned} \frac{dw_A}{dt} &\approx -\left\langle \sum_{\mu} \frac{(1 - p_\mu)^2 T_{\mu\mu}}{p_\mu} + \sum_{\tau\neq\mu} p_\mu T_{\tau\mu} \right\rangle w_A \\ &= -\left\langle 1 + \sum_{\mu} \frac{(1 - p_\mu)^2 T_{\mu\mu}}{p_\mu} - p_\mu T_{\mu\mu} \right\rangle w_A \\ &= -\left\langle 1 + \sum_{\mu} \left(\frac{T_{\mu\mu}}{p_\mu} - 2T_{\mu\mu} \right) \right\rangle w_A \\ &= -\left\langle -1 + \sum_{\mu} \frac{T_{\mu\mu}}{p_\mu} \right\rangle w_A \end{aligned} \tag{VIII4}$$

Typically, $T_{\mu\mu}$ and p_μ are of the same order, so that $dw_A/dt \approx -Cw_A$, i.e., w_A decays rapidly to zero as shown in Figure A9. The network thus predicts the 1-point statistics, $\hat{\pi}_\tau = p_\tau$.

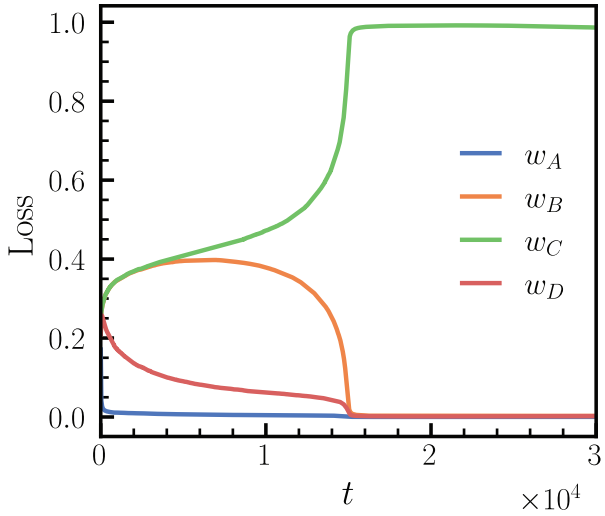

FIG. A9: Training dynamics of the mixture-of-experts weights in equation VII13.

2. The 2-Gen solution after convergence

Now, we show that 2-Gen can be implemented by setting all other parameters except w_C, δ, β to zero. Specifically, $w_C = 1, \delta \to \infty, \beta \to \infty$ corresponds to the 2-Gen solution. Recall that, by definition, $\delta = P^{(1)}_{-1}$ and $\beta = \beta^{(2)}_3$. When all other parameters are set to zero, from equation VII14, we have

$$\hat{\pi} = \sum_{i \le N} A^{(2)}_{iN} x_i, \quad \text{where} \tag{VIII5}$$

$$A^{(1)}_{ji} = \frac{\left(e^\delta - 1\right)\delta_{j(i-1)} + 1}{i + e^\delta - 1}, \quad A^{(2)}_{iN} = \frac{e^{\beta \sum_{j \le i} A^{(1)}_{ji} \delta_{\mu s_j}}}{\sum_{i' \le N} e^{\beta \sum_{j' \le i'} A^{(1)}_{j'i'} \delta_{\mu s_{j'}}}}. \tag{VIII6}$$

$\hat{\pi}_\tau$ can be written in terms of β and δ as

$$\hat{\pi}_\tau = \sum_{i \le N} \frac{\exp\left(\beta \frac{(e^\delta - 1)\delta_{s_{i-1}\mu} + k_{\mu i}}{i + e^\delta - 1}\right)}{\sum_{i' \le N} \exp\left(\beta \frac{(e^\delta - 1)\delta_{s_{i'-1}\mu} + k_{\mu i'}}{i' + e^\delta - 1}\right)} \delta_{s_i \tau}, \tag{VIII7}$$

where $k_{\mu i} = \sum_{j \le i} \delta_{s_j \mu}$ is the number of occurrences of μ in the first i states. In the limit $\delta \to \infty$, $A^{(1)}_{ji} = \delta_{j(i-1)}$, i.e., the first attention head pays attention to the previous state, so that

$$\hat{\pi}_\tau = \sum_{i \le N} \frac{e^{\beta \delta_{s_{i-1}\mu}}}{\sum_{i' \le N} e^{\beta \delta_{s_{i'-1}\mu}}} \delta_{s_i \tau}, \tag{VIII8}$$

When $\beta \to \infty$, the denominator in the above expression counts the number of occurrences of μ, whereas the numerator counts the number of co-occurrences of μ and τ (in that order). Thus, $\hat{\pi}_\tau = m_{\tau\mu}/n_\mu$, which is the 2-Gen prediction with the regularizing α term omitted.

Figure A9 shows that w_A decays rapidly, since the w_A term corresponds to predicting by simply repeating the last state. The remaining three terms initially yield the one-point prediction. The contribution associated with w_D becomes progressively worse than the one-point prediction, and consequently w_D decays. In contrast, the contribution associated with w_B remains equivalent to the one-point prediction and stays competitive with w_C until the w_C term converges to the two-point solution.

3. Acquisition of the 2-Gen solution

We now examine the kinetics of acquisition of the 2-Gen solution $w_C = 1, \beta \to \infty, \delta \to \infty$. To do this, we expand the loss to first order in β and δ around the unigram solution,

$$\mathcal{L}(\beta, \delta) = \mathcal{L}^{\text{1-Gen}} - c_\beta \beta - c_\delta \delta + \dots. \tag{VIII9}$$

Recall from Section VIII 1 that the 1-Gen solution corresponds to $w_A = 0, w_B = w_C = w_D = 1/3$ and the rest of the parameters equal to zero.

a. *First-order contributions in δ*

First-order contributions to the loss in δ appear in the second and fourth terms in equation VII14 (w_B and w_D). When computing $\hat{\pi}_\tau$, both terms have sums of the form $\sum_{j \le i} A^{(1)}_{ji} \delta_{s_j \tau}$. To first order, this sum evaluates to

$$\sum_{j \le i} A^{(1)}_{ji} \delta_{s_j \tau} = \frac{k_{\tau i}}{i} + \frac{\delta}{i} \left(\delta_{s_{i-1} \tau} - \frac{k_{\tau i}}{i} \right). \tag{VIII10}$$

For large i, $k_{\tau i}/i \to p_\tau$. However, the expectation of $\langle \delta_{s_{i-1}\tau} \rangle - p_\tau$ over all sequences *given* the transition matrix T and $s_N = \mu$ is not zero. Specifically, $\langle \delta_{s_{i-1}\tau} \rangle = P(s_{i-1} = \tau | s_N = \mu) = (T^{N-i+1})_{\mu\tau} p_\tau / p_\mu$, where Bayes' rule has been applied to obtain the latter expression. When $N - i$ is large compared to the mixing time of the Markov chain, $(T^{N-i+1})_{\mu\tau} \to p_\mu$, and the expression $(T^{N-i+1})_{\mu\tau} p_\tau / p_\mu - p_\tau$ can be neglected. However, this difference cannot be ignored for small values of $N - i + 1$.

Taking the derivative of the loss w.r.t δ and setting $A^{(2)}_{iN} = 1/N$ in equation VII14, we get

$$\begin{aligned}
c_\delta &= \left\langle \sum_{\mu,\tau} \frac{p_\mu T_{\tau\mu}}{p_\tau} \left(\frac{w_B}{N} \left(\frac{T_{\mu\tau} p_\tau}{p_\mu} - p_\tau \right) + \frac{w_D}{N} \sum_{i \le N} \frac{1}{i} \left(\frac{T^{N-i+1}_{\mu\tau} p_\tau}{p_\mu} - p_\tau \right) \right) \right\rangle, \\
&= \left\langle \sum_{\mu,\tau} T_{\tau\mu} \left(\frac{w_B}{N} \left(T_{\mu\tau} - p_\mu \right) + \frac{w_D}{N} \sum_{i \le N} \frac{1}{i} \left(T^{N-i+1}_{\mu\tau} - p_\mu \right) \right) \right\rangle, \\
&= \left\langle \sum_{\mu} \left(\frac{w_B}{N} \left(T^2_{\mu\mu} - 1 \right) + \frac{w_D}{N} \sum_{i \le N} \frac{1}{i} \left(T^{N-i+2}_{\mu\mu} - 1 \right) \right) \right\rangle, \\
&= \frac{w_B F_1}{N} + \frac{w_D}{N} \sum_{i \le N} \frac{F_{N-i+1}}{i},
\end{aligned} \tag{VIII11}$$

where the angled brackets now represent an expectation over the Dirichlet ensemble of transition matrices, and we have defined $F_d = \left\langle \sum_\mu T^{d+1}_{\mu\mu} \right\rangle - 1$. In Figure A10, we show numerically obtained values of F_d for $C = 10, \alpha = 1$. F_d is positive and decays approximately exponentially with d, which implies that the second term on the right hand side of equation VIII11 can be ignored when i is closed to 1, while the second term can also be ignored as it becomes $\sim \mathcal{O}(1/N^2)$ when i is closed to N. For $d = 1$, we obtain an analytical expression:

$$F_1 = \left\langle \sum_{\tau, \mu \neq \tau} T_{\tau\mu} T_{\mu\tau} \right\rangle + \langle T_{\tau\tau} T_{\tau\tau} \rangle - 1 = \frac{C - 1}{C^2 \alpha + C}. \tag{VIII12}$$

Therefore, we have $c_\delta \approx w_B F_1 / N$.

b. *First-order contributions in β*

First-order contributions to the loss in β appear in the third and fourth terms (w_C and w_D) in equation VII14. Both terms involve $A^{(2)}_{iN}$, which to first order in β can be written as

$$A^{(2)}_{iN} = \frac{e^{\beta k_{\mu i}/i}}{\sum_{j \le N} e^{\beta k_{\mu j}/j}} \approx \frac{1}{N} + \frac{\beta}{N} \left(\frac{k_{\mu i}}{i} - \frac{1}{N} \sum_{j \le N} \frac{k_{\mu j}}{j} \right). \tag{VIII13}$$

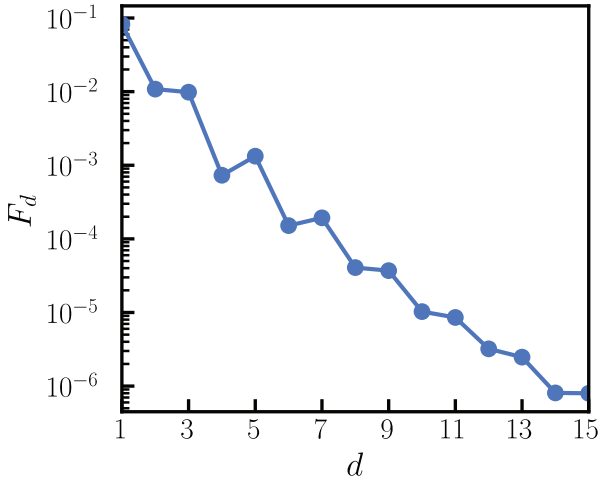

FIG. A10: Scaling of F_d with different d for $C = 10, \alpha = 1$.

It is tempting to assume that $k_{\mu i}/i \to p_\mu$ for large i and neglect the term in the parenthesis above. However, this term has a non-negligible contribution once the expectation given a transition matrix T and $s_N = \mu$ is computed. In particular, focusing on the expected contribution (given a transition matrix T and $s_N = \mu$) to $\hat{\pi}_\tau$ from the w_C term in equation VII14, we get an expression

$$\frac{k_{\tau N}}{N} + \frac{\beta}{N}\sum_{i\le N}\left\langle\left(\frac{\sum_{j\le i}\delta_{s_j\mu}}{i} - p_\mu\right)\delta_{s_i\tau}\right\rangle \approx \frac{k_{\tau N}}{N} + \frac{\beta}{N}\sum_{i\le N, j\le i}\left(\frac{\langle\delta_{s_j\mu}\delta_{s_i\tau}\rangle}{i} - p_\mu p_\tau\right) \tag{VIII14}$$

$$\approx \frac{k_{\tau N}}{N} + \frac{\beta}{N}\sum_{i\le N, j\le i}\left(\frac{p_\mu T^{i-j}_{\tau\mu}}{i} - p_\mu p_\tau\right), \tag{VIII15}$$

where we have approximated $P(s_i = \tau | s_N = \mu) \approx p_\tau$, $P(s_i = \tau, s_j = \mu | s_N = \mu) \approx p_\mu T^{i-j}_{\tau\mu}$. When $i \gg j$, $T^{i-j}_{\tau\mu} \approx p_\tau$. Approximating $T^{i-j}_{\tau\mu} \approx p_\tau$ for $i \ge j + 2$, we get

$$\frac{k_{\tau N}}{N} + \frac{\beta}{N}\sum_{i\le N, j\le i}\left(\frac{p_\mu T^{i-j}_{\tau\mu}}{i} - p_\mu p_\tau\right) \approx \frac{k_{\tau N}}{N} + \frac{\beta}{N} H_N\left(p_\mu T_{\tau\mu} + p_\mu\delta_{\tau\mu} - 2p_\mu p_\tau\right), \tag{VIII16}$$

where the harmonic number $H_N = \sum_{i\le N} 1/i \approx \log N + \gamma$ for large N ($\gamma \approx 0.577$ is the Euler-Mascheroni constant).

We omit the calculation for the w_D term noting in brief that we obtain an expression of the form

$$\frac{k_{\tau N}}{N} + \frac{\beta}{N}\sum_{i\le N}\left(\frac{k_{\mu i}}{i} - p_\mu\right)\left(\frac{k_{\tau i}}{i}\right). \tag{VIII17}$$

The latter expressions self-average, and though the expectation is non-zero, its contribution is small relative to the contribution from the w_C term.

Taking the derivative of the loss w.r.t β and using the above arguments, we get

$$c_\beta \approx w_C \frac{H_N}{N}\left\langle\sum_{\mu,\tau}\frac{p_\mu T_{\tau\mu}}{p_\tau}\left(p_\mu T_{\tau\mu} + p_\mu\delta_{\tau\mu} - 2p_\mu p_\tau\right)\right\rangle, \tag{VIII18}$$

$$\approx w_C \frac{H_N}{N}\left(\left\langle\sum_\mu\left(p_\mu T_{\mu\mu} - p_\mu^2\right)\right\rangle + \left\langle\sum_{\mu,\tau}\left(p_\mu^2 T^2_{\tau\mu}/p_\tau - p_\mu^2 T_{\tau\mu}\right)\right\rangle\right). \tag{VIII19}$$

The second expectation above (denoted I) is positive for any T, i.e.,

$$I = \left\langle\sum_\mu p_\mu^2\left(\sum_\tau\frac{(T_{\tau\mu})^2}{p_\tau} - 1\right)\right\rangle \ge 0. \tag{VIII20}$$

To show this, we use the Cauchy-Schwarz inequality $(x.y)^2 \leq ||x||^2||y||^2$ for two vectors x and y to prove $\sum_\tau (T_{\tau\mu})^2/p_\tau \geq 1$:

$$1 = \left(\sum_\tau \sqrt{p_\tau}\,\frac{T_{\tau\mu}}{\sqrt{p_\tau}}\right)^2 \leq \left(\sum_\tau p_\tau\right)\left(\frac{(T_{\tau\mu})^2}{p_\tau}\right) = \sum_\tau \frac{(T_{\tau\mu})^2}{p_\tau}. \tag{VIII21}$$

We were unable to show that the first expectation $\left\langle \sum_\mu \left(p_\mu T_{\mu\mu} - p_\mu^2\right)\right\rangle$ in c_β is always non-negative, though we find numerically that this term is negligible compared to I for the Dirichlet ensemble. To summarize,

$$c_\beta \approx w_C \frac{H_N}{N} I \approx \frac{\log N}{N} I, \quad \text{where } I = \left\langle \sum_\mu p_\mu^2 \left(\sum_\tau \frac{(T_{\tau\mu})^2}{p_\tau} - 1 \right) \right\rangle. \tag{VIII22}$$

c. *Dynamics near $\beta, \delta = 0$*

In our numerical experiments (described further below), we perform a perturbation that effectively sets $c_\delta = 0$. In this case, β increases due to the contribution from c_β and δ increases at a slower rate due to its coupling with β. To derive this scaling, we calculate the loss to the lowest-order coupling term between β and δ (that is, $O(\beta\delta)$). This contribution arises from the w_C term. We omit the calculation here; we find

$$\mathcal{L}(\beta, \delta) = \mathcal{L}^{\text{1-Gen}} - \beta(1+\delta) w_C \frac{\log N}{N} I - \delta w_B \frac{F_1}{N}. \tag{VIII23}$$

For small values of β, δ, changes in w_B, w_C remain very small. Consequently, w_B, w_C increase gradually during training until β and δ rapidly increase in the nonlinear regime and w_c eventually dominates.

In summary, the approximate gradient descent dynamics of β and δ in our theory is given by

$$\begin{aligned} &\frac{d\beta}{dt} \approx \frac{\log N}{N} w_C I, \quad \frac{d\delta}{dt} \approx \frac{\log N}{N} w_C \beta I + \frac{1}{N} w_B F_1, \\ &I = \left\langle \sum_\mu p_\mu^2 \left(\sum_\tau \frac{(T_{\tau\mu})^2}{p_\tau} - 1 \right) \right\rangle, \quad F_1 = \frac{C-1}{C^2\alpha + C}, \end{aligned} \tag{VIII24}$$

and $w_B = w_C = 1/3$ (to lowest order).

4. Testing predictions

a. *Loss landscape*

Through above analysis in previous sections, we could express the model as:

$$\hat{\pi}_\tau = w_A \delta_{\mu\tau} + w_B \sum_{i \leq N} A_{iN}^{(1)} \delta_{\tau s_i} + w_C \sum_{i \leq N} A_{iN}^{(2)} \delta_{\tau s_i} + w_D \sum_{i \leq N} \sum_{j \leq i} A_{iN}^{(2)} A_{ji}^{(1)} \delta_{\tau s_j}, \quad \text{where} \tag{VIII25}$$

$$A_{ji}^{(1)} = \frac{\left(e^\delta - 1\right)\delta_{j(i-1)} + 1}{i + e^\delta - 1} \quad \text{and} \quad A_{ji}^{(2)} = \frac{\exp\left(\beta \sum_{k \leq j} A_{kj}^{(1)} \delta_{s_i s_k}\right)}{\sum_{j'} \exp\left(\beta \sum_{k' \leq j'} A_{k'j'}^{(1)} \delta_{s_i s_{k'}}\right)}. \tag{VIII26}$$

Thus the loss landscape can be parameterized by five variables. In order to reduce it into the two parameters, δ and β, we assume that the MLP rapidly optimize over the weights of four components. Therefore, for each fixed pair (δ, β), the weights $w_{A\text{–}D}$ are determined by minimizing the loss. The resulting loss landscape predicted by our minimal model is shown in Figure 6(a). The flat region at the origin corresponds to the first plateau in the training loss curve in Figure 5(b). Our analytical model predicts that, when $\beta, \delta \sim 1$, there is a sharp cliff in the landscape that triggers abrupt learning. After the rapid transition, it reaches another flat region that corresponds to the second, final plateau in the training loss curve.

5. Ablating the first-order contribution in δ

In equation VIII24, the dynamics of δ is governed by two terms of which only the first depends on β. If we set $F_1 = 0$, the dynamics change qualitatively. F_1 is non-zero due to subtle correlations between s_{N-1} and s_{N+1}. We remove these correlations by resampling the state at position $N-1$ after generating the sequence, thereby ablating the first order term in δ. Resampling s_{N-1} breaks the short-range correlation $P(s_{N-1} = \tau, s_{N+1} = \tau | s_N = \mu)$ while preserving long-range statistics.

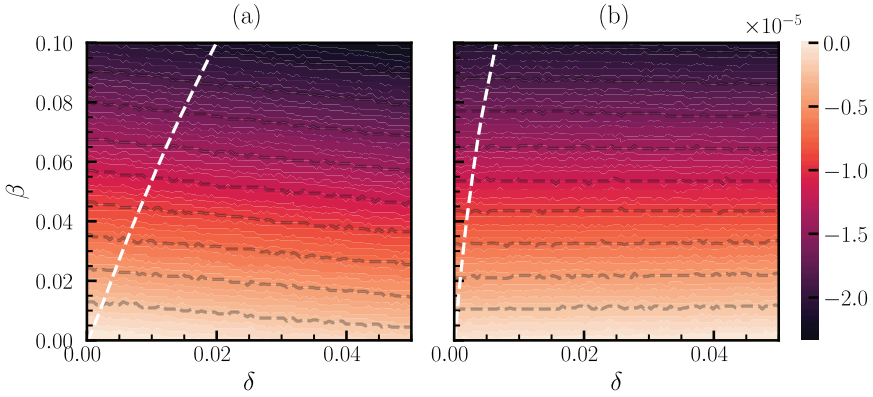

FIG. A11: Loss landscapes near $\delta, \beta \to 0$ for (a) original data and (b) data with the second-to-last state bias removed. The white dashed line is the corresponding dynamics of β and δ.

Figure A11a and A11b compares the loss landscape near the origin ($\beta, \delta \sim 0$) for the original and ablated cases, with optimal w_{A-D} for given β, δ. Our theory predicts the following behavior: in the original scenario, the loss decreases along both β and δ. In contrast, after ablating the first order contribution in δ, the landscape remains flat when $\beta = 0$. The loss decreases appreciably along the positive δ direction only once β becomes non-negligible. This accounts for the scaling of δ observed in the minimal neural network (Figure 6 b), where δ grows on average but still fluctuates until β becomes non-negligible.

Moreover, we can derive scaling relations directly from equation VIII24. In the original scenario, the β term in the δ dynamics can be neglected. In this case, we expect $\beta \sim t, \delta \sim t$. In contrast, when the first-order contribution in δ is ablated, we expect $\beta \sim t, \delta \sim t^2$. Thus, while β scales linearly with time in both cases, δ grows linearly in the original scenario but quadratically in the perturbed scenario. This distinct scaling behavior is consistent with the dynamics observed in numerical simulations using the SA-transformer described by equation VII13 (see Figure 6b).

6. The time to transition from the 1-Gen to the 2-Gen solution

Finally, a key result of our theory is an estimate for the number of iterations required to transition from the 1-Gen to the 2-Gen solution, which we call $\tau_{\text{2-Gen}}$. From equation VIII24, β grows at a constant rate until it reaches the nonlinear regime, after which it increases rapidly and saturates shortly thereafter. Since β grows more rapidly than δ, the dynamics of β predominantly determine $\tau_{\text{2-Gen}}$. This leads to the scaling law,

$$\tau_{\text{2-Gen}} \sim \frac{N}{\log N}, \tag{VIII27}$$

which is in excellent agreement with the behavior of the minimal network described by equation VII13 and the full transformer, as shown in Figure 6c.

7. Beyond the SA-Transformer

We next examine whether the predictions derived for the SA-transformer extend to a less constrained architecture. We restore learned state embeddings, query, key, and value matrices, and a final nonlinear MLP readout, while retaining the separation of the original state representation and the first-layer output into distinct subspaces. We compare relative positional bias (RPB) and rotary positional embedding (RoPE)[52], fixed-context and autoregressive

(AR) training, and SGD and AdamW optimization. These comparisons test both the context-length dependence of the transition time and the drift dynamics underlying the transition.

a. *Architecture*

Let $x_n \in \mathbb{R}^C$ denote the one-hot representation of the state at position n. We define the embedding matrix as

$$W_E = \frac{1}{\sqrt{D}} E \in \mathbb{R}^{D\times C}, \tag{VIII28}$$

where E is trainable and its entries are initialized independently from $\mathcal{N}(0,1)$. Generalizing equation VII5, the network is

$$\begin{aligned} x_n^{(0)} &= W_E x_n, \\ y_n^{(1)} &= \mathrm{Att}^{(1)}\left(x_{\le n}^{(0)}\right), \\ x_n^{(1)} &= x_n^{(0)} \oplus y_n^{(1)}, \\ y_n^{(2)} &= \mathrm{Att}^{(2)}\left(x_{\le n}^{(1)}\right), \\ x_n^{(2)} &= x_n^{(1)} \oplus y_n^{(2)}, \\ \hat{\pi}_n &= \mathrm{softmax}\left[\varphi\left(x_n^{(2)}\right)\right]. \end{aligned} \tag{VIII29}$$

Here $\oplus$ denotes concatenation, so that $x_n^{(0)} \in \mathbb{R}^D$, $x_n^{(1)} \in \mathbb{R}^{2D}$, and $x_n^{(2)} \in \mathbb{R}^{4D}$. The readout $\varphi : \mathbb{R}^{4D} \to \mathbb{R}^C$ is an MLP with two hidden layers of widths 512 and 256, GELU activations, and a final linear map to the logits. The architecture applies no layer normalization and has no MLP between the two attention layers.

Both attention layers are single-head. Let $h_i \in \mathbb{R}^{D_\ell}$ denote the input to layer ℓ at position i, with $h_i = x_i^{(0)}$ for $\ell = 1$ and $h_i = x_i^{(1)}$ for $\ell = 2$. The query, key, and value vectors are obtained through learned linear maps without biases:

$$q_i^{(\ell)} = W_Q^{(\ell)} h_i, \qquad k_i^{(\ell)} = W_K^{(\ell)} h_i, \qquad v_i^{(\ell)} = W_V^{(\ell)} h_i. \tag{VIII30}$$

Following Sec. II 2, the first index of $A_{ij}^{(\ell)}$ denotes the query position i and the second denotes the attended (key) position j. The attention weights and output are

$$\mathrm{Att}^{(\ell)}(h_{\le i}) = \sum_{j\le i} A_{ij}^{(\ell)} v_j^{(\ell)}, \quad \text{with } A_{ij}^{(\ell)} = \frac{\exp\left(s_{ij}^{(\ell)}\right)}{\sum_{j'\le i}\exp\left(s_{ij'}^{(\ell)}\right)}. \tag{VIII31}$$

The matrix dimensions are

$$W_Q^{(\ell)}, W_K^{(\ell)} \in \mathbb{R}^{d_\ell \times D_\ell}, \qquad W_V^{(\ell)} \in \mathbb{R}^{D_\ell \times D_\ell}, \tag{VIII32}$$

where $D_2 = 2D_1 = 2D, d_2 = 2d_1 = d$,with $D = d = 64$ in our experiments.

For RPB, the attention score is

$$s_{ij}^{(\ell)} = \left(q_i^{(\ell)}\right)^{\mathsf{T}} k_j^{(\ell)} + P_{j-i}^{(\ell)}, \tag{VIII33}$$

where $P_r^{(\ell)}$ is a learned bias indexed by the signed displacement $r = j - i$ and initialized to zero. The previous-state offset therefore corresponds to $r = -1$. Keeping the definition $M^{(\ell)} = (W_K^{(\ell)})^{\mathsf{T}} W_Q^{(\ell)}$ from equation VII7, the attention score can equivalently be written as $h_j^{\mathsf{T}} M^{(\ell)} h_i$.

For RoPE [52], position-dependent rotations replace the additive positional bias:

$$\begin{aligned} s_{ij}^{(\ell)} &= \left(R_i^{(\ell)} q_i^{(\ell)}\right)^{\mathsf{T}} \left(R_j^{(\ell)} k_j^{(\ell)}\right) \\ &= \left(q_i^{(\ell)}\right)^{\mathsf{T}} R_{j-i}^{(\ell)} k_j^{(\ell)}. \end{aligned} \tag{VIII34}$$

Here $R_i^{(\ell)} \in \mathbb{R}^{d_\ell \times d_\ell}$ is block diagonal, with each 2×2 block rotating each pair of elements by the angle $i\omega_a^{(\ell)}$, where

$$\omega_a^{(\ell)} = 10000^{-2a/d_\ell}, \qquad a = 0, \ldots, \frac{d_\ell}{2} - 1. \tag{VIII35}$$

The second equality in equation VIII34 follows from $(R_i^{(\ell)})^\mathsf{T} R_j^{(\ell)} = R_{j-i}^{(\ell)}$. Thus, the explicit positional dependence enters through the relative displacement $j - i$. Rotations are applied to queries and keys in both layers, while the value vectors remain unrotated. The frequency sets are determined separately for each layer through d_ℓ.

At initialization, the attention matrices have independent Gaussian entries:

$$W_\nu^{(\ell)} \overset{\text{iid}}{\sim} \mathcal{N}\left(0, \frac{\sigma^2}{2D_\ell}\right), \qquad \nu \in \{Q, K, V\}. \tag{VIII36}$$

The parameter σ controls the initialization scale of the attention matrices in both layers.

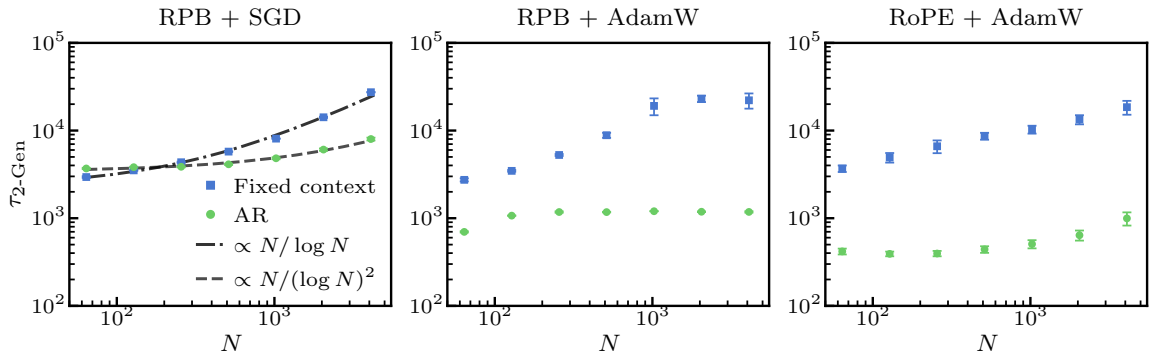

FIG. A12: Context-length dependence of the transition time beyond the SA-transformer. The network has two attention layers with learned embeddings, query, key, and value matrices, and a final MLP readout. Left: relative positional bias (RPB) with SGD. Middle: RPB with AdamW. Right: rotary positional embedding (RoPE) with AdamW. Blue squares and green circles denote fixed-context and autoregressive (AR) training, respectively. Points show the mean transition time $\tau_{2\text{-Gen}}$ over three random seeds; error bars indicate the standard error of the mean. The dash-dotted and dashed curves in the left panel are fits of the form $\tau_0 + af(N)$, with $f(N) = N/\log N$ and $f(N) = N/(\log N)^2$, respectively. We use batch size $B = 64$ throughout. Learning rates are 0.5 for SGD and 10^{-3} for AdamW.

b. *Context-length dependence of the transition time*

The invariant scaling of β under the new architecture with RPB and SGD optimizer In the architecture of equation VIII29, the order parameter β is represented by

$$\beta = \frac{1}{C} \operatorname{tr} G, \text{ with } G = W_E^\mathsf{T} \left(W_V^{(1)}\right)^\mathsf{T} M_{21}^{(2)} W_E. \tag{VIII37}$$

where $M^{(2)} = (W_K^{(2)})^\mathsf{T} W_Q^{(2)}$. Then we want to prove that the N scaling relation in β dynamics shall not change as a product of trainable matrices.

Let $\mathcal{W}$ denote all parameters entering equation VIII37 and $W \in \mathcal{W}$ indicates each weight matrix. Retaining the matching contribution to their gradient-flow dynamics gives

$$\frac{dW_{ab}}{dt} \propto -\frac{\partial \mathcal{L}}{\partial \beta} \nabla_{W_{ab}} \beta, \qquad W \in \mathcal{W}. \tag{VIII38}$$

Then

$$\frac{d\beta}{dt} = \sum_{W \in \mathcal{W}} \sum_{a,b} \frac{\partial \beta}{\partial W_{ab}} \frac{dW_{ab}}{dt} \propto -\sum_{W \in \mathcal{W}} \sum_{a,b} \frac{\partial \mathcal{L}}{\partial \beta} (\nabla_{W_{ab}} \beta)^2. \tag{VIII39}$$

The response coefficient $\nabla_{W_{ab}} \beta$ depends on the matrix factors and their initialization, but has no explicit N-dependence. So we expect that our predictions from SA-transformer still hold for the larger architecture under the same setting (relative positional bias, SGD, fixed-context training), which is verified in Fig. A12, left panel.

Effects of autoregressive training The transition time τ_{2-Gen} for fixed-context training has been given in equation VIII27. AR training averages the loss over different positions. Applying the fixed-context matching bias in equation 7 at each length gives

$$\frac{d\beta}{dt} \propto \frac{1}{N}\sum_{n=1}^{N}\frac{H_n}{n} \approx \frac{1}{N}\int_1^N \frac{\log n}{n} dn = \frac{(\log N)^2}{N}. \tag{VIII40}$$

The corresponding transition-time scale is given by

$$\tau_{\text{2-Gen}}^{\text{AR}} \propto \frac{N}{(\log N)^2}. \tag{VIII41}$$

The RPB architecture autoregressively trained with SGD exhibits trends consistent with equation VIII41 as shown in Fig. A12, left panel.

Effects of AdamW optimizer AdamW normalizes the first gradient moment by the square root of the second moment. For AR training, the matching gradient scales as $(\log N)^2/N$. If this factor is the only dependence of the gradient history on N, it also multiplies the first moment, while its square multiplies the second moment. Neglecting the numerical stabilizer ε, these N-dependent factors cancel. Adaptive normalization can therefore remove the context-length dependence arising from the gradient amplitude. Consistent with this mechanism, AR training with RPB and AdamW exhibits an approximately constant transition time as shown in Fig. A12 in the middle panel. The fixed-context case still exhibits an N-dependence due to the finite sampling noise introduced by a small batch size of 64.

Effects of RoPE Our predicted scaling fails for RoPE (see Fig. A12, right panel). Because RoPE tunes weights underlying different frequency channels directly within the embedding space (see equation VIII34) —rather than operating in a separate vector space like RPB (see equation VIII33)—it introduces non-trivial dynamics not captured by our present theory.

c. *Initialization-dependent drift dynamics*

Equation VIII37 expresses β as a product of learned matrices. Its learning dynamics therefore depend on the initialization of these constituent factors, as illustrated by Saxe et al. [59] for deep linear networks. Intuitively, larger matrix factors can increase the effective response coefficient in equation VIII39, amplifying the change in β driven by a given loss gradient. The attention initialization scale σ can therefore change the growth rate of β.

In the large-σ regime, β grows substantially faster while retaining approximately linear growth, $\beta \propto t$. The term linear in β in the δ dynamics of equation VIII24 can therefore no longer be neglected, producing a quadratic contribution to δ in both control and bias-ablation conditions. The left two panels of Fig. A13 show qualitative agreement with these predictions.

In the small-σ regime, nonlinear effects become important, and the dynamics of both β and δ depart from the simple growth laws obtained from equation VIII24 and become nontrivial as shown in the right two panels of Fig. A13.

IX: The memorization circuit

1. Task vector intuition

In the 2-Mem circuit, MLP2 primarily reads from two inputs to produce the logit: MLP1 and Att2, the latter of which averages the outputs of MLP1 over the sequence. Consider the fluctuations of these inputs over sequences sampled from the same chain. MLP1 can only read the current state and the output of Att1, the latter of which is almost exclusively a representation of the previous state. Thus, the input from MLP1 depends only on local sequence information and is expected to exhibit large fluctuations across sequence positions and sampled sequences, even those generated by the same chain. In contrast, Att2 averages the local representations produced by MLP1, and we expect the large-N average to concentrate on a vector determined by the stationary 2-point distribution. Thus, the input from Att2 will exhibit small fluctuations across sequences from the same chain. With this intuition in mind, we propose that MLP2 infers the task identity using only the aggregated sequence information produced by Att2, and refer to that output on a sequence as a task vector φ. The remaining input from MLP1 provides MLP2 with the current state to condition the next-state predictions on.

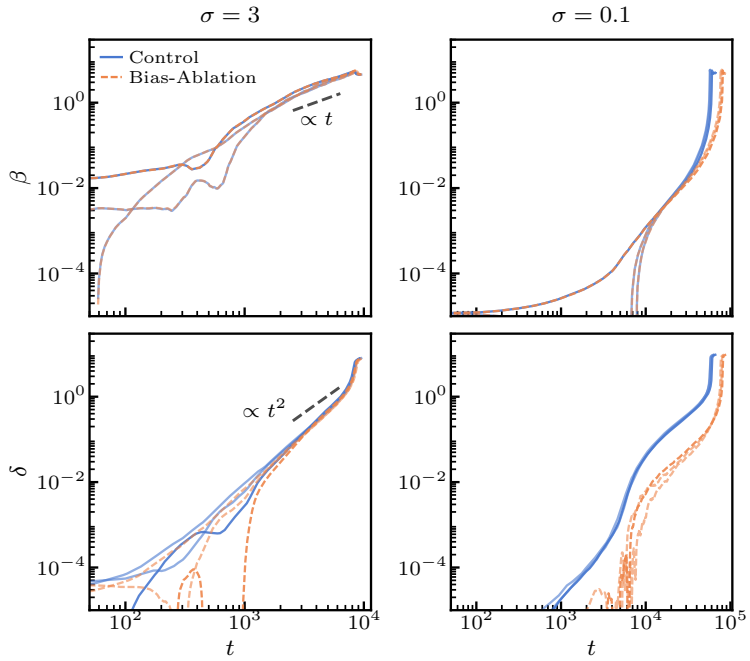

FIG. A13: Initialization-dependent drift dynamics beyond the SA-transformer. The two-layer attention architecture is the same as in Fig. A12 but trained only with RPB and fixed-context SGD at $N = 1024$, batch size $B = 256$, and learning rate 0.5. Columns correspond to attention-matrix and embedding matrix initialization scales $\sigma = 3$ (left) and $\sigma = 0.1$ (right). Rows show the effective matching parameter β (top) and the previous-state positional bias δ (bottom). Blue solid curves denote control training; orange dashed curves denote bias ablation. Each condition contains three trajectories from distinct initializations, paired between control and ablation.

2. Representation geometry

Since MLP2 infers the current task condition from φ, it is desirable for instantiations of φ to be separable when the underlying sequences are from different tasks. We computed t-SNE for φ and were able to observe task-specific clustering forming through training on data diversities up to $K = 128$. The task vectors for this computation are the cached outputs of Att2 at the last state across many sequences.

3. Task vector patching

We propose that MLP2 makes predictions using only the task information provided by φ, and test this by analyzing the transformer behavior when the φ task information is incongruent with any other alternative source of task information. The transformer first performed a forward pass on a batch of sequences $\mathcal{S}_A$ (Condition S_A). During the forward pass, the output of Att2 (i.e. the task vectors for each sequence and sequence position) was cached. The model then performed a second forward pass on a new batch of sequences $\mathcal{S}_B$ with the output of Att2 overwritten by the outputs observed on sequence $\mathcal{S}_A$ (Condition S_B (Patch A)). Consider two predictors for comparison with the model predictions. One is the 2-Mem predictor that makes predictions on $\mathcal{S}_A$ directly. The other is a 2-Mem predictor which computes the posterior distribution over transition matrices given the context in $\mathcal{S}_A$ but predicts the next-state distribution given the current state in $\mathcal{S}_B$. Denote the predictors first by the sequences used to compute the posterior and second by the sequences given as the current state; thus, the two predictors outlined are AA and AB respectively. We compared the model behavior to each predictor by averaging the divergence $D_{\mathrm{KL}}^{S_N}(\hat{\pi} \,||\, \hat{\pi}_\theta)$ (see Section IV 1) for each sequence in the batch, and converted to a similarity by normalizing this quantity by the divergence to a predictor that is uniform over the states and applying a negative exponential. We repeated the same procedure but with the roles of $\mathcal{S}_B$ and $\mathcal{S}_A$ swapped, in this case comparing to BB and BA predictors.

The similarity of a $K = 128$ transformer predictions in each condition to each predictor is shown in Figure 4b. In condition $\mathcal{S}_A$ and $\mathcal{S}_B$ the transformer is similar to predictors AA and BB respectively, as expected. In the Patch A (Patch B) conditions, the transformer is similar to predictor BA (AB). Thus, MLP2 algorithmically behaves like

a predictor that infers the posterior over tasks from the Att2 task vector φ and predicts the next-state distribution accordingly.

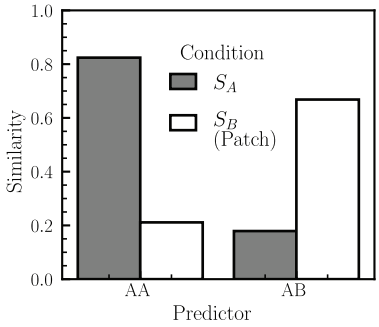

FIG. A14: The task vector patching experiment performed for M_1 by replacing the output of Att1 and comparing to the 1-Mem predictor.

We conduct the same patching experiment for M_1, but replace the output of Att1 and compare to the 1-Mem predictor instead. The similarities in each condition are shown in Figure A14 and indicate that the blocks computing the output prediction (Figure A5b) identify the transition matrix from the task vector alone. This task vector is likely localized to the vector passed from Att1 to MLP1 in the traced circuit.

4. Identifying encoded information

The task vector patching experiments (Appendix IX 3) establish that the next-state prediction is made given the posterior task distribution inferred from the task vector φ alone. However, this alone does not specify the information that is compressed into φ and used to infer the posterior. Specifically, φ could constitute either 1) a compressed representation of the posterior task distribution itself or 2) a compressed representation of the 2-point sequence statistics.

To differentiate between these two possibilities, we measured the ability for either 1) the true task identity or 2) the 2-point sequence statistics to be linearly decoded from the φ observed in a M_2 transformer. The task identity decoder $W_1 \in \mathbb{R}^{D\times K}$ was trained to minimize the cross-entropy classification loss for the latent chain identity on in-distribution sequences, averaged over sequence positions. The sequence statistics decoder $W_2 \in \mathbb{R}^{D\times C^2}$ was trained to minimize the KL-divergence from the sequence statistics on out-of-distribution sequences, restricted to observed transitions and averaged over sequence positions. We denote the final performance of each decoder for a given K by $\mathcal{L}_1^{\text{model}}$ and $\mathcal{L}_2^{\text{model}}$.

To establish baseline performance, we also trained decoders on task vectors produced by jointly optimized sequence embeddings. This baseline implementation follows the minimal model for 2-point memorization (Appendix XII), with the following modifications: the 2-point embedding function (Equation XII11) is implemented as a learned lookup table rather than an MLP, the initial state is embedded by a separate lookup table, we fix $D_\varphi = D$, and the MLP predictor is replaced by a linear decoder. Since the decoder and task vector are jointly optimized for either the task identification or sequence statistic objective, we consider the performance of these models as optimal for the corresponding objective, given the constraints of the minimal memorization model (which follow from the observed M_2 circuit, see Appendix XII for details). We denote the final performance of each baseline decoder by $\mathcal{L}_1^{\text{optimal}}$ and $\mathcal{L}_2^{\text{optimal}}$.

The objective of encoding either the task posterior or sequence statistics are similar, so optimizing for one may result in a task vector that contains a significant amount of information about the other. To account for this, we further trained cross-objective decoders which receive frozen task vectors produced by the final baseline model for the opposite objective. We denote the final performance of each cross-objective decoder by $\mathcal{L}_1^{\text{cross}}$ and $\mathcal{L}_2^{\text{cross}}$, where the subscript indicates the decoding objective.

The above decoder models were trained for 20k iterations of minibatch gradient descent with the Adam optimizer and batch size of 128 sequences. Models at the same K share the same training task distribution. Transformer models were trained for 10^5 iterations following the procedure outlined in Appendix II 3.

Figure A15, shows a comparison of the rescaled relative loss

$$\tilde{\mathcal{L}}_i \equiv \frac{\mathcal{L}_i^{\text{model}} - \mathcal{L}_i^{\text{baseline}}}{\mathcal{L}_i^{\text{cross}} - \mathcal{L}_i^{\text{baseline}}} \tag{IX1}$$

for each objective across a range of D and K. By construction, $\tilde{\mathcal{L}}_i < 1$ indicates that the transformer task vector contains more information for the corresponding objective than expected under optimization for the opposite objective. Across the considered D and K values, $\tilde{\mathcal{L}}_2 > 1$ and the transformer is not observed to be optimizing the task vector to encode the 2-point sequence statistics. Instead, $\tilde{\mathcal{L}}_1 < 1$ for sufficiently small K for each D. Thus, we conclude that the M_2 task vector is constructed to represent the posterior task distribution.

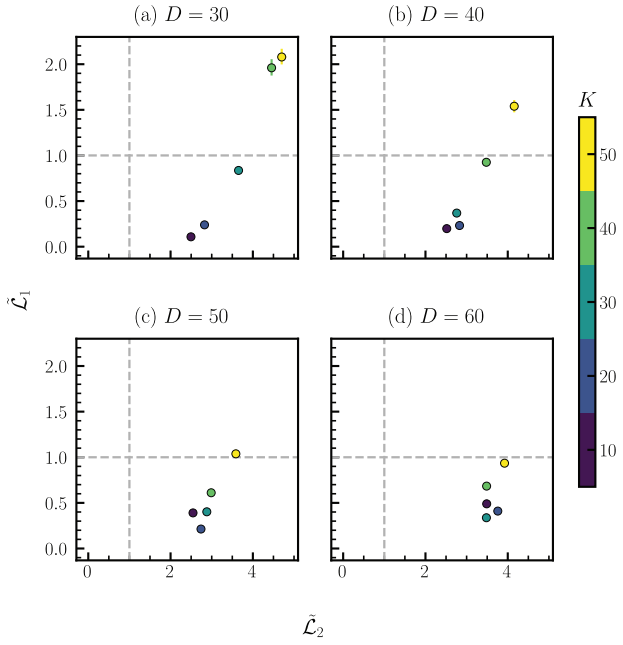

FIG. A15: Comparison of linear decoding performance from the M_2 task vector of task identity ($\tilde{\mathcal{L}}_1$) and 2-point sequence statistics ($\tilde{\mathcal{L}}_2$), varying D and K. Values of $\tilde{\mathcal{L}} < 1$ indicate significantly strong decoding of the corresponding information; dashed lines indicate this threshold. Models only ever show significant encoding of task identity. Errorbars correspond to paired-bootstrapped 95% confidence intervals from evaluations on $32 \times K$ and 2^{13} sequences for $\tilde{\mathcal{L}}_1$ and $\tilde{\mathcal{L}}_2$ respectively.

5. Measuring task information content

In M_2, the first layer attention weight concentrates on the previous state $A^{(1)}_{ni} \approx \delta(n-i-1)$, in which case Att1 may be seen as forming a representation of 2-point statistics in the residual stream $x^{(1)}_n = x_n + W^{(1)}_V x_{n-1} \equiv f(x_n, x_{n-1})$. The traced circuit indicates that MLP1 reads these pairs and produces an output vector for each that is then aggregated across the sequence by Att2 to form the task vector φ. Indeed, MLP1 is a requirement for the 2-Mem circuit because of a general limitation of linear 2-point embeddings, which the attention operation is restricted to perform. For any linear embedding function f_{lin}, averaging the embeddings along the sequence $\langle f_{\text{lin}}(x_i, x_{i-1})\rangle = f_{\text{lin}}(\langle x_i\rangle, \langle x_{i-1}\rangle)$ destroys the bigram information. As a result, the second attention layer must average a nonlinear bigram embedding computed by MLP1.

In what follows, we consider MLP1 to be equivalent to a non-linear embedding function $\lambda(s_i, s_{i-1})$ of neighboring pairs of states (s_i, s_{i-1}) and approximate the task vector φ_n at position n as an average of these embeddings

$$\varphi_n \approx \frac{1}{n}\sum_{i\le n} \lambda(s_i, s_{i-1}). \tag{IX2}$$

The embedding function acts on state pairs $(\tau, \mu) \in C \times C$ and forms an embedding matrix $\mathcal{E} \in \mathbb{R}^{D\times C^2}$. Additionally, each chain k defines a distribution over the pairs $P^{(k)}(\mu, \tau) = p^{(k)}_\mu T^{(k)}_{\tau\mu}$. Write the vector of 2-point probabilities for chain k as $p^{(k)} \in \Delta^{C^2-1}$.

We would like to analyze the optimality of a given set of embeddings. Specifically, we estimate the mutual information between the task vector as in equation IX2 and the chain identity $I(K;\varphi) = H(\varphi) - H(\varphi \mid K)$. Substituting the uniform prior over chains $p(k) = \frac{1}{K}$ into each term of this expression gives

$$H(\varphi) = \log K - \frac{1}{K} \int d^D\varphi \left(\sum_k p(\varphi \mid k) \right) \log \sum_k p(\varphi \mid k) \tag{IX3}$$

$$H(\varphi \mid K) = -\frac{1}{K} \int d^D\varphi \left(\sum_k p(\varphi \mid k) \log p(\varphi \mid k) \right) \tag{IX4}$$

and the distribution $p(\varphi \mid k)$ remains to be determined. To do so, note that the task vector resulting from a length $N+1$ sequence may be written as a sum over transitions

$$\varphi = \frac{1}{N} \mathcal{E} \boldsymbol{n}, \tag{IX5}$$

where $\boldsymbol{n} \in \mathbb{N}^{C^2}$ is the vector of counts for state pairs (τ, μ) in the sequence. Fixing a chain k, the vector of counts $\boldsymbol{n}^{(k)}$ is multinomially distributed across different sequences with pair probabilities $p^{(k)}$. As a result, the task vector is a random variable $\varphi^{(k)} \sim p(\varphi \mid k)$ with mean and covariance

$$\mathbb{E}(\varphi^{(k)}) = \frac{1}{N} \mathcal{E}\, \mathbb{E}(\boldsymbol{n}^{(k)}) = \mathcal{E} p^k, \tag{IX6}$$

$$\mathrm{Cov}(\varphi^{(k)}) = \frac{1}{N^2} \mathcal{E}\, \mathrm{Cov}(\boldsymbol{n}^{(k)})\, \mathcal{E}^T = \frac{1}{N} \mathcal{E} \left(\mathrm{diag}(p^k) - p^k p^{k\top} \right) \mathcal{E}^\top. \tag{IX7}$$

We assume N is sufficiently large such that $p(\varphi \mid k)$ is well-approximated by a D-dimensional Gaussian with mean and covariance given above. To estimate the mutual information, the integration in equation IX3 is approximated via Monte-Carlo sampling (1000 samples) from the Gaussian mixture, and equation IX4 is evaluated directly from the entropy of a Gaussian.

For a given model, we estimate the embedding function λ computed by MLP1 by caching the output of the layer at the last state of 2-state sequences representing each possible pair. The information content is estimated for a sequence length of $N = 128$ (i.e., midway through the maximum sequence length) and for embeddings projected down to a PC space that preserves 90% of the variance. This space was found to have dimension ≤ 32 at all checkpoints for the K range shown in Figure 4c. We omit the $K = 64$ setting for clarity, which displays an anomalous reduction in estimated information late in training due to a falloff of the PC space dimension that is likely an artifact of the method for estimating the embedding function.

X: Memorization to generalization transition

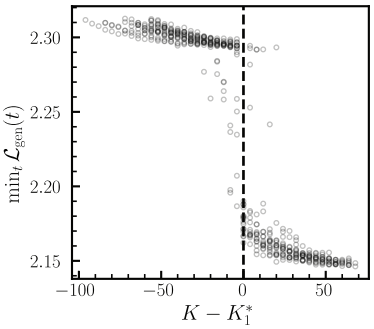

FIG. A16: The minimum achieved value of the generalization loss $\mathcal{L}_{\mathrm{gen}}$ drops sharply as the data diversity crosses over K_1^*. As in Figure 7a, each seed has been shifted along the horizontal axis by the value of K_1^* determined for that seed by the $\phi_\beta^{(2)}$ threshold criteria. Results derived using the same models used for Figure 7a.

We find a critical data diversity K_1^* below which the model transitions from G_1 to M_1 and above which the model instead abruptly transitions to G_2. Our findings (detailed below) indicate a kinetic mechanism for this transition.

We specify the transition point K_1^* as follows. The induction-head order parameter ϕ_β is nonzero only in G_2, and its maximum value over training is approximately sigmoidal as the data diversity crosses through K_1^* with the inflection point occurring approximately when $\phi_\beta = \phi_\beta^* = 0.45$. Given these observations, we consider a transformer to be in G_2 when $\phi_\beta > \phi_\beta^*$. We compute K_1^* for a transformer by performing a binary search in the data diversity with the condition that $K > K_1^*$ if $\max \phi_\beta(t) > \phi_\beta^*$ over 10^4 training iterations. To check the validity of this criteria, we measure the minimum value of the generalization loss $\mathcal{L}_{\text{gen}}$ achieved over training as the data diversity crosses over K_1^* and find that the minimum $\mathcal{L}_{\text{gen}}$ drops sharply as the data diversity crosses K_1^* (Figure A16).

The kinetic mechanism driving K_1^* can be understood from a view of the transformer as constituting a mixture-of-experts. These experts are realized as subcircuits implementing 1-Mem and 2-Gen that develop in parallel in the transformer, and which have weighted contributions to the next-state prediction. The subcircuits are in competition in the sense that improved performance of one circuit reduces the gradient driving the development of the other. In such a case, one would expect the transition to be highly sensitive to the learning kinetics of the two subcircuits. To this end, we developed two experiments to independently perturb the kinetics of each subcircuit.

1. Task injection perturbation

To implement a memorizing predictor, the model must in part learn to differentiate between sequences from different tasks. The rate at which the model learns this can be increased by providing perfect task information to the model. To do this, we allowed the model to learn D-dimensional embeddings for each chain that were injected into MLP2 via addition to the original input vector to indicate the ground truth generating chain. This task injection perturbation is seen to increase the value of K_1^* for the model by a factor ~ 10 (Figure 7c).

2. Gradient reweighting perturbation

We sought to perturb the model by modifying the effective learning rate for the 2-Gen circuit independent of the 1-Mem circuit. One way to accomplish this is by reducing the rate at which the first attention layer learns to attend to the previous state. We implemented a modified attention head in Python with a fixed weight factor $w \in [0, 1]$ such that the previous-state attention weights include a term that does not contribute to the gradient

$$A^{(1)}_{i,i-1} = w \cdot A'^{(1)}_{i,i-1} + (1-w) \cdot \text{detach}\left(A'^{(1)}_{i,i-1}\right) \quad \text{for } i \in [2, N] \tag{X1}$$

where detach$(\cdot)$ denotes detachment from the computational graph used for automatic differentiation. As a result, gradients of these attention weights during backpropagation are reduced by a factor of w while the attention weights used for the forward pass are unmodified. Using this method to decrease the previous-state attention weights' learning rate by a factor of $w = 1/10$ relative to that of the rest of the model is seen to increase K_1^* by a factor ~ 2 (Figure 7c). During the numerical simulations performed to obtain this result, we increased the total training iterations by a factor of $1/w$ to broadly account for the slower dynamics.

To check the specificity of each perturbation's effect on the target circuit kinetics, we compare the learning dynamics at small and large K (Figure A17).

XI: Transient generalization and partial memorization

We hypothesize the existence of a critical data diversity K_2^*, above which the model will converge to G_2 and never enter M_2. In an attempt to estimate this transition point, we measured the length of the interval $\Delta\tau_K \equiv \tau_{\text{2-Mem}} - \tau_{\text{2-Gen}}$ between the sharp drop in the loss at the onset of G_2 and the first crossing of the model loss with the 2-Gen predictor $\mathcal{L}_{\text{train}} < \mathcal{L}^{\text{2-Gen}}$. This quantity is observed to diverge as $(K_2^* - K)^{-\gamma}$ with $K_2^* \approx 7000$ and $\gamma \approx 2$ (Figure 8a).

To develop a complementary picture of K_2^*, we investigated the nature of the model memorization at large data diversities. To do so, we computed the average loss difference between the model and the 2-Gen predictor on sequences drawn from each chain. This quantity forms a per-task memorization score

$$\Delta^k_{\text{2-Mem}} = \left\langle \mathcal{L}(\hat{\pi}_\theta, S_{N+1}) - \mathcal{L}\left(\hat{\pi}^{\text{2-Gen}}, S_{N+1}\right) \right\rangle_{S_{N+1} \sim T^{(k)}} \tag{XI1}$$

where $\mathcal{L}(\hat{\pi}, S_{N+1})$ is the autoregressive cross-entropy loss of predictor $\hat{\pi}$ on sequence S_{N+1}. Thus, a negative score $\Delta^k_{\text{2-Mem}} < 0$ indicates that the model outperforms the 2-Gen predictor on sequences from task k. We estimated $\Delta^k_{\text{2-Mem}}$ at each training checkpoint by evaluating the model and 2-Gen predictor on a batch of $64 \times K$ training

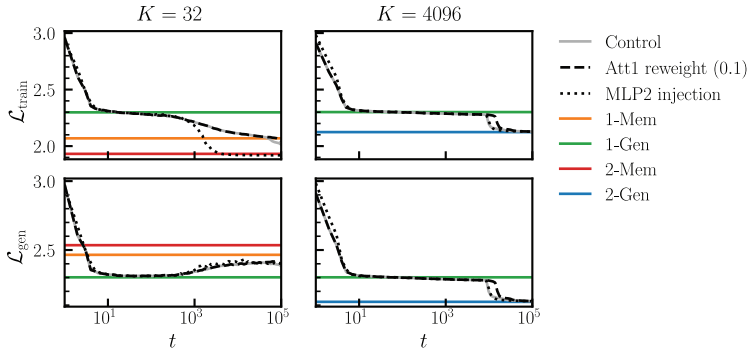

FIG. A17: The kinetic perturbations show the desired effect on the target circuit, and little off-target effect. Specifically, the task injection perturbation speeds up learning of 1-Mem for small $K < K_1^*$ while learning of 2-Gen at large $K \gg K_1^*$ remains unaffected. Likewise, the gradient reweighting perturbation delays learning of 2-Gen while learning of 1-Mem is not affected. Dynamics are shown for vanilla SGD.

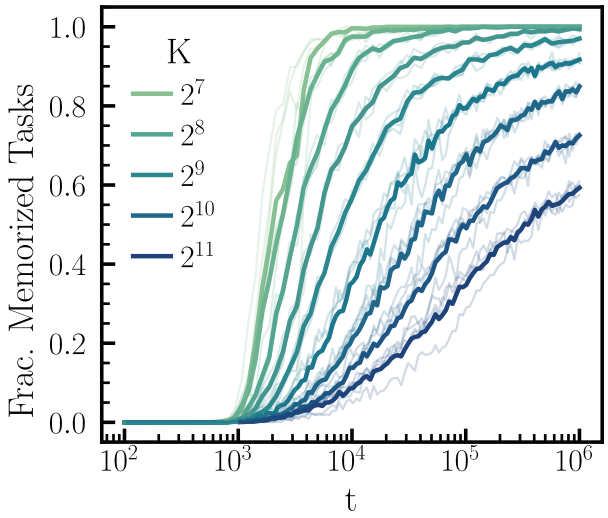

FIG. A18: The fraction of memorized tasks displays sigmoidal training dynamics across data diversities. Individual results for 4 seeds are plotted as thin traces, and the mean is plotted by a thick line.

sequences. The proportion of tasks with $\Delta^k_{\text{2-Mem}} < \epsilon$ for small ϵ (detailed below) shows sigmoidal training dynamics across a range of K (Figure A18), and we show in Figure 8(b) the asymptotic values of sigmoid functions fit to these dynamics. These data reveal that for data diversities $K \gtrsim 2^8$ the model can only memorize a subset of the training chains, and a linear extrapolation of these data suggests that the model will not memorize any chains for $K \gtrsim 5000$. This result is consistent with the estimate for K_2^* obtained from the diverging memorization time in the previous paragraph.

The model in G_2 may exhibit per-chain over-performance relative to the 2-Gen predictor due to biases in the model predictions. To account for this possibility, we train 3 models at $K = \infty$ for 10^6 iterations and evaluate the memorization score $\Delta^k_{\text{2-Mem}}$ of the final checkpoint on 10^4 randomly fixed tasks. The distribution of scores for each model each have mean and standard deviation $\approx 10^{-3}$, indicating that typical fluctuations of $\Delta^k_{\text{2-Mem}}$ are $\sim 10^{-3}$ for a G_2 model. Therefore, we require that $\Delta^k_{\text{2-Mem}} < \epsilon$ with $\epsilon = -10^{-3}$ for a chain to be considered memorized. Removing the threshold by setting $\epsilon = 0$ modifies the slope of the decay in Figure 8(b) but leaves the qualitative features unchanged.

XII: Minimal model for 2-point memorization

1. Path expansion in M_2

The circuit-tracing results for M_2 in Figure A4 indicate that the full transformer computation can be reduced to the dominant information flow shown schematically in Figure 8(c). This reduced path can be written as

$$x_n^{(0)} = W_E x_n, \tag{XII1}$$
$$y_n^{(1)} = x_n^{(0)} + \mathrm{Att}^{(1)}\left(x_{\le n}^{(0)}\right), \tag{XII2}$$
$$x_n^{(1)} = x_n^{(0)} + \mathrm{MLP}^{(1)}\left(y_n^{(1)}\right), \tag{XII3}$$
$$y_n^{(2)} = \mathrm{MLP}^{(1)}\left(y_n^{(1)}\right) + \mathrm{Att}^{(2)}\left(x_{\le n}^{(1)}\right), \tag{XII4}$$
$$x_n^{(2)} = \mathrm{MLP}^{(2)}\left(y_n^{(2)}\right), \tag{XII5}$$
$$x_n^{(3)} = \mathrm{Linear}\left(x_n^{(2)}\right). \tag{XII6}$$

Our mechanistic hypothesis may be summarized as follows. Att1 extracts the preceding state and MLP1 embeds the ordered pair (x_{n-1}, x_n) into a high-dimensional representation that distinguishes different tasks. Next, Att2 pools these pair embeddings across the sequence to form a task vector. Finally, MLP2 combines this task vector with the current-state information encoded by MLP1 to retrieve the transition probabilities conditioned on the current state. Under this hypothesis, the computation can be further simplified without loss of essential functionality.

2. Minimal network construction

First, we assume that Att1 extracts the preceding state and maps it to a subspace orthogonal to that of the current state. This allows us to replace

$$y_n^{(1)} = x_n^{(0)} + \mathrm{Att}^{(1)}\left(x_{\le n}^{(0)}\right) \longrightarrow y_n^{(1)} = x_n^{(0)} \oplus x_{n-1}^{(0)}, \tag{XII7}$$

where $\oplus$ denotes concatenation into orthogonal subspaces.

Second, we assume that Att2 performs a uniform pooling operation over the pair embeddings produced by MLP1, while the current-state information is passed directly to MLP2. This yields

$$y_n^{(2)} = \mathrm{MLP}^{(1)}\left(y_n^{(1)}\right) + \mathrm{Att}^{(2)}\left(x_{\le n}^{(1)}\right) \longrightarrow x_n^{(0)} \oplus \left\langle \mathrm{MLP}^{(1)}\left(x_{i+1}^{(0)} \oplus x_i^{(0)}\right)\right\rangle_{i<n}. \tag{XII8}$$

Combining these assumptions, the resulting minimal network is

$$x_n^{(0)} = W_E x_n, \tag{XII9}$$
$$y_n^{(1)} = x_n^{(0)} \oplus x_{n-1}^{(0)}, \tag{XII10}$$
$$x_n^{(1)} = \mathrm{MLP}^{(1)}\left(y_n^{(1)}\right), \tag{XII11}$$
$$y_n^{(2)} = x_n^{(0)} \oplus \left\langle x_i^{(1)}\right\rangle_{i\le n}, \tag{XII12}$$
$$x_n^{(2)} = \mathrm{MLP}^{(2)}\left(y_n^{(2)}\right), \tag{XII13}$$

which is equivalent to the architecture depicted in Figure 8c upon the replacement of the two attention circuits by their corresponding functional forms. In the last step, the final linear readout has been absorbed into MLP2, which now directly outputs the C-dimensional logit. For this minimal model, W_E is initialized randomly from a Gaussian distribution and kept fixed throughout training.

Relative to the full transformer, the minimal model introduces a single additional parameter: the dimension D_φ of the task vector

$$\varphi = \frac{1}{n-1}\sum_{i=2}^{n}\left\langle x_{i-1}^{(0)} \oplus x_i^{(0)}\right\rangle, \tag{XII14}$$

where $\varphi \in \mathbb{R}^{D_\varphi}$.

Unless otherwise stated, the reference model discussed in this section has an MLP2 with two hidden layers, in a departure from the full transformer model. We use autoregressive training for the minimal model with the maximum sequence length $N = 256$ to match the training process described in Appendix II 3.

3. Phase diagram

Figures 8d and 8f show that the minimal model training loss can fall below the 2-Gen baseline $\mathcal{L}^{\text{2-Gen}}$ while the generalization loss remains significantly above it. This separation demonstrates that the minimal network successfully reproduces the 2-Mem predictor.

As data diversity K increases, Figure 8d exhibits a clear crossover behavior: for small K, the minimal network outperforms the 2-Gen predictor, whereas for sufficiently large K it does not. Because the minimal network does not implement the induction-head mechanism required for 2-Gen prediction (which is present in the full transformer), this crossover is consistent with the existence of a critical data diversity K_2^* separating M_2 and G_2 in the full model. This agreement suggests that the minimal network captures the essential structure of the M_2 transformer and can therefore serve as a useful framework for analyzing the mechanisms underlying 2-Mem prediction.

We further observe that MLP2 can implement the 2-Gen prediction in certain regimes. In the limit $K \to \infty$, both training and generalization losses approach the 2-Gen baseline, provided that the task-vector dimension D_φ is sufficiently large. A large $D_\varphi \sim C^2$ is required to encode the complete empirical transition matrix of the sequence in one vector.

Indeed, if we restrict the second attention block in the full transformer model to uniform attention, we observe the usual four-phase picture when the model dimension D becomes sufficiently large (Figure A19).

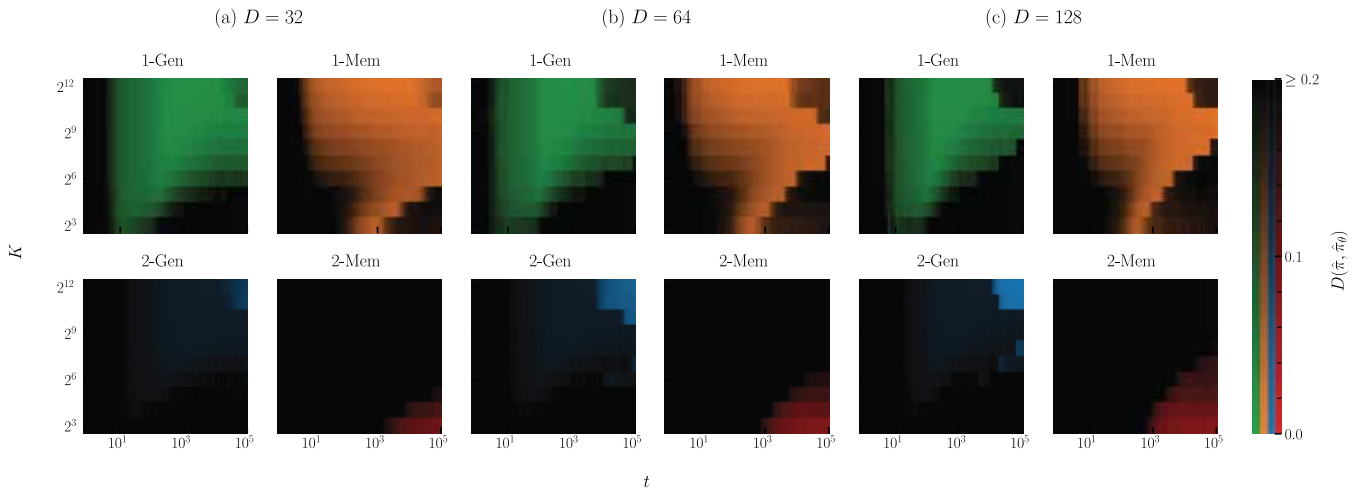

FIG. A19: Behavioral readouts of the full transformer trained with the second attention block restricted to uniform attention. As the model dimension D crosses over C^2 in panel (c), the full four phases appear. Note that these diagrams include up to $K = 2^{12}$.

4. Dependence of $\hat{K}_2^*$ on encoder and decoder capacity

We denote the estimated critical data diversity for the minimal model by $\hat{K}_2^*$ to distinguish it from K_2^* in the full model.

We hypothesize that MLP1 is primarily responsible for extracting a task vector from the sequence, while MLP2 must memorize the collection of task-specific transition matrices and select the appropriate one conditioned on the task vector. Consequently, the capacities of both MLPs can act as bottlenecks and influence the critical data diversity $\hat{K}_2^*$.

In Figure 8(e), we vary the depths of MLP1 and MLP2 to have one or two hidden layers while keeping their widths

fixed. For each architecture, we estimate $\hat{K}_2^*$ via binary search using the threshold

$$\left\langle \mathcal{L}_{\text{train}} - \mathcal{L}^{\text{2-Gen}} \right\rangle_{1000} < 0, \tag{XII15}$$

where the average is taken over 1000 iterations with a 4×10^5-iteration run.

We find that $\hat{K}_2^*$ depends only weakly on the depth of MLP1. For a fixed MLP2 depth, increasing MLP1 from one to two layers leaves $\hat{K}_2^*$ essentially unchanged (see Figure 8e). In contrast, increasing the depth of MLP2 substantially enlarges $\hat{K}_2^*$, raising it from $\sim 10^2$ to $\sim 10^3$-10^4. This indicates that memorization capacity in MLP2 is the primary bottleneck for sustaining M_2 at large K. This behavior aligns with the intuition that constructing pair embeddings (the role of MLP1) is simple compared to retrieving the transition matrix given the chain identity (the role of MLP2), which evidently demands significantly greater representational capacity. We also observe a dependence on the embedding dimension, with $\hat{K}_2^*$ increasing more rapidly with D_φ for $D_\varphi \gtrsim 60$. The scaling between the capacity of the neural network and the embedding dimension is thus non-trivial.